

IMPERIAL

ADAPTIVE ACQUISITION: A REINFORCEMENT LEARNING FRAMEWORK FOR AUTONOMOUS MRI SEQUENCE OPTIMIZATION

Author

A. ALLILAIRE

CID: 02372975

Supervised by

DR ANDREAS WETSCHEREK

PROF. WAYNE LUK

Second Marker

Prof. Ben Glocker

An Interim Report submitted in fulfillment of requirements for the degree of
Bachelor of Engineering in Computing

Department of Computing
Imperial College London
2026

Acknowledgments

I would like to thank my supervisor at the Institute of Cancer Research, Dr Andreas Wetscherek, for his continued support throughout this project: for helping shape its direction, for help analysing results and debugging anomalies, and for patiently explaining the MRI physics that underpins the work.

I am also grateful to my academic supervisor at Imperial College London, Professor Wayne Luk, for his pertinent feedback on the report and his advice throughout.

Contents

Acknowledgments	i
1 Introduction	1
1.1 Introduction	1
1.2 Research challenges and achievements	2
2 Background & Related Work	3
2.1 Principles of Magnetic Resonance Imaging	3
2.1.1 RF Pulses	4
2.1.2 T1, T2 & T2* Relaxation	4
2.1.3 Spatial Encoding & K-Space	6
2.1.4 Gibbs (Truncation) Ringing	7
2.1.5 Pulse Sequences	8
2.1.6 MRI Simulators	8
2.1.7 Magnetic Resonance Fingerprinting (MRF)	8
2.2 Digital twins and phantoms for quantitative MRI	9
2.3 Magnetic Resonance Linear Accelerator (MR-LINAC)	9
2.4 Deep Learning in MRI	10
2.4.1 Image Acquisition and Reconstruction using Deep Learning	10
2.4.2 Pulse sequence design	10
2.4.3 Adaptive MRI acquisition	11
2.4.4 Reinforcement Learning and PPO for Adaptive Acquisition	12
2.5 Summary	13
3 MRISystemPhantom.jl — Digital Twin Library	15
3.1 Chapter aim	15
3.2 The physical phantom	15
3.3 Related phantom tooling	16
3.4 Design goals and architecture	17
3.5 Geometry	18
3.5.1 Sphere voxelisation	18
3.5.2 Slicing	18

3.5.3	Pose transforms	18
3.5.4	Background water	18
3.6	Material tables	20
3.7	Augmentation	21
3.7.1	Episode-level random phantoms	22
3.8	Shipped sequences and fitting models	22
3.8.1	The Cramér–Rao baselines	23
3.9	KomaMRI integration	23
3.10	Library availability	24
3.11	Limitations and future work	24
3.12	Summary	25
4	Validating KomaMRI	27
4.1	Chapter aim	27
4.2	KomaMRI simulation model	28
4.3	Initial validation failure	28
4.4	RF edge marker collapse	29
4.5	Closing knot collapse after time rebasing	32
4.6	Validation after the fixes	32
4.7	Evaluation of the fixes	33
4.7.1	Why this mattered for qMRI	34
4.7.2	Why spoiling was a convincing false lead	34
4.8	Limitations and future work	35
4.9	Summary	35
5	Adaptive qMRI Sequence Design with Reinforcement Learning	37
5.1	Chapter aim	37
5.2	Position relative to adaptive MRI work	38
5.3	Environment formulation	39
5.4	Multi-fidelity curriculum	42
5.4.1	The cost wall and the fidelity ladder	42
5.4.2	Cached water: exploiting simulator linearity	43
5.4.3	The switch rule: bias-aware promotion	44
5.4.4	Stage hand-off and global-best checkpointing	45
5.5	Run A: full 14-sphere T1 plate	45

5.6	Run B: five-sphere continuous-T1 task	46
5.7	Memory ablation: what should the policy remember?	48
5.8	Quantitative positioning against published results	50
5.9	Limitations	51
5.10	Future work	52
5.11	Summary	54
Summary		55
5.12	Achievements	55
5.13	Limitations	55
5.14	Future work	56
5.15	Concluding remarks	56
Bibliography		59
Declarations		63
A Technical Derivation Notes		65
A.1	Spin-Echo T2 Fit	65
A.2	Inversion-Recovery T1 Fit	65
A.3	Finite-TR and Transient IR Models	66
A.4	Joint T1/T2 Identifiability	66
A.5	Finite K-space Sampling and Truncation Artefacts	66
A.5.1	Why Reconstruction Blurs and Rings	66
A.5.2	Increasing the Matrix Does Not Fix It	68
A.5.3	Apodisation: The Hamming Window	68
A.6	E2 Noise Calibration	69
A.7	Cached Background-Water Validation	71
A.8	Reinforcement Learning Background	73
A.8.1	Policy gradients	73
A.8.2	Value and advantage functions	74
A.8.3	From TRPO to the PPO objective	75

Introduction

Contents

1.1 Introduction	1
1.2 Research challenges and achievements	2

1.1 Introduction

In 2021, there were 395,000 cases of cancer in the UK, with 1 in 2 people in the UK being diagnosed with cancer in their lifetime. [1] Radiotherapy (RT) is a crucial component of cancer treatment; indeed, it is required for more than 50% of cancer patients. [2] Image-guided radiation therapy (IGRT) is used to increase the precision of treatment and ensure accurate dose delivery. Conventionally, images are taken using a high-resolution X-ray technique called Cone Beam Computed Tomography. This is still the most common technique. [3]

However, poor image quality (caused by excessive scattering of X-ray photons) and ionising radiation are key drawbacks. Instead, Magnetic Resonance Imaging (MRI) can be used to improve soft tissue contrast (vital for liver or pancreatic cancer) and increase the accuracy of tumor target delineation. MR-guided radiation therapy (MRgRT) has been shown to significantly improve patient prognosis. [3]

MRI and a linear accelerator (the source of radiation) have been coupled into a hybrid Magnetic Resonance Accelerator (MR-Linac), the latest innovation in IGRT. The MR-Linac allows on-table adaptation of treatment plans, to respond to tumour movements and progression. However, one of the clinical bottlenecks is the extra time required for image acquisition. [4] The high-resolution images needed for accurate delineation require extended scan time. This also increases the risk of motion artifacts (inaccuracies due to patient movement).

There has been significant research into accelerating image acquisition, focusing mainly on using Deep Learning (computational neural network approaches) for image reconstruction (translating the raw MR signal into an image), leading to significant improvements to image robustness, artifact reduction and image quality when using undersampled data. [5] DL and Reinforcement Learning (agent-based approaches) have also been used to develop novel pulse sequences, traditionally designed by MR experts, showing promise in speeding up acquisition while maintaining image quality.[6], [7]

There is now growing interest in adaptive MR, a personalised approach in which "incoming subject data is used to update and fine-tune the pulse sequence parameters in real time"[8], rather than fixing all measurement parameters in advance. To date this has been realised with a Bayesian framework, which maintains a running estimate of the target tissue parameter and uses it to choose the next acquisition, accelerating quantitative measurement by up to $2.5\times$ over static protocols.[8]

This project brings reinforcement learning to that adaptive setting for quantitative MRI. Rather than using a static sequence, an agent observes the current fitted tissue-parameter estimates and chooses the next acquisition. Unlike the Bayesian framework above, the policy is learned end-to-end rather than derived from a model. Training such a policy directly on scanner hardware would be impractical, so the

project first builds and validates the simulation infrastructure: a digital twin of the phantom, a strict end-to-end test of the Bloch simulator, and a multi-fidelity training environment, before evaluating the adaptive policy itself.

1.2 Research challenges and achievements

Realising this learned, adaptive approach exposes three practical challenges.

C1: simulator validation for qMRI. The RL agent learns entirely from simulated MRI data, so the simulator must be numerically trustworthy before any learned policy can be evaluated. A visually plausible image is not sufficient: for qMRI, the pipeline must recover the correct quantitative tissue parameters, such as T_1 and T_2 , from a known phantom.

C2: computational efficiency under expensive simulation. A live Bloch-equation simulator is much slower than the analytical environments normally used in reinforcement learning. This makes training compute-bound. The project therefore needs a multi-fidelity strategy: fast, lower-fidelity configurations for learning, with more accurate simulations reserved for validation.

C3: adaptive sequence design from tissue-parameter estimates. The final objective is not simply to run a fixed protocol faster, but to choose acquisitions based on the current estimate of the tissue parameters. The agent should use the signals observed so far to decide which inversion times, echo times, or flip angles are most informative next.

The report is organised around three corresponding achievements, one per technical chapter.

A1: a digital twin of the calibration phantom (Chapter 3). The project built `MRISystemPhantom.jl`, a configurable open-source digital twin of the QalibreMD Model 130 MRI system phantom that returns standard KomaMRI objects. It also supplies the ground-truth tissue parameters and fitting pipeline needed to recover quantitative maps, together with the per-episode randomisation the RL loop requires. This is the foundation for C1: a known phantom against which quantitative maps can be checked.

A2: a validated Bloch-simulation pipeline (Chapter 4). Validating the simulator by parameter recovery rather than qualitative imaging exposed two floating-point time-discretisation bugs in KomaMRI, visible only at the long cumulative sequence times RL training produces. Reducing each to a minimal reproducer, tracing its mechanism, and fixing it upstream took the no-noise validation from 39.4% to 0.48% mean error, completing C1 and contributing back to a widely used simulator.

A3: adaptive qMRI sequence design via multi-fidelity RL (Chapter 5). The project built a Python/Julia Gymnasium environment in which a reinforcement-learning agent chooses acquisition timings from the current tissue-parameter estimates. A multi-fidelity training curriculum makes the expensive Bloch-in-the-loop problem tractable (addressing C2). On a five-sphere active pool the best learned policy reaches 2.93% mean fitted- T_1 error, beating the matched fixed schedule by roughly three percentage points with 95.8% of episodes under 5% (addressing C3); the adaptive advantage holds over this narrowed T_1 range, while the full fourteen-sphere ladder remains too wide for the policy.

The core novelty is, to our knowledge, the first reinforcement-learning agent for adaptive quantitative MRI: a policy that conditions each acquisition on the current fitted tissue parameters and is trained and evaluated directly on fitted- T_1 error. Prior adaptive qMRI derives its acquisition rule from a Bayesian model, and prior RL in MRI targets non-quantitative objectives such as shape classification or k-space sampling. Two further contributions support it: a reusable executable twin of a calibrated phantom, and the validation-by-parameter-recovery that surfaced and fixed two upstream KomaMRI bugs. Chapter 2 positions each against the corresponding prior work.

Background & Related Work

Contents

2.1 Principles of Magnetic Resonance Imaging	3
2.1.1 RF Pulses	4
2.1.2 T1, T2 & T2* Relaxation	4
2.1.3 Spatial Encoding & K-Space	6
2.1.4 Gibbs (Truncation) Ringing	7
2.1.5 Pulse Sequences	8
2.1.6 MRI Simulators	8
2.1.7 Magnetic Resonance Fingerprinting (MRF)	8
2.2 Digital twins and phantoms for quantitative MRI	9
2.3 Magnetic Resonance Linear Accelerator (MR-LINAC)	9
2.4 Deep Learning in MRI	10
2.4.1 Image Acquisition and Reconstruction using Deep Learning	10
2.4.2 Pulse sequence design	10
2.4.3 Adaptive MRI acquisition	11
2.4.4 Reinforcement Learning and PPO for Adaptive Acquisition	12
2.5 Summary	13

It is necessary to understand the current process of MRI Pulse Sequence design before attempting to use DL or RL to improve upon it. This requires an understanding of MRI-Physics; a short introduction follows.

2.1 Principles of Magnetic Resonance Imaging

The signal generated by an MRI uses the Nuclear Magnetic Resonance (NMR) of protons. All subatomic particles have intrinsic angular momentum, which is a quantum property called spin. For example, a proton has a spin- $\frac{1}{2}$, while a neutron has a spin $\frac{1}{2}$. For a nucleus to have a net spin, it cannot have the same number of neutrons and protons.

A Hydrogen nucleus is made of a single proton, so it has a net spin- $\frac{1}{2}$. The angular momentum, combined with the mass and charge of a proton, results in a magnetic moment μ . When a uniform magnetic field (B_0) is applied, protons will align their magnetic axis with the field. (See Figure 2.1) However, a proton wobbles around the field's axis, which is called precession. Essentially, the proton behaves like a microscopic bar magnet rotating around its axis.[9]

The frequency of this rotation is known as the Larmor frequency (ω_0) and is directly proportional to the magnetic field (B_0). The constant of proportionality is called the gyromagnetic ratio (γ). For protons, $\gamma/2\pi = 42.58 \text{ MHz T}^{-1}$. [10]

$$\omega_0 = \gamma B_0 \quad (2.1)$$

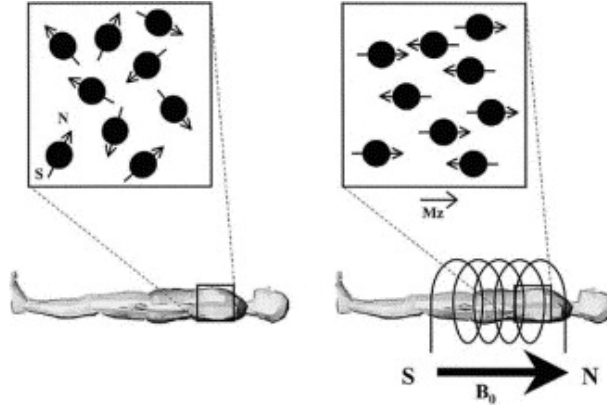


Figure 2.1: Protons align in parallel or antiparallel to the external field. Parallel alignment is a lower energy state, thus slightly more popular, leading to a net magnetisation vector (M_z). Figure from [10]

Here γ is the angular-frequency value. In the KomaMRI simulations later in this thesis, the cycle-frequency convention is used instead, with $\gamma = 42.58 \text{ MHz T}^{-1}$. Accidentally using the angular-frequency value in this context introduces an extra factor of 2π , which shrinks the encoded field of view by 2π .

2.1.1 RF Pulses

The overall net magnetisation from all protons is represented by the vector $\mathbf{M} = (M_x, M_y, M_z)$. At equilibrium, M_z is aligned with the static B_0 field. The transverse components M_x and M_y , which rotate about B_0 , are usually grouped into the complex notation $M_{xy} = M_x + iM_y$.

The receiver coils measure transverse magnetisation M_{xy} , not longitudinal magnetisation M_z directly. A perpendicular radio-frequency field B_1 , applied at the Larmor frequency, tips some or all of M_z into the transverse plane. This creates M_{xy} , which precesses about B_0 and induces a voltage in receiver coils placed perpendicular to the main field, as shown in Figure 2.2. The B_1 pulses are called radio-frequency (RF) pulses because the Larmor frequency lies in the radio-frequency range. [10][9]

The resulting angle between M and B_0 is called the flip angle (α) and can be calculated using:

$$\alpha = \gamma \int B_1(t) dt \quad (2.2)$$

2.1.2 T1, T2 & T2* Relaxation

After an RF pulse is switched off, M_z recovers towards thermal equilibrium along B_0 , while M_{xy} decays in the transverse plane (the free induction decay). These are independent processes: longitudinal relaxation transfers energy back to the surrounding lattice, while transverse relaxation describes loss of phase coherence between spins.

Longitudinal, or spin-lattice, relaxation is characterised by T_1 : the time taken for M_z to recover 63% of the way back to its equilibrium value M_0 after excitation. Because this recovery depends on energy exchange with the local molecular environment, T_1 is an intrinsic tissue parameter.[10]

Transverse, or spin-spin, relaxation describes the decay of M_{xy} as spins lose phase coherence after excitation. Irreversible dephasing from spin-spin interactions is characterised by T_2 , while reversible dephasing from static B_0 inhomogeneity is written as T_2' . The observed transverse decay T_2^* combines both effects:

$$\frac{1}{T_2^*} = \frac{1}{T_2} + \frac{1}{T_2'}. \quad (2.3)$$

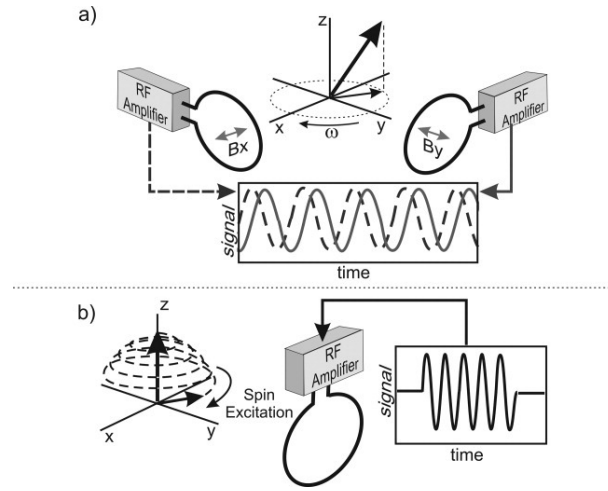


Figure 2.2: A voltage is induced via electromagnetic induction, making an MR signal. (A) The coils are perpendicular and can measure the phase and amplitude of the transverse magnetic vector. (B) RF pulses progressively "flip" the z -axis magnetisation vector into the transverse plane. Figure from [9]

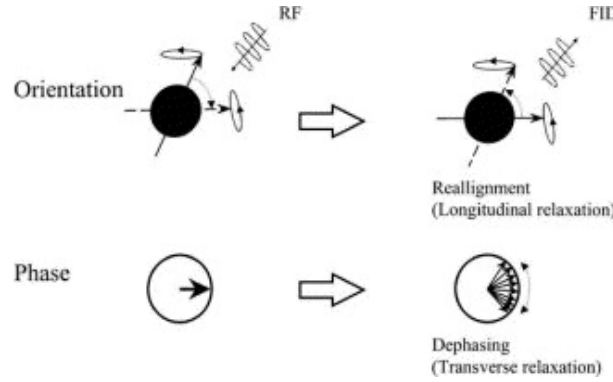


Figure 2.3: **Upper Row:** Longitudinal relaxation (decreasing the flip angle). **Lower Row:** Transverse relaxation (spin dephasing) Figure from [10]

A 180° refocusing pulse reverses the static dephasing term T_2' , which is why spin-echo sequences measure a signal governed primarily by T_2 relaxation. In biological tissue, T_2 is generally shorter than T_1 . [9]

Bloch-equation model. The Bloch equations formalise the relaxation behaviour above. In the rotating frame at ω_0 , with equilibrium magnetisation M_0 along $+z$, the relaxation dynamics can be written as:

$$\frac{dM_z}{dt} = -\frac{M_z - M_0}{T_1}, \quad \frac{dM_{xy}}{dt} = -\frac{M_{xy}}{T_2} + i\Delta\omega M_{xy} \quad (2.4)$$

Here $\Delta\omega$ is the local off-resonance frequency relative to the rotating frame. The first equation describes longitudinal recovery: M_z exponentially returns to M_0 with time constant T_1 . The second describes transverse decay: the magnitude of M_{xy} falls with time constant T_2 , while off-resonance produces additional phase rotation.

Between RF pulses, these independent differential equations have the free-precession solutions:

$$M_z(t) = M_0 + (M_z(0) - M_0)e^{-t/T_1}, \quad |M_{xy}(t)| = |M_{xy}(0)|e^{-t/T_2} \quad (2.5)$$

These are the signal-decay relationships used by the conventional quantitative MRI fitting methods

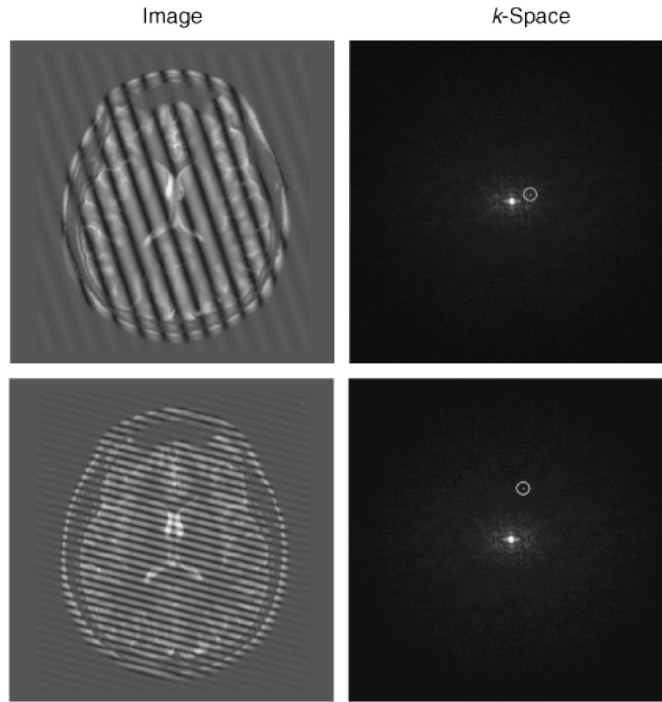


Figure 2.4: Image space and k-space are Fourier-pair representations of the same data: an MR image can be viewed either as a matrix of intensities $I(x, y)$ or as spatial-frequency amplitudes $S(k_x, k_y)$. Each k-space point weights a sinusoidal wave across the field of view; the vector angle (k_x, k_y) sets the wave-pattern orientation, and its distance from the origin sets the spatial frequency. The centre of k-space contains low-spatial-frequency contrast and mean signal, while the outer samples contain high-spatial-frequency detail. Figure from [11].

used later in this work. Under the hard-pulse approximation, an RF pulse is treated as an instantaneous rotation by flip angle α about a transverse axis. If the longitudinal magnetisation immediately before excitation is M_z^- , the pulse leaves $\cos(\alpha)M_z^-$ on the longitudinal axis and tips $\sin(\alpha)M_z^-$ into the transverse plane. The received signal is proportional to this transverse component.

After readout, the sequence models used in this work assume that residual transverse magnetisation is removed by gradient spoiling or by relaxation before the next acquisition block. The only state carried between shots is therefore the scalar longitudinal magnetisation, updated by RF rotations and T_1 recovery. This reduces the Bloch dynamics to simple recurrence relations; without this assumption, transverse coherences would need to be tracked explicitly using a coherence-pathway method such as Extended Phase Graphs (EPG).

2.1.3 Spatial Encoding & K-Space

Relaxation determines how much signal a spin can produce, but an image also requires spatial localisation. MRI does not measure image pixels directly. Instead, the scanner samples k-space: a spatial-frequency representation of the image, shown in Figure 2.4. The centre sample of k-space, $S(0, 0)$, has no gradient moment along either axis, so all spins contribute without a spatial phase ramp. It is therefore the integral of the whole image, or mean signal, conventionally called the DC term.

Let G_x , G_y , and G_z denote magnetic-field gradients along the scanner axes. These gradients make the Larmor frequency position-dependent by varying the local field strength, providing the spatial information missing from the raw MR signal.

The G_z gradient performs slice selection. During the RF pulse, G_z maps z -position to Larmor frequency, so a narrow-band RF pulse excites only the spins whose frequencies lie within the RF bandwidth. This selects a thin slice of the object.

The in-plane coordinates are then encoded using G_y and G_x . A short G_y pulse before readout provides phase encoding: it gives spins at different y -positions different accumulated phases. A G_x gradient is kept on during ADC readout for frequency encoding: successive samples trace spatial frequency along k_x . Repeating the sequence with different G_y amplitudes steps through k_y , so each repetition fills a different row of k-space.

For a gradient waveform $\mathbf{G}(t)$, the sampled k-space location is determined by the accumulated gradient moment:

$$\mathbf{k}(t) = \gamma \int_0^t \mathbf{G}(\tau) d\tau. \quad (2.6)$$

Stronger or longer gradients move the acquisition further through k-space. At a given k-space location, the acquired signal is

$$S(\mathbf{k}) = \int m(\mathbf{r}) e^{-i2\pi \mathbf{k} \cdot \mathbf{r}} d\mathbf{r}. \quad (2.7)$$

Here $m(\mathbf{r})$ is the transverse magnetisation at position $\mathbf{r}$. The exponential term applies a position-dependent phase rotation to each complex M_{xy} contribution before the receiver sums over the object. Changing $\mathbf{k}$ changes this phase pattern, so each measurement extracts a different spatial-frequency component of the image. The image is recovered by applying an inverse Fourier transform to the sampled k-space.

Two design quantities follow directly from this sampling geometry. The k-space sample spacing determines the field of view: $\text{FOV} = 1/\Delta k$. If the object extends beyond this FOV, it aliases back into the image. The k-space extent determines the nominal pixel size, $\Delta x = \text{FOV}/N \approx 1/(2k_{\text{max}})$. Acquiring a wider k-space window improves spatial resolution, but it requires more samples. In particular, higher k_y resolution means acquiring more phase-encode rows, which usually requires additional sequence repetitions.

FFT ordering. The Fourier relationship above defines the k-space/image-space mapping, but not the array ordering used to store samples. KomaMRI returns Cartesian ADC data in acquisition order, with the DC sample $S(0,0)$ near the centre of the stacked k-space matrix. For an even matrix size N , the standard acquisition convention samples one extra negative line,

$$-N/2, -N/2 + 1, \dots, 0, \dots, N/2 - 1,$$

so for $N = 32$ this is $-16, \dots, 0, \dots, 15$. DC therefore lies at index $N/2 + 1$ in one-based indexing. Standard FFT routines instead expect DC at the first array index, so reconstruction requires

$$I = \text{fftshift}(\text{ifft}(\text{ifftshift}(S))),$$

Here `ifftshift` converts the centre-DC acquisition ordering to FFT ordering, and `fftshift` recentres the reconstructed image. Without these shifts, the image is reconstructed under the wrong indexing convention, causing a half-FOV shift and possible checkerboard phase modulation.

2.1.4 Gibbs (Truncation) Ringing

Finite k-space support. The Fourier relationship is exact only if all of k-space is measured. In practice, the acquisition covers a finite $N_{pe} \times N_{fe}$ rectangle. This is equivalent to multiplying the ideal infinite k-space by a rectangular window. By the convolution theorem, multiplication in k-space becomes convolution in image space:

$$I_{\text{recon}}(\mathbf{r}) = I_{\text{true}}(\mathbf{r}) * \text{PSF}(\mathbf{r}). \quad (2.8)$$

The point-spread function (PSF) is the Fourier transform of the sampling window. For a one-dimensional rectangular window, it is the Dirichlet kernel,

$$\text{PSF}(x) = \frac{\sin(\pi Nx/\text{FOV})}{N \sin(\pi x/\text{FOV})}. \quad (2.9)$$

This PSF has a narrow main lobe but slowly decaying side-lobes. At sharp intensity transitions, such as the water–sphere boundaries in the phantom, those side-lobes do not cancel and appear as Gibbs ringing. Increasing N makes the ringing finer in physical units, but it does not remove the side-lobes; the artefact is caused by the rectangular truncation itself. The usual suppression method is apodisation, such as a Hamming window, which down-weights outer k-space to reduce ringing at the cost of a wider PSF and therefore slightly lower spatial resolution. Appendix A.5 derives this trade-off, since it is used directly by the phantom library’s Hamming reconstruction option.

2.1.5 Pulse Sequences

A pulse sequence is the timed series of RF pulses, gradients, and signal acquisitions that produces an image. Its timing parameters set the image contrast and govern the trade-off between signal-to-noise ratio (SNR) and scan time. The two most important are the repetition time (TR), the interval between successive excitations, which controls T_1 weighting and largely sets the total scan time, and the echo time (TE), the delay from excitation to readout, which controls T_2 weighting. Choosing these values is traditionally a manual craft: sequences are “handcrafted by MR experts, who combined strong intuitions and an understanding of spin physics with mathematical solutions of the Bloch equations”[5]. It is exactly this hand-tuning of acquisition timings that the learned, adaptive approach of this project seeks to automate.

2.1.6 MRI Simulators

Bloch-equation simulation is a mature area. KomaMRI, the Julia-based simulator used in this project, is a high-performance, multi-threaded solver that sits alongside the established JEMRIS [12] and MRiLab [13], with the differentiable PyTorch framework MRzero a further alternative [7]. Its publication benchmarked KomaMRI against both, reporting mean absolute signal differences below 0.1% versus JEMRIS with faster runtimes and better usability scores [14]. Crucially, these are cross-simulator agreement checks on short reference sequences, not validations of a quantitative pipeline. At the long sequence times this project requires, KomaMRI produced systematically inaccurate signals, biasing the fitted tissue parameters; Chapter 4 diagnoses and fixes the cause.

2.1.7 Magnetic Resonance Fingerprinting (MRF)

Conventional MRI predominantly produces qualitative, “weighted” images where pixel intensity represents a relative contrast. This depends on acquisition parameters such as coil sensitivity, and makes it difficult to standardise across different scanners. In contrast, Quantitative MRI (qMRI) aims to measure the fundamental physical properties of tissues, such as the longitudinal (T_1) and transverse (T_2) relaxation times mentioned earlier. A significant advancement in this field is Magnetic Resonance Fingerprinting (MRF), which replaces traditional steady-state sequences with “pseudorandomized acquisition parameters”. By varying the flip angles and repetition times (T_R) throughout the scan, MRF generates a unique signal evolution—or “fingerprint”—for every tissue type. These signals are then pattern-matched against a pre-computed dictionary of Bloch equation simulations, allowing for the simultaneous and rapid mapping of multiple tissue properties in a single acquisition. For MR-guided radiotherapy, this quantitative approach offers the potential for “biological gating”, where treatment adaptation is driven by functional changes in tumour physiology rather than just anatomical shifts. [15] [16]

2.2 Digital twins and phantoms for quantitative MRI

Because the target of qMRI is a fitted physical parameter, an image can look plausible while the fitted T_1 values are biased. Validating a qMRI method therefore requires reference objects with known parameter values. Physical phantoms fill this role: recent qMRI phantom reviews treat them as traceable reference systems for repeatability studies, protocol comparison, and cross-scanner validation, not merely as image-quality test objects [17]. The most established example is the ISMRM/NIST system phantom, designed as a common scanner-characterisation baseline with SI-traceable reference materials spanning clinically relevant T_1 , T_2 , and proton-density ranges, monitored long-term by a national metrology institute [18]. Chapter 3 builds a digital twin of exactly this device.

Digital twins extend the reference idea from hardware to executable models. A 2026 systematic review of digital twinning in MRI defines a twin as a dynamic virtual replica linked to its real-world counterpart through data, simulation, and analytics, and maps 51 studies from 2020–2025 [19]. Two of its findings locate this project. First, the MRI-twin literature concentrates on diagnosis, treatment planning, and monitoring (63% of studies); hardware, protocol, and infrastructure applications — the category this project contributes to — account for 20% and are identified as underdeveloped. Second, the review names barriers that this report confronts directly: heterogeneity of MRI data across vendors, sequences, and field strengths; the computational cost of physics simulation; and validation. This project addresses them in a bounded setting — a standardised phantom model, a simulator validated by parameter recovery, and a multi-fidelity training loop that attacks simulation cost. The term “digital twin” is used in a correspondingly bounded sense throughout: the twin represents a calibration phantom for simulation, validation, and policy training, not a patient or a scanner connected to live clinical data.

Table 2.1 summarises how the approaches reviewed in this chapter relate to the adaptive-qMRI problem and what each leaves unresolved.

Related approach	Strength	Remaining gap for this project
Physical qMRI phantoms [17], [18]	Known T_1/T_2 /PD values; repeatable scanner and protocol validation	A hardware object cannot generate randomised simulation episodes or cheap RL rollouts
MRI digital twins [19]	Executable virtual replicas for prediction and optimisation	Existing work targets diagnosis/treatment planning; protocol-level twins are underdeveloped
MRI simulators [12]–[14]	Bloch-equation forward models; sequence testing before scanner execution	Benchmarked on short reference sequences, not end-to-end qMRI recovery under long adaptive acquisitions
Offline sequence optimisation [7], [16]	Learns or optimises non-trivial acquisition schedules	Schedule is fixed before the scan; no conditioning on the current fit
Adaptive MR and RL scanner control [6], [8], [20], [21]	Closed-loop acquisition demonstrably helps	Prior tasks differ: Bayesian T_2 /MRS adaptation, shape classification, 1-D physics, or k-space sampling — not fitted T_1 mapping on a calibrated phantom

Table 2.1: Related approaches, their strengths, and the gap each leaves for closed-loop qMRI sequence design. The project combines the rows: an executable phantom twin (Chapter 3) drives a validated simulator (Chapter 4), and an RL agent is trained and evaluated on the quantitative recovery objective (Chapter 5).

2.3 Magnetic Resonance Linear Accelerator (MR-LINAC)

The MR-LINAC couples real-time MR imaging with dose delivery, replacing the pre-treatment ‘snapshot’ of conventional IGRT with a continuous feedback loop that supports Adaptive Radiotherapy (ART). Imaging the patient on the table lets clinicians correct for inter-fraction change and intra-fraction motion, tightening the margins added around the target (in prostate cancer, planning margins of ≈ 2 mm) and

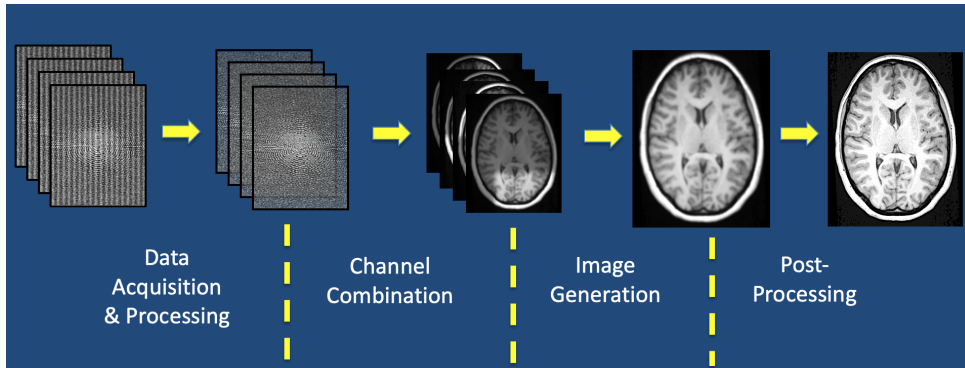


Figure 2.5: Courtesy of Allen D. Elster, MRIquestions.com. [24]

so sparing healthy tissue[22]. Its clinical value is, however, throttled by two bottlenecks this project’s context touches: the extra acquisition time needed for high-quality imaging, and the manual effort of on-table re-planning[4]. AI-based auto-segmentation already addresses the contouring load[23], and the longer-term direction is to move beyond anatomical contrast toward quantitative, biologically-targeted acquisition — the setting this project studies.

2.4 Deep Learning in MRI

Long acquisition times represent a major barrier in clinical MRI, primarily due to the inherent trade-off between image quality and speed of imaging. Indeed, the 3D images crucial for tumour delineation require longer acquisition times, which increases the risk of motion artifacts. There is a large body of research into reducing acquisition time while maintaining image quality. Notable breakthroughs include Parallel Imaging (multiple receiver coil arrays) and Compressed Sensing. Unfortunately, Parallel Imaging quickly reaches practical Signal-to-Noise Ratio (SNR) limits, while compressed sensing is prohibitively computationally expensive. [5]

To supplement, DL has been widely applied, both in research and through commercial products, to reduce imaging time and improve image quality. For example, DL is used in post-processing to remove noise, improve image resolution and reduce image inaccuracies (called artifacts) from patient movement or field inhomogeneities (e.g. caused by metal patient implants or pacemakers). [24]

2.4.1 Image Acquisition and Reconstruction using Deep Learning

Post-processing operates only on the output image, which allows leveraging traditional imaging techniques (CNN, image kernels) as well as being vendor-neutral (so hardware-agnostic). However, not having the raw signal limits the information available.

This has given rise to more complex networks that replace both the Fast Fourier Transform algorithm used for reconstruction and post-processing to create higher quality images.

2.4.2 Pulse sequence design

Recent research has explored the use of artificial intelligence to optimise the underlying physics of MR acquisition through the autonomous design of pulse sequences. Two differing approaches have been tested: an RL game-based approach and a more traditional supervised learning approach.

The MRzero framework introduces a supervised learning approach that treats the entire scanning and reconstruction pipeline as a fully differentiable system. By simulating RF events, gradient moments, and delay times through a Bloch equation layer, the model can be optimised directly against a target

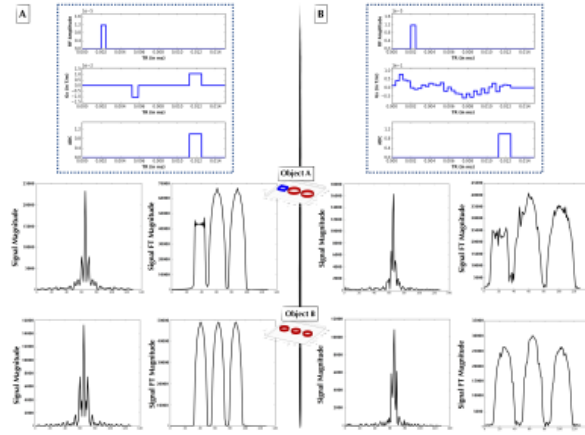


Figure 2.6: (B) Novel pulse sequence successfully developed by AUTOSEQ to mimic a gradient echo sequence. Figure from [6]

contrast of interest (e.g. a T_1 contrast map). The cost function that was optimised by the model took into account: data fidelity, Specific Absorption Rate (SAR) penalties and scan time. The researchers successfully demonstrated that MRzero could “learn” to replicate conventional gradient-echo sequences from scratch. This was then successfully tested on a clinical 3T scanner for in vivo human brain imaging. This demonstrates that deep learning-derived sequences are clinically viable and can be optimised for specific hardware constraints.[7]

By recasting pulse sequence development as a “game of perfect information,” researchers have implemented a reinforcement learning (RL) framework where an AI agent interacts with a Bloch equation-based physics simulator to generate novel sequence actions. This approach utilises a Bayesian derivative of RL, employing Gaussian processes and Bayesian neural networks to navigate the non-linear dynamics of nuclear magnetisation and maximise an acquisition function based on image reconstruction accuracy. Preliminary 1-D experiments demonstrate that such agents can independently “discover” canonical sequences, such as the gradient echo, and generate novel pulse sequences that approximate Fourier spatial encoding. While this was a 1-D experiment, further exploration could yield novel pulse sequences that reduce acquisition time and maximise SNR, alleviating the current acquisition bottleneck faced by the MR-Linac.[6]

2.4.3 Adaptive MRI acquisition

The above work focuses on the design of novel, but ultimately static pulse sequences. Instead, Adaptive MR is a promising new research direction. Here, a probability distribution of the tissue parameter of interest (e.g. T_2) is continuously updated with each new reading. The next pulse sequence is chosen based on the estimate of T_2 . This has been shown to accelerate acquisition time by a factor of 2.5. This is a general paradigm shift in pulse sequence design that motivates and validates further exploration of adaptive MR.[8]

Adaptive MR also has the potential to be combined with RL to autonomously control in real-time MRI hardware. Walker-Samuel demonstrated this by using a Deep Deterministic Policy Gradient (DDPG) algorithm to control an MRI simulation environment where the acquisition process had been turned into a reward-based game. The RL agent was tasked with identifying phantom shapes while being penalised for increased acquisition time. This forces the system to optimise the trade-off between Signal-to-Noise Ratio (SNR) and image acquisition time. Notably, the agent autonomously learned to perform “active sensing,” acquiring only four to six outer lines of k -space and dynamically adjusting T_E , T_R , and flip angles to act as an edge detector. With a reported shape-classification accuracy of 99.8%, this research suggests that RL-controlled scanners could move beyond human-readable imaging to perform high-speed, task-specific “autonomous sensing”. However, this was purely simulation-based and a very simple task, whether this method can be applied to more complex situations is unproven.

A separate active-acquisition literature applies RL to k-space sampling: Pineda et al. train a policy that selects which k-space lines to acquire next so that an undersampled reconstruction improves fastest [21]. This work also treats MRI acquisition as a sequential decision problem, but it controls a different object: sampling locations under a reconstruction-quality objective. It does not choose relaxation-preparation timings such as TI and TR, and it does not optimise fitted quantitative parameter accuracy — the control problem and objective of this project.

2.4.4 Reinforcement Learning and PPO for Adaptive Acquisition

Reinforcement learning (RL) studies problems where an agent improves its behaviour by interacting with an environment. At each step the agent observes the current state of the environment, chooses an action, and receives a scalar reward. The objective is not to predict a labelled target directly, as in supervised learning, but to learn a policy that selects actions leading to high cumulative reward.[25]

The standard mathematical model is a Markov decision process (MDP),

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \rho_0, \gamma), \quad (2.10)$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s_{t+1} | s_t, a_t)$ is the transition model, R is the reward function, ρ_0 is the initial-state distribution, and $\gamma \in [0, 1]$ is the discount factor. A trajectory is a sequence

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots). \quad (2.11)$$

In this project, the environment is the simulated adaptive qMRI acquisition loop. A state contains the information available to the agent so far, such as the current acquisition history and the current estimate of the phantom parameters. An action specifies the next acquisition choice, for example timing and RF parameters such as inversion time, echo time, repetition time, and flip angle. The simulator then returns reconstructed measurements and a reward based on whether the new acquisition improves the quantitative parameter estimate, typically measured through reduction in T_1 error.

For a stochastic policy $\pi_\theta(a | s)$ with parameters θ , the return from time t is

$$G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}. \quad (2.12)$$

The RL objective is to choose policy parameters that maximise expected return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [G_0]. \quad (2.13)$$

The expectation is over trajectories generated by the current policy interacting with the environment. This is important in simulation-in-the-loop MRI: the training data are not a fixed dataset. They are samples produced by repeatedly running the current policy, simulating the resulting sequence, reconstructing the image, fitting the tissue parameters, and computing the reward.

Proximal policy optimisation

This project optimises the policy with Proximal Policy Optimisation (PPO) [26], an on-policy policy-gradient method. Policy gradients estimate $\nabla_\theta J(\theta)$ from sampled rollouts, weighting each sampled action by an advantage estimate $\hat{A}_t$ that measures whether it beat the return the policy already expected; the advantages are computed by Generalised Advantage Estimation [27] from a learned critic $V_\phi(s)$. Crucially, this gradient flows only through the differentiable policy $\log \pi_\theta(a_t | s_t)$ and not through the environment, so KomaMRI can stay a black-box physics simulator used only to sample transitions and rewards. A

plain policy-gradient step can move the policy too far and invalidate the on-policy batch; PPO prevents this with a clipped surrogate objective,

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right], \quad r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}, \quad (2.14)$$

where the ratio $r_t(\theta)$ compares the new and old action probabilities and ϵ bounds how far one update may move them. The full objective adds a critic loss and an entropy bonus. The policy-gradient derivation, the value/advantage definitions, GAE, and the complete objective are given in Appendix A.8.

PPO suits adaptive qMRI for three reasons: it handles continuous action spaces (the sequence timings), it explores stochastically while the clipped objective prevents sudden policy collapse, and its on-policy simplicity makes the physics-and-reward path easier to debug than an off-policy learner. The cost is sample efficiency: every update needs fresh rollouts, and one rollout step here is not a cheap game transition but a full MR sequence build, KomaMRI Bloch simulation, image reconstruction, ROI extraction, T_1 fit, and reward evaluation. The number of simulator calls is therefore the dominant training bottleneck, which is what motivates the multi-fidelity training loop of Chapter 5.

Recurrent policies for partial observability

The PPO actor and critic above are feed-forward networks that map the current observation s_t to an action distribution and a value. This implicitly assumes s_t is a sufficient statistic of the history, i.e. the Markov property holds. When the observation does not capture everything relevant about the past, the problem is a partially observed MDP (POMDP) and a memoryless policy can be suboptimal. Adaptive qMRI is partially observed in exactly this way: a compact observation of the current fitted estimates and remaining budget does not record which acquisition timings have already been played, yet two histories with the same current fit can carry different information for the next block.

A recurrent policy adds an internal memory. The feed-forward trunk is preceded by a recurrent network, here a Long Short-Term Memory (LSTM) cell [28], that maintains a hidden state h_t updated each step from the previous state and the current observation, $h_t = \text{LSTM}(h_{t-1}, s_t)$. The action and value are conditioned on h_t rather than s_t alone, so the policy can summarise the acquisition history into h_t and condition the next timing on it; the LSTM’s gating lets it retain information over many steps without the vanishing gradients of a plain recurrent network. Training uses the recurrent variant of PPO, identical to the above except that rollouts store hidden states and gradients are propagated back through each rollout’s recurrent steps (truncated backpropagation through time). Chapter 5 evaluates such a policy as an ablation against the compact feed-forward observation, testing whether explicit memory of the acquisition history improves adaptive timing.

2.5 Summary

The MRI physics above (relaxation, spatial encoding, pulse sequences) underpins the simulator and fitter of Chapters 3–4, while the deep-learning and reinforcement-learning sections position the adaptive contribution of Chapter 5. The related work also exposes the gaps this project targets, summarised in Table 2.1. Existing adaptive-MRI and automated sequence-design work is either *offline* — the schedule is fixed before the scan, as in magnetic resonance fingerprinting and MRzero [7] — or, where it is closed-loop, is validated on tasks other than calibrated quantitative mapping, such as AUTOSEQ’s 1-D physics simulation [6], Walker-Samuel’s shape classification [20], or k-space sampling for reconstruction quality [21]. On the infrastructure side, qMRI phantoms provide traceable ground truth but no training episodes [17], MRI digital twins remain concentrated on diagnosis rather than protocol development [19], and simulator benchmarks stop short of end-to-end quantitative recovery tests [14]. No prior work pairs an observation-conditioned RL policy with a validated digital-twin simulator and a fitted- T_1 objective, nor confronts the simulation-cost wall a Bloch solver in the training loop creates; these two gaps motivate C1–C3 and the achievements that follow.

3

MRISystemPhantom.jl — Digital Twin Library

Contents

3.1 Chapter aim	15
3.2 The physical phantom	15
3.3 Related phantom tooling	16
3.4 Design goals and architecture	17
3.5 Geometry	18
3.5.1 Sphere voxelisation	18
3.5.2 Slicing	18
3.5.3 Pose transforms	18
3.5.4 Background water	18
3.6 Material tables	20
3.7 Augmentation	21
3.7.1 Episode-level random phantoms	22
3.8 Shipped sequences and fitting models	22
3.8.1 The Cramér–Rao baselines	23
3.9 KomaMRI integration	23
3.10 Library availability	24
3.11 Limitations and future work	24
3.12 Summary	25

3.1 Chapter aim

This chapter delivers A1 and lays the ground for C1: it builds `MRISystemPhantom.jl`, a digital twin of the QalibreMD Model 130 calibration phantom that supplies the ground-truth tissue parameters, geometry, and randomisation the reinforcement-learning loop trains against. The hardest engineering problem is the background water, which is roughly 80% of the spins and therefore dominates simulation cost while contributing little to the per-sphere measurement the agent cares about; the chapter’s main novelty over a fixed phantom model is an independent, coarsenable water grid (Section 3.5.4) that later enables the multi-fidelity cost savings of Chapter 5. In the terminology of Section 2.2, the result is a bounded calibration-object twin rather than a patient twin; Section 3.3 positions it against existing open phantom hardware and analysis tooling. The remaining sections cover the geometry and pose model (Sections 3.5–3.7), the calibrated material tables (Section 3.6), the T_1 sequence and fitter the experiments use (Section 3.8), and how the twin plugs into KomaMRI (Section 3.9).

3.2 The physical phantom

The QalibreMD NIST/ISMRM System Standard Model 130 (manufactured by Caliber MRI, formerly QalibreMD) is a hardware phantom designed to mimic the geometry of a human head and to provide a wide, calibrated range of quantitative MRI tissue parameters (T_1 , T_2 , and proton density). It is the

standard reference device for quantitative MRI system validation in clinical and research settings, and its design is described in the *QalibreMD System Standard Model 130 User Manual* (available here). It is the commercial implementation of the ISMRM/NIST system-phantom standardisation effort introduced in Section 2.2: its contrast spheres carry SI-traceable reference values used for scanner characterisation and multi-site qMRI comparison [17], [18], so the digital twin inherits a recognised calibration target rather than a bespoke test object.

The phantom consists of a spherical water-filled housing (100 mm internal radius) containing four internal structures at fixed axial positions:

Plate	z (mm)	Contents
T1 array	+56.5	14 NiCl_2 spheres (15 mm radius), T1 range ~ 24 ms – 1.9 s at 3 T
T2 array	+16.5	14 MnCl_2 spheres (15 mm radius), T2 range ~ 87 ms – 2.8 s at 3 T
PD array	-23.5	14 spheres of varying proton density (15 mm radius)
Fiducial grid	distributed	57 small spheres (5 mm radius) on a 40 mm cubic lattice

Each contrast plate arranges its 14 spheres in two concentric rings: 10 on an outer ring of radius 65 mm and 4 on an inner ring of radius 28 mm. This geometry matches the phantom manual photographs but has not been verified against the physical device. The current digital twin models the contrast arrays and fiducial grid, but not the resolution insets or the two slice-profile wedges.

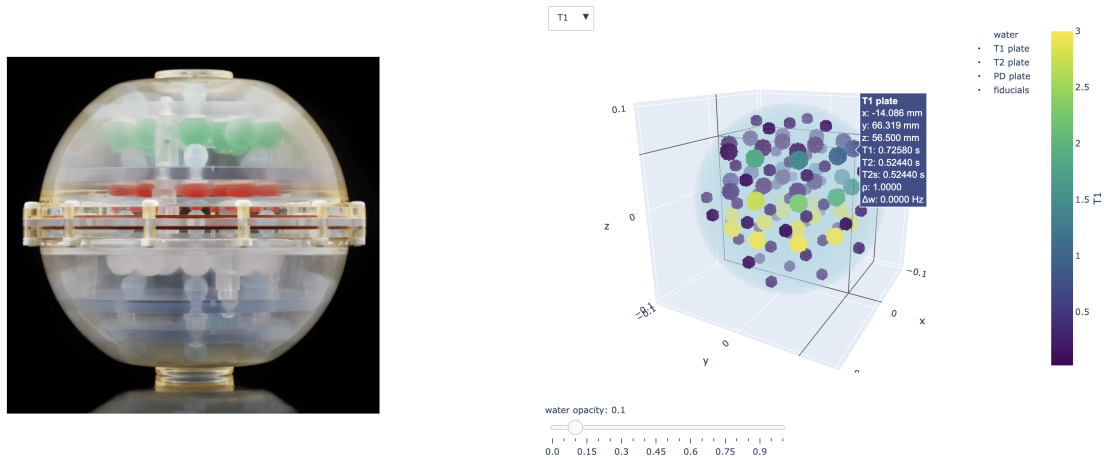


Figure 3.1: **The QalibreMD NIST/ISMRM System Standard Model 130 hardware phantom and its digital T1 array.** The physical phantom photo (left) is reproduced from Caliber MRI. The voxelised plot of the digital phantom (right), coloured by T1 value, can be reproduced by running `examples/plot_phantom_3d.jl` in MRISystemPhantom.

The relaxation values are field-strength-dependent, so separate reference values are provided for 1.5 T and 3 T. The user manual also distinguishes between legacy and modern serial-number classes; both sets of reference values are included in the library.

3.3 Related phantom tooling

The qMRI community already publishes open artefacts around this phantom at two layers. At the hardware layer, NIST releases CAD/STL design files for an open-source MRI phantom, and the ISMRM/NIST

system-phantom repository ships the device’s hardware design files, making the physical reference object itself reproducible.¹ At the analysis layer, PhantomViewer provides a Python GUI with virtual-phantom descriptions for manual ROI analysis of acquired NIST phantom images [29], and MR-BIAS automates that workflow — ROI detection plus Levenberg–Marquardt fits of inversion-recovery and variable-flip-angle T_1 models and multi-echo T_2 models — reducing inter-observer variability and analysis time on acquired system-phantom images [30].

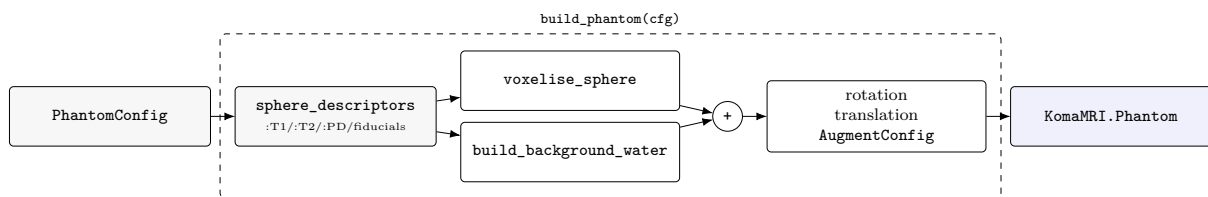
Both layers stop at the physical scan: one makes the hardware reproducible, the other analyses images after acquisition. Neither generates simulated acquisitions, and the MR-BIAS fitting models could not be reused in the adaptive setting of Chapter 5, where the agent changes the timing block by block: the fitter must accept per-block TR and excitation angle and model the finite- N_{pe} transient across phase-encode shots (Section 3.8), whereas MR-BIAS fits one fixed protocol per acquired image series. `MRISystemPhantom.jl` is the complementary simulator-side layer: it represents the phantom *before* acquisition as a `KomaMRI` object with ground-truth material labels and randomisation hooks, and embeds the fitting model inside the simulation/RL loop so it can generate synthetic measurements and rewards for sequence-design experiments.

3.4 Design goals and architecture

A digital twin of this phantom is needed for two reasons. First, it provides a ground-truth environment for RL training: the agent needs to query a simulator that returns physically meaningful signals and can report the true tissue parameters for reward computation. The material tables (Section 3.6) and the geometry/sequence/fitter pipeline (Sections 3.5 and 3.8) address this point. Second, it must be **flexible**: the RL training loop needs to randomise pose, inject noise, and jitter reference values across episodes without modifying library code, and different experiments require different phantom sections or resolution. The configuration contract below and the augmentation/random-phantom machinery (Section 3.7) address this second point.

The library is structured around a single contract: a `PhantomConfig` struct that carries the builder configuration. A single function, `build_phantom`, returns a `KomaMRI.Pantom`, which can then be fed directly to `KomaMRI.simulate`. This design was chosen deliberately over a mutable phantom object or a collection of keyword arguments as the config is serialisable, diffable, and reproducible (the RNG seed is embedded), and the builder has no hidden state. This is useful for comparing different RL runs and also simplifies the Python-Julia boundary used by the training code.

The pipeline from `PhantomConfig` to `KomaMRI.simulate()` is:



The two-stage design — descriptors first, voxels second — means all geometry and material decisions happen before any memory is allocated for spin arrays. Descriptors are also individually modifiable and removable: you can simulate only a subset of the 14 spheres per plate, with the water housing automatically filling the vacated volume. Because each descriptor carries its physical centre coordinate, `sphere_descriptor_pixels` can convert those centres directly to pixel indices in the reconstructed image — using the same DC-at-centre convention as `kpace_to_image` — simplifying ROI extraction after reconstruction.

¹<https://github.com/usnistgov/open-source-mri-phantom>; <https://github.com/MRISStandards/SystemPhantom>.

3.5 Geometry

The geometry implementation is judged by whether it preserves the reconstructed sphere measurements, not by whether every background-water spin is represented exactly.

3.5.1 Sphere voxelisation

Each sphere is voxelised on a cubic grid at spacing `delta_x = voxel_size_mm * 1e-3` metres. A spin is included if its grid point falls within the sphere radius:

$$(x_i - c_x)^2 + (y_i - c_y)^2 + (z_i - c_z)^2 \leq r^2$$

The grid is constructed by iterating over the bounding box $[c - r, c + r]$ on each axis. For a typical 15 mm radius sphere at 2 mm voxels this produces around 500 spins per sphere. Across all three contrast plates (42 spheres), the combined spin count at 2 mm voxels is ~9,100, occupying ~570 KB (8 Float64 fields per spin: $x, y, z, \rho, T_1, T_2, T_2^*, \Delta\omega$), small enough that the full set fits comfortably in memory and the per-step simulate call is fast.

3.5.2 Slicing

For 2D imaging experiments, only spins within a thin axial slab need to be simulated, reducing the spin count, and therefore simulation time, substantially. A `Slab` is defined by a centre point $\mathbf{c}$, a unit normal $\hat{n}$, and a thickness t . A spin at position $\mathbf{p}$ is included if its signed distance from the midplane satisfies:

$$|(\mathbf{p} - \mathbf{c}) \cdot \hat{n}| \leq t/2$$

The normal defaults to $\hat{z}$ but any orientation is supported. A 0.1 μm tolerance is applied at the boundary to avoid floating-point rejections of on-edge voxels. At 1 mm voxels, a 5 mm axial slab centred on the T1 plate retains ~33,000 of the full phantom's ~4.2 million spins, a 128 $\times$ **reduction**. This matches the excited volume of a slice-selective RF pulse without needing to encode one explicitly.

3.5.3 Pose transforms

`apply_transform!` rotates and translates all spin positions by the Euler XYZ rotation and translation specified in `PhantomConfig.rotation` and `translation_mm`. The rotation matrix is built from three successive Rx, Ry, Rz multiplications. The transform is applied after voxelisation and before augmentation noise so that the sphere geometry is exact before any stochastic perturbation. This was designed to make it easy to reset the phantom and reduce overfitting.

3.5.4 Background water

The housing is voxelised as a 100 mm-radius sphere with `water_voxel_size_mm` spacing (defaulting to `voxel_size_mm`), with all contrast and fiducial sphere volumes cut out by the negated form of the squared-distance test used for voxelisation. The water housing is by far the largest spin population, yet it is not a quantity the phantom exists to measure, so it is the natural place to trade fidelity for simulation speed. We do this by increasing `water_voxel_size_mm` to coarsen the grid, thus reducing the spin count. We then reweight each spin's proton density by $(\text{water_dx} / \text{sphere_dx})^3$. This conserves the total water magnetisation, so the bulk water signal amplitude is correct regardless of the coarsening factor.

This coarsening must also handle anisotropic resolution as slices are typically thick (3–8 mm) relative to the in-plane resolution (1–2 mm). So, when a slice is selected the housing is projected onto a grid

of stacked plane-sheets (`water_throughplane_voxel_size_mm` apart). Again, these are re-weighted to conserve total magnetisation.

The figure below quantifies the fidelity–cost trade for an IR-SE-2D acquisition.

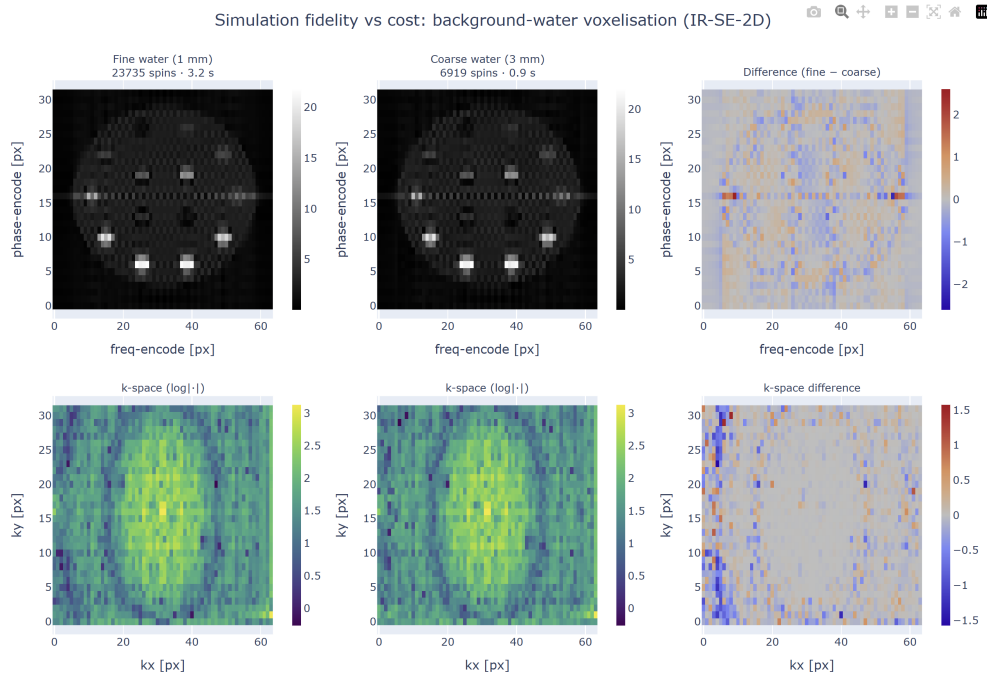


Figure 3.2: **Fidelity–cost trade-off of background-water voxelisation.** The same IR-SE-2D acquisition (TI/TE/TR = 400/80/1500 ms, 32×64 , FOV 0.2 m) is simulated through two phantoms differing *only* in background-water discretisation. **Top row:** magnitude images; **bottom row:** log-magnitude k-space; **right column:** fine - coarse difference. The figure and numbers are reproduced by `examples/compare_water_coarseness.jl`.

Conserving the bulk water signal is necessary but not sufficient. The k-space DC term changes by only 0.35%, but the phantom is used to measure the sphere signals. The coarse 3 mm grid uses **6,919 vs 23,735 spins** ($3.4\times$ fewer) and is $\sim 3.4\times$ **faster to simulate** (0.9 ± 0.1 s vs 3.2 ± 0.6 s, mean $\pm$ std over 20 runs). Using the `sphere_descriptor_pixels` mapping, the single centre-pixel coarse-vs-fine NRMSE at the 32×64 matrix is **3.7%**.

The larger single-pixel error reflects where the coarse grid differs from the fine one. Reweighting preserves the amount of water, but not the exact water/sphere boundary: the 3 mm water grid stair-steps the sphere cut-outs, changing high-spatial-frequency edge content while leaving low-frequency k-space essentially unchanged. The boundary error is on the scale of the reconstructed pixels here: for a 200 mm FOV and 32×64 matrix, the spacing is 6.25 mm in phase encode and 3.125 mm in frequency encode. After band-limited reconstruction, this boundary mismatch appears as a slightly different Gibbs-ringing pattern around the spheres.

Most of this error is concentrated in the near-null spheres at this inversion time, especially T1-spheres 13-14, whose signals move by about 9%, while the bright spheres change by $\leq 0.7\%$. This is expected because the ringing leakage is approximately additive, set mostly by the surrounding bright water.

If the sphere signal is measured as a small central ROI rather than a single point sample, the error falls to **1.5%** for a 3×3 ROI. This is the more relevant operational metric for an ROI-based phantom readout; the single-pixel value remains useful as a worst-case sensitivity measure because it exposes pixel-phase effects, Gibbs oscillation, and sub-pixel centre rounding.

The same pattern strengthens at 64×64 : with unchanged spin counts and DC term, the errors rise to **7.6%** for a single centre pixel and **2.3%** for the 3×3 ROI, consistent with a wider k-space window admitting more of the edge mismatch.

Because the discrepancy is carried by truncation side-lobes, it can be suppressed at reconstruction.

The library provides a 2-D Hamming window through `hamming_window_2d` and `kpace_to_image(...; hamming=true)`. As derived in the background chapter, this reduces the PSF side-lobes from -13 dB to -43 dB, at the cost of a wider main lobe.

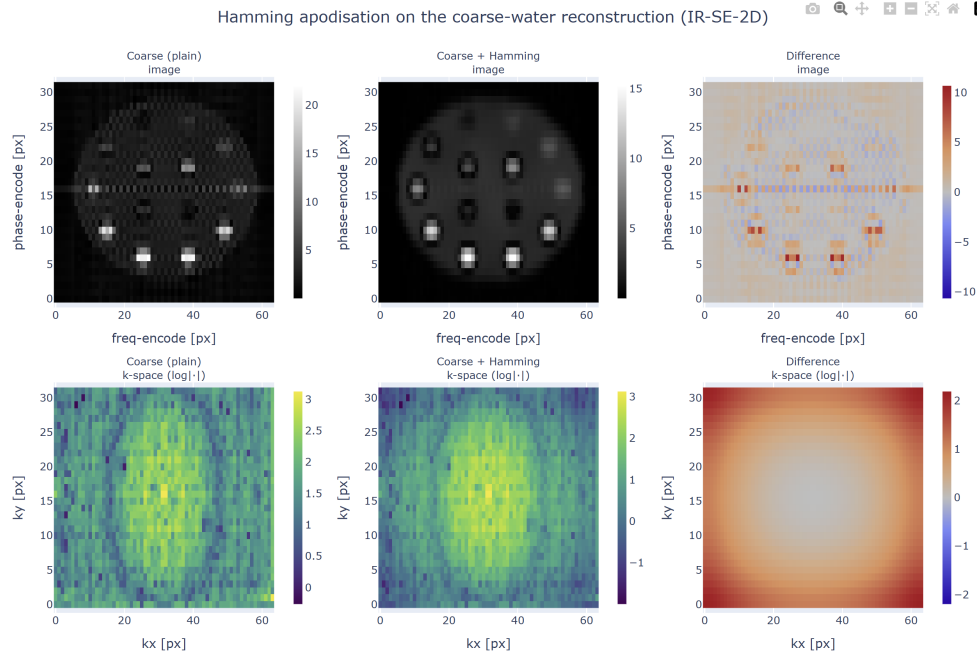


Figure 3.3: **Effect of Hamming apodisation on one reconstruction.** A coarse-water phantom reconstructed without a Hamming window (left), with a 2-D Hamming window (centre), and their difference (right). Produced by `examples/hamming_apodisation.jl`.

Reconstructing both grids with the Hamming window reduces the single-pixel coarse-vs-fine NRMSE by four-fold, from **3.7% to 0.9%**. This confirms that the single-pixel discrepancy was dominated by truncation ringing. The 3×3 -ROI error improves more modestly, from **1.5% to 1.1%**, because ROI averaging already cancels much of the oscillatory side-lobe structure. Apodisation and spatial averaging therefore suppress largely the same error component, rather than providing independent gains. The remaining $\approx 1.1\%$ is the residual coarse-grid discrepancy under this ROI-based measurement protocol.

The Hamming window down-weights outer k-space, visible as the darkened periphery of the centre k-space panel in Figure 3.3. This widens the PSF main lobe and slightly blurs the spheres, but suppresses ringing side-lobes. The difference is localised to sphere edges and high-frequency k-space, consistent with suppression of truncation ringing rather than a change in the smooth low-frequency signal. The window is a reconstruction-side element-wise multiplication and costs only a fraction of a millisecond, so it does not affect the roughly $3\times$ simulation speed-up from the coarse grid. Its cost is resolution, through the wider PSF main lobe.

A natural future improvement is a boundary-aware water grid: keep the smooth water bulk coarse, but voxelise a thin shell around the housing surface and sphere cut-outs at fine resolution so that coarsening no longer changes the boundary geometry.

3.6 Material tables

The relaxation values are read from four module-level constants defined from the phantom manual:

Constant	Spheres	Source
<code>T1_ARRAY[:T3] / [:T15]</code>	14 T1-plate spheres	NiCl ₂ recipe, manual table, Serial Number ≥ 0042

Constant	Spheres	Source
T1_ARRAY_LEGACY[:T3] / [:T15]	same spheres	manual table, SN pre-0042 (different values)
T2_OF_T1_ARRAY	T2 companions for T1 plate	same manual table
T2_ARRAY[:T3] / [:T15]	14 T2-plate spheres	MnCl ₂ recipe
T1_OF_T2_ARRAY	T1 companions for T2 plate	same manual table
PD_FRACTIONS	14 PD-plate spheres	proton density fractions
BACKGROUND_WATER	housing water	distilled water at 20°C

The T1 range at 3 T spans two decades: 23 ms to 1.84 s for the NiCl₂ plate, covering the full range of biological tissue T1 values from white matter to cerebrospinal fluid. The T2 range spans ~87 ms to 2.76 s. These wide ranges are why the phantom is used for system validation and make it useful for RL training: to ensure the learned policies can produce reliable estimates across the full physiological range.

The `serial_number_class` field switches the T1 table between current and legacy recipes. All reference values are currently fixed at 20°C; the manual includes temperature-correction graphs, but these are not yet implemented.

These tables are not merely nominal manufacturer constants. A 2025 multi-centre study measured T_1 and T_2 in the ISMRM/NIST phantom on 13 scanners across three vendors and seven sites, finding excellent correlation with the reference values and median inter-scanner coefficients of variation below 7% [31]. This supports using the tables as a quantitative validation target. The same study tabulates 3 T reference values at six temperatures between 16 and 26°C, which is why the library records its 20°C assumption explicitly: temperature correction is the known gap to close before any simulation-to-real comparison.

Accuracy caveats. Two entries are approximate: PD-plate T1/T2 values (`pd_t1`, `pd_t2`) use bulk water regardless of proton-density fraction (the manual does not tabulate per-sphere relaxation for the PD plate), and fiducial relaxation values (`FIDUCIAL_PROPS`) are generic CuSO₄ placeholders not verified against any calibration record. Neither affects the T1- or T2-plate experiments, but both should be replaced before any simulation-to-real comparison involving the PD plate or fiducial grid.

3.7 Augmentation

`AugmentConfig` enables domain randomisation for RL training. All fields default to zero. When non-zero values are provided, `apply_per_spin_noise!` draws from the relevant distributions using the config's `rng_seed`:

Field	Effect
<code>T1_sigma_rel</code>	Multiplies each sphere's T1 by $\exp(\text{sigma} * z)$ where $z \sim N(0,1)$, drawn once per sphere
<code>T2_sigma_rel</code>	Same for T2
<code>PD_sigma_abs</code>	Adds $N(0, \text{sigma}^2)$ to each spin's proton density
<code>position_sigma_mm</code>	Adds $N(0, \text{sigma}^2)$ to each spin's (x, y, z) position independently
<code>B0_sigma_Hz</code>	Adds $N(0, \text{sigma}^2)$ per-spin off-resonance, stored in Δw (rad/s after $\times 2\pi$)
<code>drop_sphere_p</code>	Drops each sphere independently with Bernoulli probability p before voxelisation

The T1/T2 jitter is applied at the sphere level (one draw per sphere, not per spin) to simulate manufacturing variability in the doping concentration. Of the fields above, only `T1_sigma_rel` and `T2_sigma_rel` were used in the experiments of this work; the remaining fields are implemented for completeness but were not exercised.

The `rng_seed` is the only source of stochasticity: fixing it makes `build_phantom` deterministic, so the same configuration always produces the same phantom. Fixing the seed reproduces the exact same phantom, making evaluation runs reproducible. Incrementing it each training episode draws a statistically independent phantom, preventing the agent from memorising a fixed configuration.

3.7.1 Episode-level random phantoms

A fixed calibrated phantom is unsuitable for RL training because the agent can exploit the known sphere ordering. Instead, each episode should draw a new plausible phantom. Earlier RL code did this manually by editing sampled `SphereDescriptors` and assembling a fresh `PhantomConfig`. `RandomPhantomConfig` makes this explicit: it is a distribution over deterministic `PhantomConfigs`. The base config fixes build settings such as field strength, voxel size, included plates and water model; the random config specifies what changes between episodes: active contrast spheres, sampled material values and phantom pose.

This is distinct from `AugmentConfig`. `AugmentConfig` adds low-level simulation perturbations after the phantom has been voxelised, such as per-spin off-resonance or position noise. It is useful for modelling measurement imperfections, but it does not change the episode’s true sphere-level material values. In contrast, `RandomPhantomConfig` samples descriptor-level properties before voxelisation. For example, one training episode might select five T1 spheres, draw one T1 value for each selected sphere from a log-normal distribution around its nominal value, set T2 by preserving the sphere’s nominal T2/T1 ratio, and then apply a small in-slice rotation and translation. `sample_phantom_config` returns both the sampled `PhantomConfig`, which is safe to pass to `build_phantom`, and a hidden truth record used for rewards and evaluation.

The pose sampler is experiment-dependent. Thin-slice acquisitions use only an in-plane rotation and x/y translation, because a full 3D rotation would move spheres out of the acquired slab. Volumetric experiments can instead use `UniformS03PoseSampler`, which draws a true uniform orientation on $SO(3)$ and stores it as a 3×3 rotation matrix. For this reason, `PhantomConfig` accepts both Euler-angle triples and precomputed rotation matrices.

Samplers are explicit objects, using `Distributions.jl` where appropriate [32], so assumptions such as material jitter, sphere dropout and pose noise are visible rather than hidden in scattered `randn` calls. Material distributions can also depend on other sampled tissue parameters: for example, T1 can be drawn from `Uniform(0.3, 2.5)` while T2 is defined by `PreserveNominalRatio(:T2, :T1)`. These property dependencies are evaluated in order and checked for cycles, so invalid specifications fail at sampling time. The API remains generic: selectors, material samplers and pose samplers operate on `SphereDescriptors` and `PhantomConfigs`, not on RL states, observations or reward code.

3.8 Shipped sequences and fitting models

All sequence builders return standard `KomaMRI.Sequence` objects, and the fitting routines are pure Julia shared by the conventional baselines and the RL environment, so reported gains are always measured against the same estimators. The library ships a broader family than this project exercises — single-spin and 2-D builders for spin echo (`se`), CPMG/multi-echo (`mse`), turbo spin echo (`ir_tse_2d`) and spoiled gradient echo (`gre_2d`), with matching `fit_t2_se` and joint `fit_t1_t2_generalized_ir` estimators for the T2 and PD plates — but the experiments in this report use only the T_1 path, so this section describes that path and leaves the rest to the package documentation.

The T_1 acquisition is `ir_se_2d_sequence(TI, TE, TR; ...)`: an inversion-recovery spin echo parameterised by excitation angle α_{exc} , FOV, `Nfe` and `Npe`, acquiring one k-space row per shot. The matching estimator is `fit_t1_generalized_ir`, which accepts per-block inversion and excitation angles, optional TRs, and `Npe`: passing `Npe = nothing` selects the fully relaxed steady-state model, while an integer `Npe` selects the transient mean over the acquired phase-encode shots, accounting for the ramp from thermal equilibrium when TR is short. Both estimators are log-grid scans rather than gradient fits: `fit_t1_generalized_ir` evaluates 300 log-spaced T_1 candidates over $[5 \times 10^{-3}, 5]$ s (the long-TR `fit_t1_ir` uses 400, the joint T_1/T_2 fit a 200×120 grid) and keeps the lowest-SSE candidate with a

closed-form amplitude at each. The $\approx 2.3\%$ spacing between candidates sets a $\approx \pm 1.1\%$ quantisation floor on any fitted T_1 , so the reported T_1 MAPEs cannot fall below $\approx 1\%$ by construction — this is the “fitter grid floor” referred to in Chapter 5. It and the conventional long-TR `fit_t1_ir` estimator share the longitudinal forward models (`steady_state_mz_at_excite`, `transient_mz_at_excite_npe`, `transient_mz_per_shot`), which are reused unchanged by the cached background-water model (Chapter 5) and the Cramér–Rao baselines, so the same physics underlies the simulator, the fitter and the fixed comparators.

All image-producing sequences use the same reconstruction path, `raw_to_kspace` then `kspace_to_image`, with sphere-centre ROI extraction through `sphere_descriptor_pixels` sharing the DC-at-centre convention so no separate image-space mapping is hand-coded in the experiments. Appendix A gives the fit derivations that matter for interpreting the estimators.

3.8.1 The Cramér–Rao baselines

The fixed-schedule comparators used in Chapter 5 are not hand-chosen: they are Cramér–Rao-optimal schedules, and because the bound recurs throughout that chapter it is worth stating what it is. The Cramér–Rao lower bound (CRLB) is a floor on the variance of *any* unbiased estimator of a parameter, set by the measurement design and the noise alone. For Gaussian noise of variance σ^2 , the information a measurement at inversion time TI_k carries about T_1 is the squared sensitivity of the signal to T_1 , $\sigma^{-2}(\partial s_k / \partial T_1)^2$; summed over measurements this is the Fisher information I , and $\text{Var}(\hat{T}_1) \geq I^{-1}$. With an unknown amplitude A as well, I becomes the 2×2 Fisher matrix $\sigma^{-2} J^\top J$ (J the Jacobian of the signal in (T_1, A)) and the T_1 bound is the (T_1, T_1) entry of its inverse — larger than the single-parameter value, because the information must be shared with the nuisance A , which is why an informative design needs at least two well-placed points. The sensitivity $\partial s_k / \partial T_1$ vanishes at very short and very long TI and peaks near $TI \approx T_1$, so the most informative inversion times sit on the steep part of the recovery; Jones et al. [33] show the exponential optimum is a few peaky points (one near zero, the rest near $1.28 T$).

Because the slope is evaluated at a specific T_1 , the CRLB is optimal *for an assumed value*. To design one schedule for a whole range, `cr_optimize` minimises the mean relative variance $M^{-1} \sum_j \text{Var}(\hat{T}_{1,j}) / T_{1,j}^2$ over a fleet of representative T_1 values — the normalisation makes the amplitude cancel and weights each T_1 fairly — which is exactly the static-average objective of Beracha et al. [8]. This objective is non-convex in the block timings, so it is solved numerically: not by gradient descent or a TI grid, but by multi-start coordinate descent, scoring random log-uniform schedules and refining the best by coordinate moves on each (TI, TR) under the scan-time budget, swept over the number of blocks. Both TI and TR are free and are evaluated on the same transient forward model as the fitter, so the solver can trade short, partially-recovered blocks against longer-TR probes (Chapter 5 reports the resulting schedules and a seed-stability check).

This bound is also the right lens on the fitter itself: `fit_t1_generalized_ir` returns the maximum-likelihood point (the minimum-SSE grid candidate), which is the peak of the same Gaussian likelihood whose curvature is the Fisher information — so the CRLB is the asymptotic width of the posterior the fitter implicitly maximises. An adaptive scheme that retained the whole posterior rather than its peak would obtain that width as a calibrated uncertainty for free (Chapter 5, future work); the point fitter must instead bootstrap it.

3.9 KomaMRI integration

`MRISystemPhantom.jl` is built entirely on top of `KomaMRI.jl` and returns standard `KomaMRI.Photom` objects. The whole phantom is assembled using a single operation: the concatenation operator `+` on `Phantom`. Each voxelised sphere becomes a small `Phantom`, the spheres of each T1, T2 and PD plate are concatenated into that plate, the three plates are concatenated into a sphere stack, and finally the water background is concatenated on top. The result is a single array of spins with heterogeneous relaxation values, which is exactly what `KomaMRI.simulate` expects.

Because the output is an ordinary `KomaMRI.Photom`, existing KomaMRI features work out of the box, such as the interactive plotting tools which are used extensively in the example scripts.

The matching scanner is obtained from `scanner_for_field(cfg)`, which returns a `Scanner` with B_0 set to 3.0 T for `:T3` or 1.5 T for `:T15`. Pairing a phantom with a `Scanner` of a different B_0 silently produces the wrong relaxation values - a mistake the author made in practice - so this helper exists to make the coupling explicit.

3.10 Library availability

`MRISystemPhantom.jl` is developed as a standalone open-source Julia package, with source on GitHub and API documentation on GitHub Pages. It ships a complete test suite (~373 tests) and a set of runnable example scripts covering phantom construction, T_1 mapping, the conventional fixed-sequence baseline, NEMA MS-1 SNR calibration, and the water-coarsening/apodisation fidelity checks reported here. Packaging the work as a tested library rather than experiment-specific scripts is a deliberate contribution: it provides a reusable, reproducible digital twin of a widely deployed calibration device that returns standard `KomaMRI.Photom` and `Sequence` objects, so other groups can check simulated qMRI methods against known T_1/T_2 reference values before scanner deployment.

3.11 Limitations and future work

Beyond the accuracy caveats drawn together in the conclusion (Chapter 5.11), the most consequential limitation is the shipped fitter. It is one-to-one with the sequence family it inverts: each estimator is a closed-form model for a single signal equation (generalized IR-SE), so every new acquisition family needs its own bespoke fit, which does not scale. The estimators are also discrete log-grid scans, so their accuracy is additionally capped by the grid spacing — about 1% at the 300 points used in training and $\approx 0.5\%$ at 500 (Section 3.8). Replacing them with an EPG-based fitting library would both lift that resolution ceiling and break the one-to-one coupling, modelling the spin history of echo trains and partial spoiling across families; integrating a learned deep-learning reconstruction or an end-to-end fit is the further-out alternative, and needs research.

Several extensions would broaden the twin’s reach. Pulseseq import/export would let it drive real scanners and consume the large community bank of vendor-neutral sequences, instead of the hand-built IR-SE and SE blocks it generates today, which would also give the RL agent a far larger sequence action space (Chapter 5). The voxelisation and spin-construction logic are currently coupled to `KomaMRI`; that coupling was productive here but limits portability, and the voxelisation validated in Chapter 4 should be exportable to other simulators — for example to plug the phantom into `MRzero` — reusing the spin logic rather than re-deriving it. The ROI extraction is sphere-centric; supporting arbitrary, non-spherical regions would generalise it to other phantom and anatomy geometries. The water-cache and coarsening fidelity were also validated narrowly: only on the T_1 plate, only for spherical sphere boundaries, with a Hamming apodisation window, and only for the single thin axial slab used here. Gibbs ringing depends on boundary shape, the Hamming window is only the most common of several apodisation kernels, and the cache has not been checked on the T_2 /PD plates or for true 3-D acquisitions — all of which must be confirmed before relying on it outside this setting.

Finally, the library could better support the RL use case it was built for. Its visualisation is currently static recovery-curve and image plots; richer tooling — episode-averaged parameter choices, or videos of the evolving signal rather than snapshots — would make adaptive-policy behaviour far easier to inspect. The longer-term goal is to remove the hand-written fitter entirely and learn end to end from the reconstructed signal; supporting that would mean a different Gymnasium contract, exposing raw images or k-space as the observation and deriving the reward from signal agreement rather than from a fitted estimate.

3.12 Summary

This chapter delivered A1: a configurable, serialisable digital twin of the QalibreMD Model 130 phantom built on a single `PhantomConfig` $\rightarrow$ `build_phantom` contract. The two design decisions that matter for the rest of the report are the independent, coarsenable background-water grid — the lever that makes multi-fidelity training affordable in Chapter 5 — and the explicit, dependency-checked random-phantom samplers that let each RL episode draw a fresh, plausible phantom without touching library code. The known accuracy caveats (placeholder PD/fiducial values, fixed 20°C tables) do not affect the T_1/T_2 experiments but bound any future sim-to-real comparison. With the twin in place, Chapter 4 turns to whether the simulator that consumes it is numerically trustworthy.

4 Validating KomaMRI

Contents

4.1 Chapter aim	27
4.2 KomaMRI simulation model	28
4.3 Initial validation failure	28
4.4 RF edge marker collapse	29
4.5 Closing knot collapse after time rebasing	32
4.6 Validation after the fixes	32
4.7 Evaluation of the fixes	33
4.7.1 Why this mattered for qMRI	34
4.7.2 Why spoiling was a convincing false lead	34
4.8 Limitations and future work	35
4.9 Summary	35

4.1 Chapter aim

KomaMRI is the simulation environment that all RL agents in this project interact with, so simulator validation is a prerequisite for every result in this project. Although KomaMRI is a peer-reviewed, open-source, and actively maintained MRI simulation framework [14], the validation experiments in this project found two floating-point time-discretisation failures. Both were triggered by long cumulative sequence time. The failures were reduced to minimal reproducers and fixes were contributed upstream: issue #779 / PR #780, now merged, and issue #788 / PR #789, open at the time of writing.

This chapter delivers A2, the validated Bloch-simulation pipeline, and together with the digital twin of Chapter 3 (A1) it answers C1: simulator validation for qMRI. The key contribution is the validation itself rather than the upstream reports alone. The two failures were reduced to minimal reproducers, traced to specific floating-point timing mechanisms, fixed upstream, and then verified by recovering the phantom’s known T_1 values in the full qMRI pipeline. The hardest part was diagnostic, not coding: the symptom was a biased T_1 fit that looked exactly like an MRI-physics error (imperfect spoiling, residual transverse magnetisation), and the chapter shows how that false lead was ruled out before the true cause — a relative-precision collapse of a fixed absolute ϵ time marker at large cumulative time — was isolated. To our knowledge this floating-point failure mode has not been reported for an MRI simulator before.

This validation is deliberately different from published simulator benchmarking. KomaMRI’s own publication compared it against the established simulators JEMRIS and MRiLab, reporting mean absolute signal differences below 0.1% versus JEMRIS [12]–[14], so cross-simulator agreement on short reference sequences is already established. What those benchmarks do not test is the level this project depends on: whether a long sequence assembled from many adaptive blocks still preserves quantitative T_1 recovery after simulation, reconstruction, ROI extraction, and fitting (Table 4.1). A timing error of one RF sample biases the fitted T_1 while the image remains visually plausible, which is exactly the failure class the experiments below exposed.

Validation level	Example	What it checks	What it misses
Cross-simulator benchmark	KomaMRI vs JEMRIS/MRiLab [14]	Bloch-solution agreement, speed, usability on reference sequences	Long-cumulative-time behaviour; project-specific pipelines
Qualitative image check	Reconstructed phantom looks plausible	Geometry, rough contrast, reconstruction conventions	Systematic signal errors that bias T_1/T_2 fits
This chapter	Noise-free IR-SE T_1 recovery on the digital T1 plate	End-to-end quantitative correctness at long cumulative sequence times	Hardware imperfections; other sequence families

Table 4.1: Levels of simulator validation. Published benchmarks cover the first row; qMRI requires the third.

4.2 KomaMRI simulation model

KomaMRI simulates the magnetization of each spin of a **Phantom** for variable magnetic fields given by a **Sequence** [14]. All **Phantom** objects used in this project were produced by `MRISystemPhantom.jl`. Each **Phantom** is a cloud of spin isochromats: groups of nearby nuclei that are assumed to share a single position (x, y, z) and one set of tissue parameters $(T_1, T_2, T_2^*, \rho, \Delta\omega)$. Here ρ is proton density and $\Delta\omega$ is off-resonance: the offset of that isochromat’s Larmor frequency from the nominal scanner frequency. Each isochromat is therefore represented by a magnetisation vector (M_x, M_y, M_z) evolving according to the Bloch equations.

The user-facing interface is:

```
raw = simulate(phantom, sequence, scanner; sim_params)
```

Here **scanner** defines the scanner hardware used to play the sequence. In these experiments it is an idealised hardware model: eddy currents and field inhomogeneities are not modelled. This means RF pulses are treated as exact, although in reality a 180° refocus pulse can vary by a couple of degrees across the excited volume.

A **Sequence** is made up of RF pulses, which rotate the magnetisation vector, as well as gradients, delays, and ADC readouts. Internally, **simulate** represents the continuous sequence on a discretised time grid. Gradient events use a time step Δt , while RF pulses are sampled more finely using Δt_{rf} . These time points are then split into blocks. In excitation blocks, where RF is active, KomaMRI integrates the full magnetisation rotation. In precession blocks, where RF is off, the update reduces to relaxation and phase accrual, which can be computed analytically with exponentials and is cheaper to evaluate. At each ADC sample, the simulator sums the transverse magnetisation over all spins to form the complex received signal $\text{sig}[t]$.

The key computational assumption is that spins evolve independently and only interact through the final signal sum. This is a standard MRI simulation approximation and is less restrictive than the hardware idealisations above. It is also what makes KomaMRI practical for this project: spins can be partitioned across CPU threads or GPU kernels, then summed only at the receiver. This parallelism motivated the use of KomaMRI and helps overcome C2, the cost of simulation. KomaMRI also models bulk T_2^* decay rather than microscopic spin-spin interaction. Overall, these are standard MRI simulator assumptions that model the relevant physics to an acceptable error, so learned policies should plausibly generalise to real hardware.

4.3 Initial validation failure

The first validation test was deliberately simple. With no added noise, the test simulated an inversion-recovery sweep across the 14 T_1 spheres and fitted the standard monoexponential recovery model:

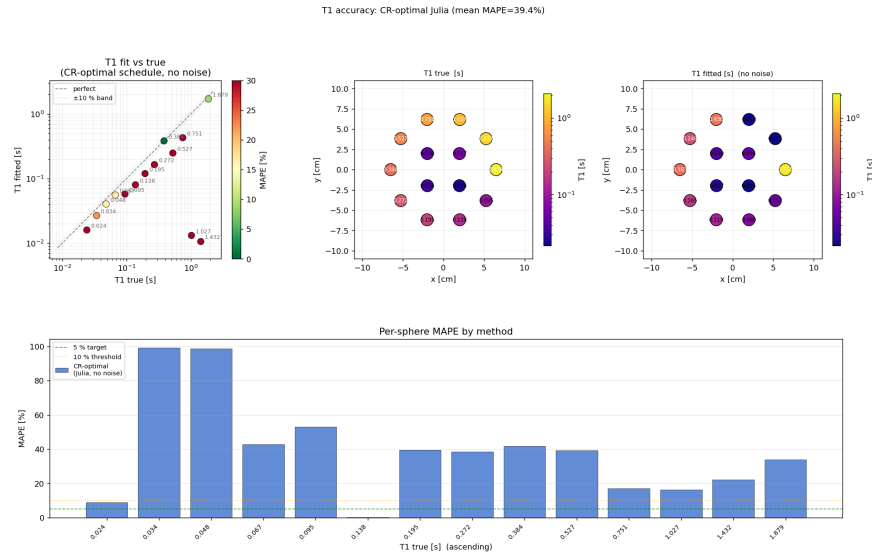


Figure 4.1: Noise-free T_1 recovery before the KomaMRI fixes. The fitted values should lie on the diagonal. Instead the mean MAPE is 39.4%, with only two out of the 14 spheres in the $\pm 10\%$ band.

$$M_z(\text{TI}) = M_0 \left(1 - 2e^{-\text{TI}/T_1} \right).$$

In a noise-free digital phantom, this should be almost exact. Instead, the fitted T_1 values had a mean absolute percentage error (MAPE) of 39.4%, with some spheres close to 100% error. The recovery points did not lie on a monoexponential curve. Either the simulator, sequence, reconstruction, or fitting pipeline was wrong, or there was relevant MRI physics not captured by the simple recovery model.

The first step was to rule out plausible MRI explanations. Imperfect transverse spoiling was the most likely candidate: residual M_{xy} can survive one shot, be refocused by later RF pulses, and contaminate the next echo. The standard way to suppress this is to add spoiler gradients, typically around the end of the shot and around refocusing pulses, to dephase any remaining transverse magnetisation. However, when implemented this did not remove the error; it changed mean MAPE from 39.4% to 44.0%. Increasing TR also made the fit worse, even though longer recovery time should reduce both residual transverse coherence and incomplete longitudinal recovery. This was the first strong clue that the failure was driven by total simulated time, not by the physical state at the start of each shot.

Reconstruction explanations were also tested. Hamming-window filtering and ROI masking did not remove the error, making ordinary Gibbs ringing unlikely. A phase-sensitive reconstruction was also checked to ensure the magnitude image was not hiding a sign or phase convention error. The in-house FFT reconstruction was further cross-checked against MRIReco.jl, a general Julia MRI reconstruction framework [34]: for the fully sampled Cartesian data used here the two agreed, ruling out the reconstruction conventions as the cause. (The in-house path is kept in the RL loop because MRIReco’s encoding-operator construction added roughly 60% overhead per reconstruction, paid at every environment step.) These tests changed details of the images, but not the failure mode.

The decisive diagnostic was to inspect k-space directly. Rather than showing random fitting error, the measured k-space collapsed after a fixed amount of simulated time, even though the acquisition should have remained symmetric. This shifted the investigation away from missing MRI physics and toward the simulator’s handling of long absolute times.

4.4 RF edge marker collapse

The first minimal reproducer was `koma_bug_minimal.jl`. It removed the phantom geometry and reconstruction and repeated the same simple shot at increasing absolute simulation times. The expected result

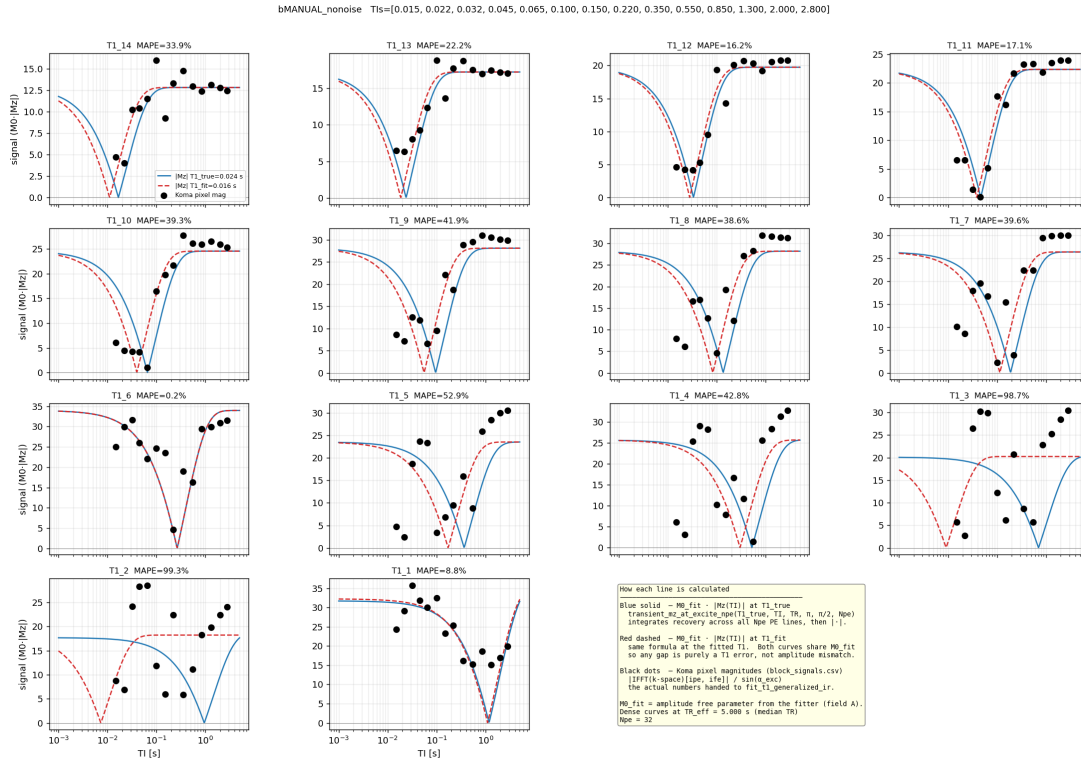


Figure 4.2: Recovery points before the KomaMRI fixes. The simulated samples do not lie on the best-fit monoexponential curves. The deviation is structured, not random, so it points to a repeatable simulation or analysis error.

was a constant per-shot signal. Instead, sharp jumps appeared once cumulative simulated time reached roughly 70–150 s. For example, with $TR = 15$ s the signal jumped at shot 6 (~90 s) by 21%; with $TR = 8$ s it jumped at shot 9 (~72 s) by 19%; and with $TR = 30$ s it jumped at shot 4 (~120 s) by 21%. This showed that the threshold was time-driven, not shot-count-driven, and raised the next question: was KomaMRI discretising the same RF pulse differently at large absolute time?

The RF-edge investigation revealed that the first failure was caused by the way KomaMRI represents sharp RF edges on its simulation time grid. Tracing the RF pulse discretisation led to `points_from_key_times`, and then to the edge-handling logic inside `get_variable_times`. In `get_variable_times`, KomaMRI creates a small set of key times around each RF pulse:

```
points_from_key_times(sort([t1, t1 + MIN_RISE_TIME, tc,
                           t2 - MIN_RISE_TIME, t2])); dt=Δt_rf)
```

where $t1$ and $t2$ are the start and end of the pulse, tc is the centre. KomaMRI encodes the sharp RF edge using a fixed offset $\epsilon = 10^{-14}$ s, implemented as `MIN_RISE_TIME`. The pair $(t1, t1 + \text{MIN_RISE_TIME})$ is used to encode the rising edge of the RF pulse. At $t1$, interpolation should return zero RF amplitude; immediately after the edge, at $t1 + \epsilon$, it should return the full RF amplitude. This creates a near-discontinuous step while still letting KomaMRI store the RF waveform using a small number of key points, with interpolation filling in the values between them.

The problem is that ϵ is a fixed absolute time, but Float64 precision is relative. The spacing between t and the next representable Float64 number, calculated by `eps(t)` in Julia, grows with the magnitude of t : $\text{eps}(t) \approx t \cdot 2^{-52}$. Once $\text{eps}(t)/2 > \epsilon$, adding ϵ no longer produces a distinct Float64 value. In practice this happens at about $t \geq 128$ s. Beyond this point, $t1 + \text{MIN_RISE_TIME} == t1$ bit-for-bit. The two key times that should define the rising edge therefore collapse onto the same value. When the time vector is sorted and deduplicated, one of them is removed.

The isolated test `koma_bug_isolate.jl` backed this up by asking whether the same 1 ms RF pulse is converted into the same time points at different absolute times:

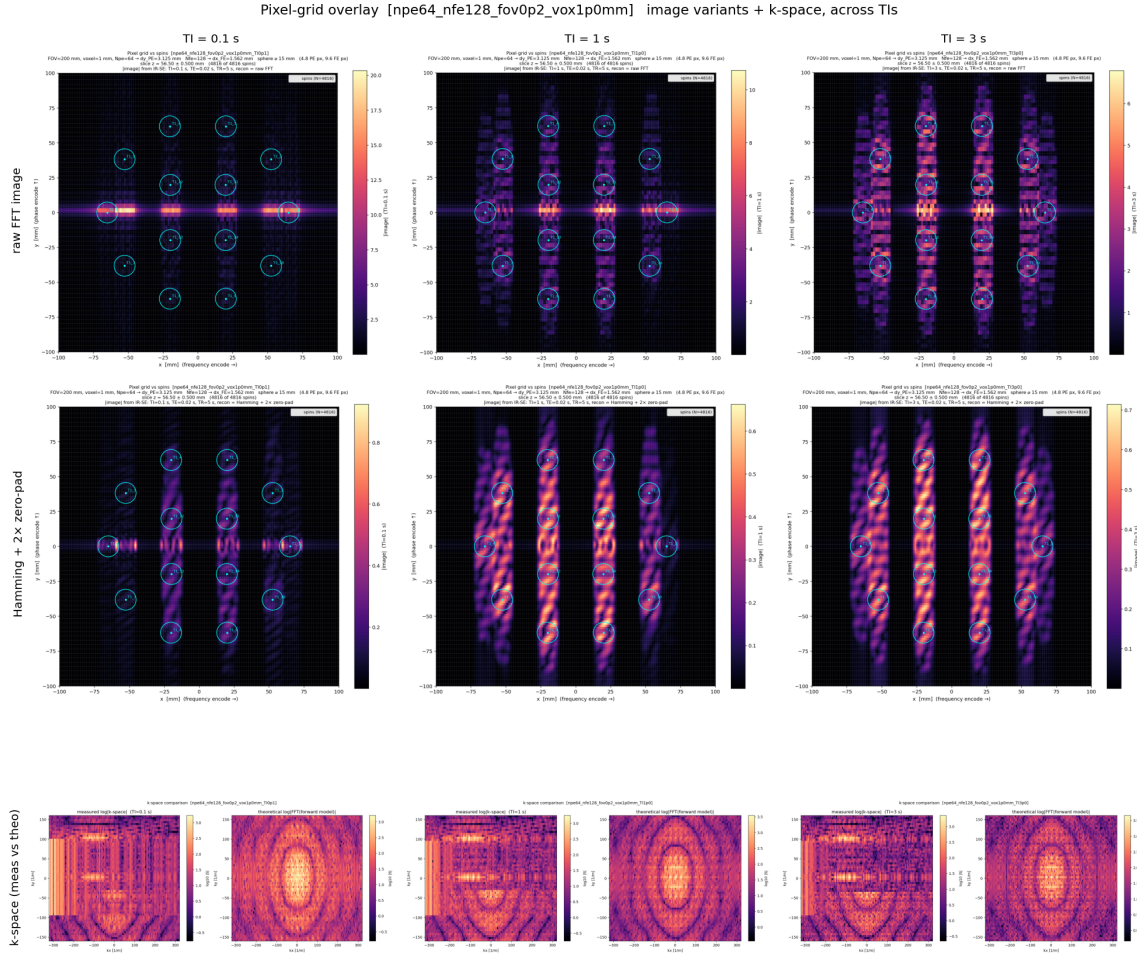


Figure 4.3: Image-space symptom of the timing failure. The figure shows an IR-SE acquisition at three inversion times ($TI = 0.1, 1.0$, and 3.0 s). The first row shows the reconstructed image, the second row applies a Hamming window, and the bottom row compares theoretical and measured k -space. Because the phase-encode direction (k_y) is acquired shot by shot, moving along k_y corresponds to advancing in simulated time. The early phase-encode lines reconstruct correctly, but later lines collapse into horizontal streaks after a fixed cumulative time. This is too large and structured to be ordinary ringing, and is not removed by the Hamming window. Generated by `pixel_grid_overlay.jl`.

t_0 [s]	nraw	nunique	eps(t_0)
0.0	24	23	5.0×10^{-324}
3.0	24	23	4.44×10^{-16}
67.0	24	23	1.42×10^{-14}
99.0	24	23	1.42×10^{-14}
200.0	26	21	2.84×10^{-14}
1000.0	26	21	1.14×10^{-13}

The table shows the collapse directly: early pulses retain 23 unique time points, but at $t_0 = 200$ s this falls to 21 as $\text{eps}(t_0)/2$ exceeds ϵ . This confirms that the RF edge markers have collapsed before simulation.

The collapsed points are not redundant: they represent the two sides of the RF step. At t_1 , RF is zero; at $t_1 + \epsilon$, RF has reached the pulse amplitude. Deduplicating them removes the edge, so the Bloch integrator applies the wrong RF value for one full Δt_{rf} sample. For KomaMRI's default $\Delta t_{\text{rf}} = 5 \times 10^{-5}$ s and $T_{\text{RF}} = 1$ ms, one missing sample is 5% of the RF area. The ϵ -scale timing collapse therefore becomes a 5% flip-angle error, producing a 20–25% signal jump and later a biased T_1 fit.

The fix in PR #780 made the edge markers adaptive:

```
next_time(t) = max(t + MIN_RISE_TIME, nextfloat(t))
prev_time(t) = min(t - MIN_RISE_TIME, prevfloat(t))
```

`nextfloat` and `prevfloat` step to the adjacent representable Float64 value, so the marker remains distinct even at large absolute time. At small times the original spacing is preserved; at large times the

marker becomes one floating point step away rather than collapsing. This fixed the first failure and was merged into KomaMRI.

4.5 Closing knot collapse after time rebasing

After PR #780 the direct checks all passed: the isolated RF-edge script was stable to 1000 s, the minimal multi-shot reproducer no longer drifted over the tested shots, and the new RF-area regression test showed that the discretised pulse area matched the expected trapezium area at large absolute time. The original failure mechanism had genuinely been fixed.

However, the full no-noise T_1 validation was still inaccurate. Further investigation revealed another fixed- ϵ collapse, but at a different point in the time pipeline. KomaMRI also separates a block's closing knot, or block edge, using the same fixed offset ϵ in block-relative time. This is initially safe because the local block times are small (e.g. 1 ms RF pulses). Later, however, `get_variable_times` and `get_samples` rebase these local times to absolute sequence time using $T_0 + t$, where T_0 is the absolute start time of the block. This creates the same floating-point problem as before: at large values of T_0 , the ϵ gap is below Float64 precision. The closing knot rounds onto the block-end knot, the two times become bit-identical, and a later `unique!` step removes one of them. The marker therefore disappears much later in the code than where it was first created, which made this second failure harder to track down.

The reproducer showed that the remaining timing error still mattered. It used one spin and a gradient-free repeated inversion-recovery shot:

```
180 deg inversion -> TI -> 90 deg excitation -> one ADC sample -> TR delay
```

Every shot is physically identical. At $TR = 15$ s, all 24 shots should return the same signal. Instead, the first 17 shots were stable, shot 18 jumped by 52%, and later shots settled to a new wrong plateau:

```
shot 1..17 (sim_time <= 255 s): |signal| = 0.476267 stable
shot 18    (sim_time = 270 s): |signal| = 0.722479 +52%
shot 19..24 (sim_time >= 285 s): |signal| = 0.677159 new wrong plateau
```

The fix in PR #789 re-separates the closing knot after rebasing, when the correct absolute-time spacing is known. In implementation terms, the closing knot is moved to at most $t_{\text{end}} - \max(\epsilon, \text{eps}(t_{\text{end}}))$. The gap is adaptive: it remains ϵ at small times, but widens when Float64 spacing becomes larger. The re-separation is also idempotent: if the closing knot is already distinct, it is left alone. The fix is applied at each rebasing site: both RF sampling paths in `get_samples`, and the gradient timing path in `get_variable_times`. The latter also re-applies an adaptive `_strictly_increasing_knots!` check so gradient knots remain ordered after rebasing. With this patch, the 24-shot reproducer is flat and the KomaMRI tests still pass.

At the time of writing, issue #779 is merged upstream via PR #780. Issue #788 is fixed in a project fork pinned by this repository, with PR #789 still open.

4.6 Validation after the fixes

With both fixes applied, the original no-noise phantom test behaves as expected. The reconstructed images no longer develop time-driven streaks, the recovery points lie on monoexponential curves, and the fitted T_1 values return to the diagonal. Mean MAPE falls from 39.4% to 0.48%, with a maximum sphere error of 1.21%.

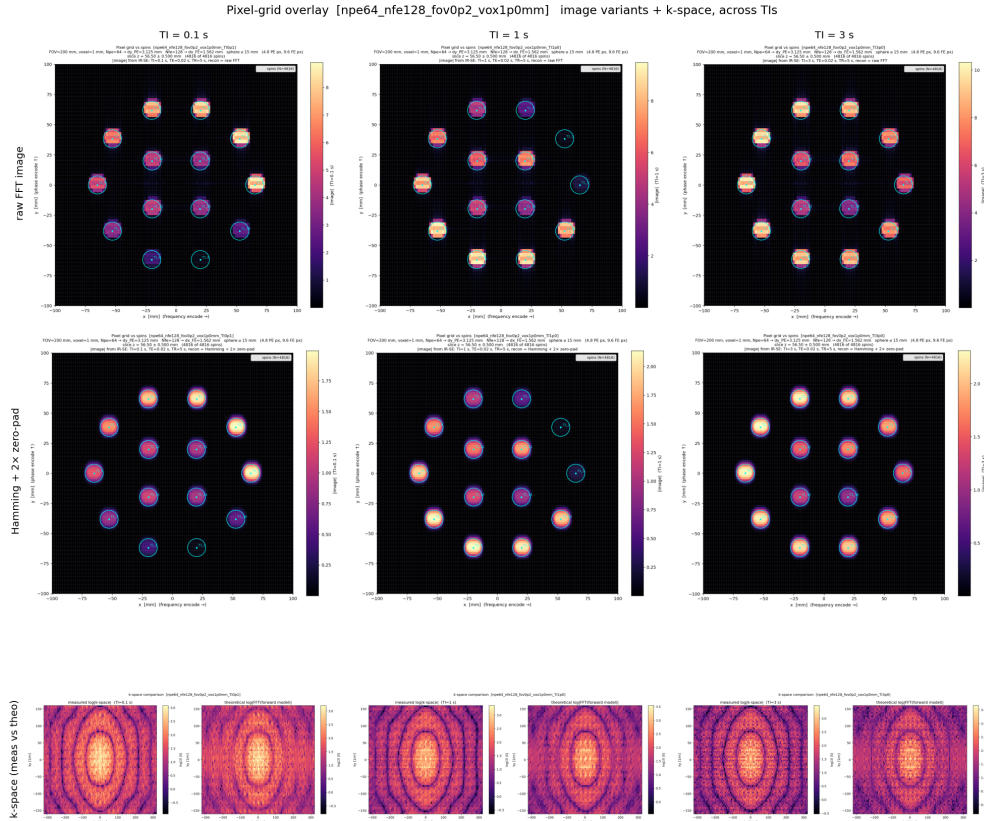


Figure 4.4: Reconstructed images after both fixes. The spheres remain well-localised across the full phase-encode direction. The time-dependent streaking in Figure 4.3 is removed.

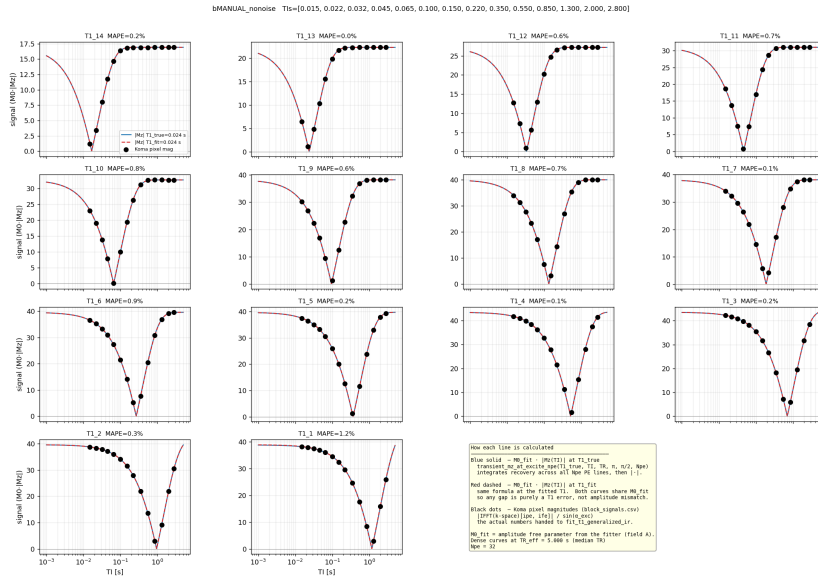


Figure 4.5: Recovery curves after both fixes. The simulated samples now lie on the fitted inversion-recovery curves.

4.7 Evaluation of the fixes

The fixes were evaluated at multiple levels, from isolated timing behaviour to the full qMRI validation task.

Level	Test	Before fix	After fix
Timing representation	RF-edge discretisation at large absolute time	RF edge markers collapse once cumulative time reaches the 70–150 s regime	Stable to 1000 s
Minimal signal reproducer	Identical IR shots, no gradients (<code>koma_bug_minimal.jl</code>)	19–21% signal jump once cumulative time reaches 72–120 s	Flat over tested shots
Residual timing reproducer	24 identical IR shots, no gradients (<code>koma_bug_residual.jl</code>)	+52% jump at shot 18 (270 s), then wrong plateau	Flat over all 24 shots
Full qMRI validation	No-noise 14-sphere T_1 recovery	39.4% mean MAPE	0.48% mean MAPE (1.2% max MAPE)
Regression safety	KomaMRI test suite	New timing tests fail on unfixed code	Passes after both fixes

Table 4.2: Summary of KomaMRI timing-fix validation.

The minimal reproducers show that the specific floating-point failures were removed, while the phantom validation shows that the corrected simulator produces accurate quantitative maps.

4.7.1 Why this mattered for qMRI

The timing failure had likely gone unnoticed because it is triggered by long cumulative sequence time, while most examples and tests are much shorter, never reaching the ~ 128 s absolute-time regime where Float64 spacing becomes comparable to the fixed ϵ marker. This threshold is still well within the duration of many clinical MRI acquisitions, which often last several minutes. The largest effect also appears for hard RF pulses with sharp edges: losing one Δt_{rf} sample is a large fraction of a 1 ms rectangular pulse. In practical slice-selective sequences, shaped RF pulses and gradient ramps make the same timing error a smaller perturbation.

A second reason is that most MRI simulation checks are qualitative: a reconstructed image can still look plausible when noise and contrast variation hide small systematic signal errors. qMRI is stricter. The validation compares fitted T_1 values against known phantom ground truth, so a systematic signal error directly appears as parameter bias. This project exposed the failure because RL training needs long sequences and qMRI validation needs quantitative agreement, not just plausible images.

This validation covers the IR-SE sequence family used for the main experiments; other sequence families would require the same timing and no-noise recovery checks before being used for quantitative claims.

4.7.2 Why spoiling was a convincing false lead

Although presented here as a single validation narrative, the investigation happened in two stages. Before either fix, transverse spoiling looked plausible because it reduced the catastrophic 924.5% MAPE failure to 92.6%, suggesting that residual transverse magnetisation might be part of the problem. After the first timing failure was fixed, the remaining error was smaller, around 30–40%, and therefore more consistent with a possible sequence effect. Spoiling had to be rechecked before the second timing bug was isolated. In the intermediate simulator shown in Figures 4.1 to 4.3, however, spoiling increased mean MAPE from 39.4% to 44.0%. After both fixes, spoiled and unspoiled no-noise fits are both sub-percent (0.43% and 0.48% mean MAPE respectively). Spoiling remains important for realistic sequence design, but it was not the cause of this validation failure.

4.8 Limitations and future work

The timing failure has implications beyond KomaMRI. The same mechanism — Float64 spacing overrunning a fixed absolute ϵ at long cumulative time — could silently degrade other Bloch simulators, most of which are checked only against plausible-looking images. KomaMRI’s own published validation reports $\approx 0.1\%$ agreement with established simulators such as JEMRIS [12], but those benchmarks were run on short sequences; they should be re-checked at the long cumulative times stressed here, and a regression test for long-duration sequences with hard RF pulses should be added so the failure is caught before it silently degrades output.

Broader validation is needed before the simulator is trusted for other qMRI pipelines. The no-noise recovery check should be repeated with shaped (sinc) RF pulses and slice selection, not only hard pulses — the regime where the timing error becomes a smaller perturbation (Section 4.7.1) but other approximations enter. This is a prerequisite for the action-space and slice-selective extensions proposed for the RL agent (Chapter 5): those would have to be validated here first.

4.9 Summary

This chapter delivered A2 and, with the twin, completes the answer to C1. The qMRI validation that A1 makes possible — demanding correct T_1 recovery from a known phantom rather than a plausible-looking image — exposed two floating-point time-discretisation bugs in KomaMRI that only surface beyond ~ 128 s of cumulative sequence time, precisely the regime that RL-driven long acquisitions reach. Both were isolated to minimal reproducers, explained by the relative precision of Float64 overrunning a fixed absolute ϵ marker, fixed upstream, and verified end to end: the no-noise 14-sphere fit improved from 39.4% to 0.48% mean MAPE. The lasting lessons are that quantitative validation catches systematic errors that qualitative image inspection hides, and that a plausible physical explanation (spoiling) must still be ruled out rather than assumed. The corrected simulator is the trusted foundation for the adaptive experiments of Chapter 5.

5

Adaptive qMRI Sequence Design with Reinforcement Learning

Contents

5.1	Chapter aim	37
5.2	Position relative to adaptive MRI work	38
5.3	Environment formulation	39
5.4	Multi-fidelity curriculum	42
5.4.1	The cost wall and the fidelity ladder	42
5.4.2	Cached water: exploiting simulator linearity	43
5.4.3	The switch rule: bias-aware promotion	44
5.4.4	Stage hand-off and global-best checkpointing	45
5.5	Run A: full 14-sphere T1 plate	45
5.6	Run B: five-sphere continuous-T1 task	46
5.7	Memory ablation: what should the policy remember?	48
5.8	Quantitative positioning against published results	50
5.9	Limitations	51
5.10	Future work	52
5.11	Summary	54

5.1 Chapter aim

The previous chapters established the simulator and digital phantom. This chapter uses them for the central adaptive acquisition experiment: can a reinforcement learning (RL) agent choose inversion-recovery spin-echo blocks that estimate T_1 more efficiently than a fixed protocol?

This chapter addresses C2 and C3 from Chapter 1. C2 is the compute problem: a full KomaMRI Bloch simulation is too slow to use naively inside RL. C3 is the adaptive design problem: the best next acquisition should depend on what has already been measured, not only on a fixed protocol chosen before the scan. The main technical contribution is therefore a multi-fidelity RL training loop, plus a controlled evaluation of when adaptivity actually helps.

The chapter’s strongest finding is the *memory-mechanism result* (Section 5.7): what the policy *remembers* about its own acquisitions matters as much as whether it adapts, and an order-invariant histogram of executed inversion times outperforms both a learned recurrent memory and the fitter’s own uncertainty estimate. Reaching that result depended on the multi-fidelity curriculum’s *bias-aware promotion rule* (Section 5.4.3): a cheap, biased rung can lower its own score by exploiting its bias, so promotion is gated on held-out full-simulator validation.

The result splits along task difficulty. Prior adaptive qMRI (Section 5.2) has only ever been demonstrated on a *single* voxel. The 14-sphere T_1 plate is the opposite extreme — a fixed set of spheres deliberately spanning the full clinical T_1 range, so a single well-chosen fixed schedule is already near-optimal. This is the hardest possible case for adaptivity, and as expected strong fixed schedules still beat the learned policy there (which nonetheless stays non-collapsed and adaptive). On the smaller five-sphere continuous- T_1 task, the RL policy does beat the matched fixed log-grid comparator on strict held-out episodes at two scan budgets, and adding the inversion-time histogram produces the best result in the

project (2.93% mean error versus 6.04% for the best fixed schedule). Adaptivity helps exactly where there is something to adapt to.

5.2 Position relative to adaptive MRI work

This chapter borders four literatures: adaptive model-based MRI, automated sequence design, reinforcement-learning control of MRI scanners, and active k-space acquisition. Table 5.1 takes one representative work from each, stating what it established and how its setting differs from the simulation-in-the-loop qMRI environment used here.

Work	What it demonstrates	Critical comparison
Adaptive model-based MR [8]	Bayesian update of tissue-parameter uncertainty during a real-time multi-echo experiment, with simulation, phantom and healthy-volunteer validation	Stronger external validation than this project, including real hardware. It is not an RL controller, and the reported task is a model-based adaptive T2/MRS experiment rather than RL control over a 2-D qMRI phantom acquisition.
MRzero [7]	End-to-end differentiable optimisation of sequence parameters, including a clinical 3T scanner demonstration	Much stronger hardware evidence. However, the optimisation is supervised and offline: it learns a sequence, not a closed-loop policy that changes actions from the current fitted tissue estimates.
AUTOSEQ [6]	Bayesian RL-style search over sequence actions in preliminary 1-D physics simulation experiments	Important early evidence that MR sequence design can be framed as sequential control, but the task is deliberately simplified. It does not evaluate adaptive qMRI estimation on a spatially encoded calibration phantom.
Walker-Samuel [20]	DDPG control of a simulated scanner for task-specific active sensing and shape classification	The closest direct deep-RL comparator. It shows an RL agent can actively choose scanner actions, but the objective is shape classification, not calibrated parameter mapping. Walker-Samuel’s task is more open-ended in one important respect: the agent has to discover useful scanner behaviour from lower-level controls. This project gives the agent higher-level IR-SE building blocks and a fitter. The trade-off is realism of the quantitative evaluation: this chapter uses 2-D k-space reconstruction, a digital QalibreMD phantom, noise, and fitted T_1 error as the reward. It is still weaker because it remains simulator-only.
MRF optimisation [15], [16]	Offline design of time-varying schedules for multi-parameter fingerprints	Directly relevant to quantitative imaging and multi-parameter contrast, but the schedule is fixed before acquisition. The missing ingredient, which this chapter tests in a narrower T1 setting, is observation-conditioned adaptation during the scan.
Active k-space RL [21]	RL selection of which k-space lines to acquire next, improving undersampled reconstruction on retrospective datasets	Sequential acquisition control, but over a different object: it chooses sampling masks under a reconstruction-quality objective, whereas this chapter chooses relaxation-preparation timings (TI, TR) under a fitted- T_1 objective with fixed Cartesian sampling.

Table 5.1: Position of this chapter relative to the main adaptive and automated MRI sequence-design papers discussed in Chapter 2.

The positive Run B result is evidence that RL can learn a useful closed-loop qMRI timing strategy in a quantitative evaluation pipeline. It is simulator-bound for concrete reasons: although the forward model includes B_0 inhomogeneity, the noise model is idealised complex Gaussian and omits B_1 inhomogeneity, gradient and eddy-current effects, and motion; and the sequence family and fitter are supplied, so the agent tunes timings rather than the whole pipeline. Closing those gaps — a real-scanner replay against the physical Model 130 phantom and a wider action space — is the natural continuation, and is the same simulation-to-hardware step that Walker-Samuel [20] also leaves to future work.

The closest single comparator is the adaptive model-based scheme of Beracha et al. [8]. They maintain an explicit Bayesian posterior $p(T_1, \text{PD})$ over a tissue-parameter grid, update it after every excitation, and choose the next contrast timing by a precomputed Cramér–Rao lookup: the posterior mean indexes the CRLB-optimal sampling pattern for that value, and the next acquisition is the one that moves the realised sampling toward it. Their multi-echo T_2 scheme is validated in simulation, phantom and seven healthy volunteers and accelerates acquisition 1.7–3.3 $\times$; the inversion-recovery T_1 variant that is the direct analogue of this chapter is simulation-only, reporting 1.3–2.0 $\times$ better precision (equivalently a 1.7–4 $\times$ acceleration). Their external validation is therefore stronger than this simulator-only chapter’s.

Crucially, Beracha’s scheme solves a one-tissue, one-scalar problem: each step it picks the single inversion or echo time that is optimal for one estimated value, by analytic lookup. This chapter’s setting is neither. A single 2-D acquisition images a *fleet* of spheres spanning the clinical T_1 range, so one (TI, TR)

block must serve them all at once and no single timing is optimal for every sphere — the per-value lookup has nothing to look up. And because the acquisition is spatially encoded, each timing acts through a full 2-D reconstruction rather than on a directly observed signal: the fitted T_1 depends not just on the timing but on reconstruction artefacts such as Gibbs ringing and partial-volume mixing at sphere edges, none of which the analytic lookup sees. Both are why a learned policy is needed; extending Beracha’s scheme instead would require a posterior per sphere aggregated into one shared action, the baseline left to future work (Section 5.10).

A second difference concerns the repetition time. Beracha et al. do not optimise TR for information: in their inversion-recovery model the recovery delay is slaved to the estimate, $\Delta = 5(\langle T_1 \rangle + 3\text{SD})$, and the reported T_1 acceleration comes largely from *shortening* that delay as the posterior sharpens — that is, from relaxing the static scheme’s obligation to assume the longest plausible T_1 for full recovery, rather than from more informative inversion-time placement. The Cramér–Rao baseline used here instead treats TR as a free design variable jointly optimised with TI under an explicit scan-time budget, using a transient signal model that does not assume full recovery (Chapter 3); the solved schedule accordingly pairs short, partially-recovered blocks at short TI with longer-TR probes at long TI. It is therefore a stronger static comparator than a full-recovery schedule would be, because it already captures the budget efficiency Beracha et al. obtain only by adapting Δ .

5.3 Environment formulation

Each episode simulates a 2-D inversion-recovery spin-echo acquisition through the T1 plate of the digital phantom. At each decision point the agent chooses the timing of the next block. The reported runs use $N_{\text{pe}} = 32$, $N_{\text{fe}} = 64$, 1 mm sphere voxels, full background water at target fidelity, and fixed echo time $\text{TE} = 20$ ms. The scan-time budget is 240 s for the main experiments in this chapter. Since one phase-encode line is acquired per shot, each block costs approximately $N_{\text{pe}} \cdot \text{TR}$ seconds. Figure 5.1 shows the resulting agent–environment cycle; the remainder of this section details each stage.

Noise model. Independent complex Gaussian noise with absolute standard deviation $\sigma = 50$ is added to every k-space sample. The value is calibrated, not arbitrary: the signal scale is a sum over simulated spins, so the $\sigma \rightarrow \text{SNR}$ map depends on voxel size, slice geometry and matrix size, and must be derived at the training configuration. At this geometry $\sigma = 50$ gives a NEMA MS-1 dual-acquisition SNR of ≈ 17 averaged across the T1 plate at the reference block, rising to ≈ 28 on the brightest spheres (measured by `scripts/e2_snr_report.jl`). This sits squarely in reported clinical inversion-recovery image quality: a normal-brain FLAIR study measured mean tissue SNRs of 16–30 at 1.5T across the structures analysed [35], while broader fMRI SNR work emphasises the dependence of SNR on voxel volume and acquisition design [36]. The agent is therefore stressed with clinically realistic noise rather than an arbitrary level. A 25-seed noise sweep (Appendix A.6) further shows that T1 fits degrade gracefully up to $\sigma \approx 85$ and collapse beyond, so $\sigma = 50$ leaves the task noise-limited but not estimation-broken.

Action space. The policy outputs a vector in $[-1, 1]^n$ which the wrapper maps to physical timings. The bounds were fixed from the phantom and pulse physics rather than tuned:

- $\text{TI} \in [0.01, 3.0]$ s, chosen to bracket the plate’s inversion nulls (where signal contrast and T_1 information peak), from the shortest sphere’s ≈ 17 ms null ($T_1 \ln 2$ at $T_1 = 24$ ms) to the longest’s ≈ 1.3 s ($T_1 = 1.88$ s). By 3 s the longitudinal magnetisation has nearly fully recovered for every sphere, so the signal is insensitive to T_1 . The 10 ms floor is not a hard physical limit (RF pulses occupy < 1 ms at $B_1 = 20 \mu\text{T}$) but is realistic and never binds.
- $\text{TE} \in [5, 80]$ ms, fixed at 20 ms in all reported runs: TE carries T_2 , not T_1 , information, and freezing it removes a degree of freedom that only dilutes exploration on this task.
- $\text{TR} \in [0.5, 5.0]$ s. The upper bound does not reach full recovery for the longest sphere (the $5 T_1$ rule of thumb would want ≈ 9.4 s at $T_1 = 1.88$ s): the intended regime is steady-state partial recovery, never a fully relaxed start. The fitter models that partial recovery (the generalised IR model of

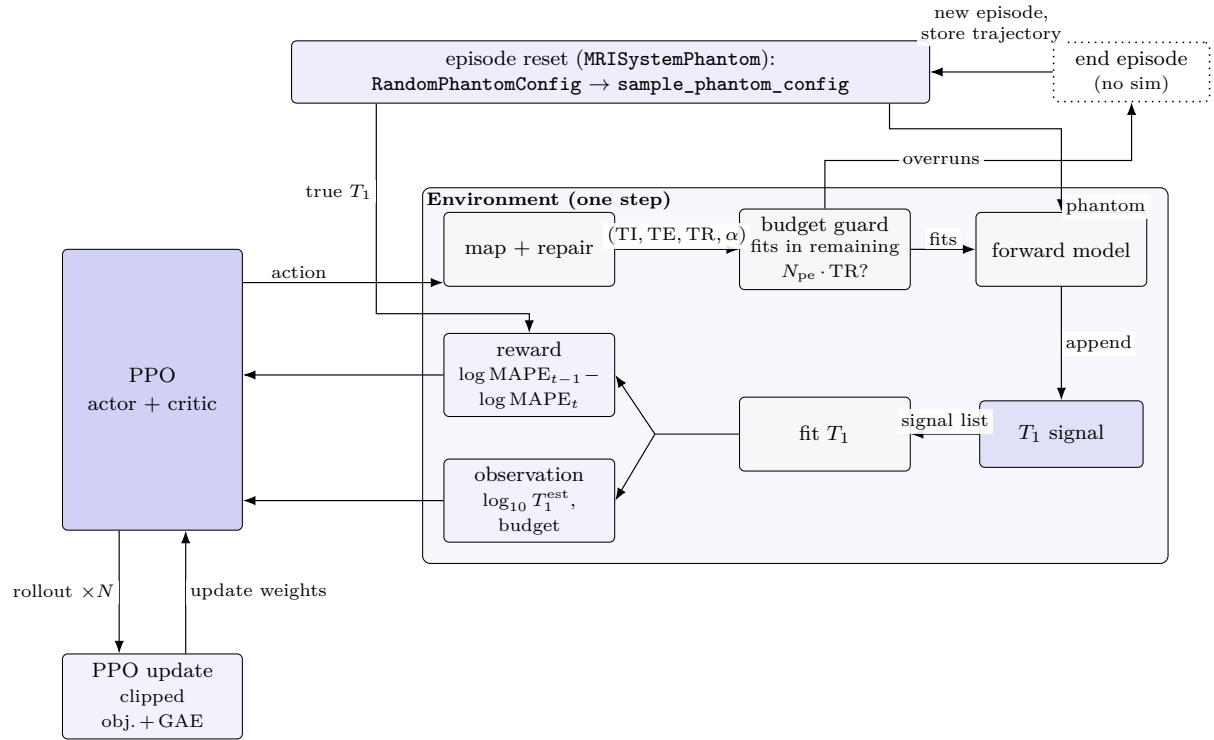


Figure 5.1: One environment step (the inner RL loop), and the per-episode reset that precedes it. Each episode begins by drawing a fresh phantom from the `MRISystemPhantom` library’s `RandomPhantomConfig` sampler, which returns both the KomaMRI phantom the forward model simulates and the hidden ground truth T_1 used only by the reward. Within the episode the PPO actor emits a normalised action; the wrapper maps it to physical timings and repairs infeasible requests; a budget guard ends the episode without simulating if the block would overrun the scan-time budget; otherwise the forward model (cheap analytic surrogate or full KomaMRI simulate–reconstruct–ROI) produces per-sphere signals, the T_1 fit is updated over the accumulated history, and the change in log-MAPE is returned as reward with the next observation. After N such on-policy environment steps, PPO updates the actor–critic weights using the clipped objective and generalised advantage estimates before collecting the next rollout.

Chapter 3), so short-TR blocks are genuinely informative rather than merely corrupted, and the agent trades recovery against the $N_{\text{pe}} \cdot \text{TR}$ block cost.

- $\alpha \in [5^\circ, 90^\circ]$ when learned (Run A), spanning the full Ernst regime.

TI uses a logarithmic action mapping, $\text{TI}(u) = \text{TI}_{\min}(\text{TI}_{\max}/\text{TI}_{\min})^u$ for $u \in [0, 1]$, giving constant action resolution per decade of T_1 ; under a linear map the informative TIs for the shortest spheres occupy roughly the bottom 1.5% of the interval, which starves them of policy resolution. In the later five-sphere ablations α was fixed at 90° . This was an evidence-based simplification: in Run A the learned flip angles collapsed to 90° , and the Cramer–Rao fixed-schedule baseline independently chose 90° because it reduced the CRLB. The flip-angle degree of freedom was therefore redundant for these T1 experiments.

Action repair. The environment repairs infeasible timing requests rather than terminating the episode: the executed repetition time is

$$\text{TR}_{\text{exec}} = \max(\text{TR}_{\text{req}}, 0.5, (\text{TI} + \text{TE})/0.9),$$

i.e. TR must contain the inversion and echo with 10% headroom. Hard rejection would be cleaner semantically, but early random policies would burn rollouts learning the action-space geometry instead of sequence design, and the resulting penalties push PPO towards conservative long-TR choices. The guardrail is audited rather than hidden: every step emits both requested and executed timings plus a repaired flag, and the evaluation scripts report repair rates and mean TR lift. A policy that leans on the repair is therefore detectable; the reported Run B 240 s policy executes zero repairs, and the 560 s policies repair 0–1.8% of blocks.

Observation. The observation is deliberately compact: $\log_{10}$ of the running per-sphere T_1 estimates (a zero sentinel before the first valid fit, which requires at least two measurements), the fraction of scan time used, the fraction of blocks used, and a constant bias input. Observations and rewards are normalised online with clipping at ± 10 . The agent never sees the reconstructed image or raw k-space: it acts only on the fitter’s T_1 estimates and budget, the same interface a qMRI pipeline would expose. This keeps the policy on the genuine estimation objective rather than on simulator-specific pixel patterns that may not generalise to real hardware. Later ablations append an uncertainty proxy from the fitter or an executed-TI coverage histogram (Section 5.7); the main 240 s result uses the base observation. After each block, the environment reconstructs the image, extracts each sphere’s region-of-interest signal, refits T_1 , and computes the reward. The main reward is the improvement in log-MAPE:

$$r_t = \log(\text{MAPE}_{t-1} + \epsilon) - \log(\text{MAPE}_t + \epsilon).$$

This reward is dense and local: it rewards a block for reducing the current estimation error, rather than only awarding a terminal score. This was important because an initial single-voxel experiment (not reported here) collapsed to a nearly fixed policy when a large terminal bonus made it sufficient to finish the episode rather than optimise the measurement sequence.

The form was chosen after a reward-screening study on the fast analytic RL environment, before committing compute to Bloch-in-the-loop training (Table 5.2). The first candidate, a direct per-step $-\text{MAPE}_t$ penalty, looked natural but was misaligned: because the reward is summed over blocks, it penalised taking additional informative measurements. Under a short PPO screen it collapsed to a zero-block policy (final MAPE 1.00), as did its log variant $-\log \text{MAPE}_t$. A purely terminal-MAPE reward removed the misalignment but was too sparse, giving a weaker learned policy. Progress rewards were the best compromise: both ΔMAPE and $\Delta \log \text{MAPE}$ ranked the lowest-error scripted schedule correctly in the cheap probe, and the short PPO screen gave the lowest learned-policy MAPE for $\Delta \log \text{MAPE}$ (0.830 versus 0.839 for ΔMAPE). Worst-sphere pressure was also tested through an aggregation $\alpha \text{mean} + (1 - \alpha) \text{max}$ over per-sphere errors, motivated by the risk that a few poorly constrained T_1 estimates are hidden by the mean; in practice $\alpha < 1$ worsened the screen (e.g. 0.942 at $\alpha = 0.5$), most likely because the identity of the worst sphere is noisy early in an episode when the fits are still underconstrained. A more thorough exploration at longer rollouts would be useful future work. However, the reported runs therefore use the mean $\Delta \log \text{MAPE}$ form above.

Reward	Aggregation	30k PPO final MAPE
$\Delta \log \text{MAPE}$	mean	0.830
ΔMAPE	mean	0.839
terminal only	mean	0.896
ΔMAPE	0.5 mean + 0.5 max	0.866
$\Delta \log \text{MAPE}$	0.5 mean + 0.5 max	0.942
$-\text{MAPE} / -\log \text{MAPE}$	mean	1.000 (no blocks)

Table 5.2: Reward-screening study on the analytic RL environment (14-sphere T1 plate, 30k-step PPO screen). Progress rewards were necessary — the summed per-step penalties collapsed to a zero-measurement policy — and mean $\Delta \log \text{MAPE}$ gave the best learned policy. Worst-sphere weighting ($\alpha < 1$) did not help in the cheap screen.

Training uses PPO [26] with the Stable-Baselines3 `MlpPolicy`: a small multilayer perceptron with two hidden layers of 256 units (`tanh`) shared between the policy and value heads, trained at learning rate 10^{-4} , $\gamma = 0.99$, $\text{GAE-}\lambda = 0.95$, entropy coefficient 0.005, and 512-step rollouts. PPO was chosen because it is a robust on-policy baseline and can be used with continuous actions without needing a replay buffer. This matters for the Python–Julia simulator loop: each rollout step triggers simulator, reconstruction, fitting, and reward code across the Python–Julia boundary. During development, a simple on-policy learner made it easier to diagnose whether failures came from the MRI model, the fitter, the reward, or the policy update itself. However, the use of a replay-buffer and RL algorithms that are more simulator efficient should be explored as an alternative to the multi-fidelity approach taken below.

5.4 Multi-fidelity curriculum

The environment step of Figure 5.1 is too expensive to run naively at target fidelity, so the agent is trained up a ladder of forward-model fidelities that trade accuracy against simulation time, warm-starting each stage from the last. The subsections below justify the ladder (Section 5.4.1), the cached-water rungs (Section 5.4.2), and the promotion rule that decides when to move up it (Section 5.4.3).

5.4.1 The cost wall and the fidelity ladder

The main engineering difficulty is simulator cost. Every environment step triggers a multi-shot Bloch simulation of the whole phantom, followed by 2-D reconstruction and a refit. The cost is set mainly by how many RF pulses the integrator must resolve: KomaMRI steps through the free-precession delays (TI, TR) with large time steps, but each RF pulse needs fine time discretisation, so the per-step cost scales with the number of shots N_{pe} (one excitation and refocusing pulse per phase-encode line) and the spin count n_{spins} , and only weakly with the requested timing. At the target configuration (T15, 64×32 , 1 mm voxels, 1 mm slab through the T1 plate) the sliced phantom contains 23,735 spins, of which 18,919 (80%) are background water. The sphere material that the agent actually measures is only 4,816 spins. A full-Bloch step measured about 4 s on CPU, so a 200k-step PPO run is roughly nine days of simulation before any evaluation overhead. The reported Run B family was trained on GPU, where the same stages ran some 5–7 $\times$ faster (≈ 0.06 s/step at the cached and coarse-water stages); extrapolated by spin count to the 1 mm full plate this is still ≈ 0.2 s/step, leaving a single-fidelity full-Bloch run on the order of a day before evaluation overhead. Single-fidelity full-Bloch training therefore does not fit the project compute budget on either device.

Because 80% of the spins are water, both cost levers target the water: *coarsen* it or *cache* it. Coarsening reuses the digital twin’s independent water grid (Chapter 3): water spins are laid out on a coarser grid with proton density rescaled to conserve total magnetisation, and a 3 mm water grid cuts the water spin count by 9 $\times$ ($18,919 \rightarrow 2,103$), taking the phantom to 6,919 spins. Caching, described next, removes the water from the per-step simulation entirely. Together with the analytic surrogate these give the ladder in Table 5.3, spanning a $\sim 30\times$ per-step cost range.

Fidelity	Water model	Bloch spins	s/step	Main bias
analytic	closed form	0	0.034	no reconstruction, no water crosstalk
cached3	cached template, 3 mm	4,816	0.34	cached and coarse water
cached	cached template, 1 mm	4,816	≈ 0.34	cached water, target geometry
full13	full Bloch, 3 mm	6,919	0.42	coarse water discretisation
full	full Bloch, 1 mm	23,735	1.03	target fidelity

Table 5.3: Fidelity ladder used for RL. “Bloch spins” is the number simulated per step at the 14-sphere plate; the cached rungs Bloch-simulate only the 4,816 sphere spins and add the water from a template built once and amortised over many episodes. The s/step are measured Run A CPU stage averages, set mainly by N_{pe} and the spin count rather than the requested timing; the cached rungs measured $8.0\times$ faster than matched full Bloch. Spin counts reproducible using `scripts/e2_spin_counts.jl`.

5.4.2 Cached water: exploiting simulator linearity

Since the Bloch simulation is linear in spins, the acquired k-space separates exactly:

$$S_{\text{full}} = S_{\text{spheres}} + S_{\text{water}}.$$

And because the water is a single homogeneous material, its contribution to phase-encode line k (acquired on shot k) factorises into a scalar shot amplitude and a fixed geometric profile:

$$S_{\text{water}}[k, :] = M_z^{(k)}(\text{TI}, \text{TR}, \alpha) \sin \alpha W_\alpha[k, :].$$

Here $M_z^{(k)}$ is the water’s longitudinal magnetisation at the k -th excitation, available in closed form from the same transient recovery model the fitter inverts, and W_α is a template capturing everything else — spin positions and Fourier encoding, T_2 decay at the reference echo time, static B_0 phase. The idea is to separate what we can compute from what we cannot. How the acquired signal scales with the timing (TI, TR) is known in closed form: it enters only through the scalar $M_z^{(k)}$, the same recovery curve the fitter inverts. Everything else — the spatial and encoding physics that no closed form captures — does *not* depend on the timing, so it can be measured once by the simulator and then reused. The template is therefore obtained once, by running a single Bloch simulation of the water alone and dividing the $M_z^{(k)} \sin \alpha$ factor out of each k-line; thereafter a new timing only changes the scalar, never the template.

Evaluating a new (TI, TR) then costs $O(N_{\text{pe}})$ arithmetic for the new $M_z^{(k)}$ plus a row-wise rescale of the template; only the sphere spins are re-simulated. This is the $\sim 8\times$ per-step saving of the cached rungs. The alternative of modelling the water analytically as well is measurably too crude: it leaves visible reconstruction and crosstalk residuals and per-sphere T_1 deviations from the full simulation of up to $\sim 40\%$, whereas the cached model matches full-Bloch T_1 fits to 0.12% on average (worst sphere 1.15%, the fitter’s discrete-grid floor). Appendix A.7 shows the validation images and fits.

However, the factorisation is exact only at the α the template was built with. Two effects were measured. First, the closed-form transient mis-scales across α : at 90° , $\cos \alpha = 0$ and every shot starts from the same thermal state, but at 30° residual magnetisation carries between shots and the ratio of simulated to analytic per-shot amplitude drifts from 1.01 on the first k-line to 1.40 on the sixteenth. Second, the template itself changes with α through spoiling and echo formation, by 7–21% spatially between a 30° and a 90° template; this error is TI-independent, so no TI rescaling can correct it. The implementation therefore stores a *bank* of α -matched templates on a grid spanning the action range. At a matched α a template is exact to $\sim 10^{-8}$ relative k-space error; an off-grid α is evaluated by computing the two bracketing templates, each at its own α , and blending linearly in α . A template is never rescaled to a different flip angle analytically — that would reintroduce exactly the error the bank exists to avoid.

One constraint follows from the construction: the template is keyed to the spin positions, so the cached rungs hold the phantom pose fixed, while the full-Bloch rungs and all full-fidelity probes apply the in-plane pose jitter. Cheap stages therefore optimise under slightly narrower conditions than the target measures — a stated fidelity compromise, accepted because every promotion and checkpoint decision is scored on the full environment.

5.4.3 The switch rule: bias-aware promotion

The design question is *when* to promote: fixed per-stage step counts waste budget, and a cheap rung cannot be trusted to judge its own readiness, since it can keep raising its own score by overfitting to its biased simulation environment. Promotion and checkpointing are therefore scored on a held-out full-simulator probe, not the cheap rung’s own number. This follows two ideas from multi-fidelity RL — selecting the fidelity with the best information gain per unit cost [37], and promoting once low-fidelity samples stop helping the real task [38] — though neither is implemented literally, as no posterior over policies is maintained. An alternative is to pool rollouts from all fidelities concurrently with bias correction or control variates [39], [40]; this project uses a serial curriculum because warm-starting one policy up the ladder keeps the orchestration simple and reuses the single in-process Julia simulator (Section 5.10 returns to the parallel design).

Concretely, every fourth PPO rollout (2,048 steps at $n_{\text{steps}} = 512$) the current policy is probed for four episodes on the current-fidelity environment and four on the full environment, with identical seeds. The full-probe score is

$$P_H = -\log_{10}(\text{clamp}(\text{MAPE}_H, 10^{-3}, 1)),$$

where the clamp models each rung’s accuracy floor so a saturated cheap stage reads as flat. Two step bounds act as practical guardrails around the adaptive logic: a per-stage *minimum*-step gate, below which no promotion signal is even evaluated (so a stage cannot be abandoned on a few noisy probes), and a hard per-stage *maximum* step cap, which forces a promotion if the adaptive signals never fire. Between those bounds, the trainer promotes out of a cheap stage when any of four signals fires:

1. **Target plateau:** the relative gain in P_H over the last three decision points falls below 2% — the cheap rung has given what it can.
2. **Ranking breakdown:** the Spearman correlation between cheap-probe and full-probe MAPE across the stage’s decision points falls below a per-rung guardrail (0.7 for **analytic**, 0.8 for the cached rungs, 0.9 for **full13**), evaluated only once at least four points exist. A cheap rung may be biased, but it must still *rank* policies like the target.
3. **Bias intolerance:** the cheap/full probe gap exceeds one percentage point of mean MAPE or two points of p90.
4. **Budget reserve:** remaining wall-clock falls to the 20% of the total budget reserved for the final full stage. This overrides every other gate, guaranteeing the target fidelity always receives compute.

The whole curriculum also runs under a hard wall-clock deadline (9 h for the reported runs). On every promotion the *fidelity gap* is recorded — the cheap rung’s own final MAPE minus the full-probe MAPE of the same policy — as a diagnostic of how much of the cheap rung’s apparent progress was real rather than bias. The decision logic is a pure Python module, unit-tested on synthetic metric histories separately from the PPO and Julia orchestration, and every decision is appended to a JSON history together with the signal that fired, so each run’s switching behaviour can be audited afterwards.

It should be stressed that these signals are pragmatic rather than principled. The Spearman guardrails (0.7–0.9), the one-percentage-point bias tolerance, the probe cadence (every four rollouts) and the per-stage step caps are all hand-tuned thresholds, chosen to make the curriculum behave sensibly on the reported runs rather than derived from theory. They worked, but a promotion criterion with fewer free numbers would be more defensible (Section 5.10), and the thresholds above may need re-tuning for a different ladder or task. In practice the comparison gates did most of the promoting: across the Run B family the plateau rule fired only twice in twenty promotions, while ranking breakdown drove about half and bias intolerance about a third. A further “V2” guardrail was implemented to check, before promoting on a plateau, that the rung above would actually improve faster — by cloning the policy onto the next fidelity for one rollout and comparing the improvement slopes — but it triggered only once across all reported runs and never changed a promotion. That single firing (Run A, Section 5.5) is reported below as a negative result rather than as a working feature.

5.4.4 Stage hand-off and global-best checkpointing

Only the forward-model and water settings change between rungs; the observation and action layout, reward, noise and resolution are held fixed so the warm-started weights remain valid. The observation-normalisation statistics carry across each switch — the observation is physical ($\log_{10} T_1$ in seconds), so its running statistics are comparable across fidelities — but the reward normalisation is reset, because the reward scale shifts with each rung’s bias.

Finally, the trainer maintains a global-best checkpoint across the whole curriculum, selected on the target fidelity. This was added after Run A showed that the final policy of a curriculum can be worse than one it had already passed through (Section 5.5). Selection is two-tier because four-episode probes are far too noisy to pick checkpoints: every full probe (switch decisions, stage starts and ends, periodic stage evaluations) only *screens* candidates, and a candidate that beats the incumbent on its screen is re-evaluated on twelve independent episodes at a separate seed offset. Only a confirmed improvement overwrites the checkpoint, and its metadata records both the screening and the confirmed scores. The fixed twelve-episode confirmation is itself crude: a more principled rule would sequentially add episodes and stop once the confidence intervals separate the candidate from the incumbent at, say, the 5% level, rather than fixing the count in advance. Report-grade numbers are taken from neither screen nor confirmation: all tables in this chapter come from fresh 24-episode evaluations with confidence intervals.

5.5 Run A: full 14-sphere T1 plate

Run A was the first end-to-end V2 curriculum. It used the full 14-sphere T1 plate, a 240 s scan budget, T_{15} field values, $\sigma = 50$, fixed TE, and learned α . The ladder was

$$\text{analytic} \rightarrow \text{cached3} \rightarrow \text{full3} \rightarrow \text{full}.$$

The run was trained under a 9 h wall-clock budget; results are in Table 5.5 below.

Stage	Steps	Wall [h]	s/step	End full-Bloch MAPE
analytic	8.2k	0.08	0.034	16.60%
cached3	41.0k	3.28	0.338	10.37%
full3	73.7k	3.94	0.416	14.03%
full	79.5k	1.64	1.03	12.26%

Table 5.4: Run A stage summary. The productive stage was **cached3**; moving to **full3** degraded the target-fidelity score, and the final full stage did not recover the best cached checkpoint.

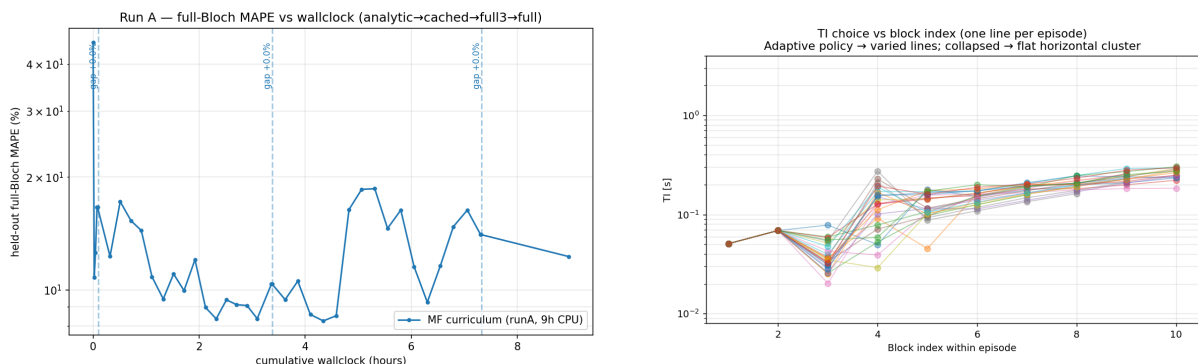


Figure 5.2: Run A curriculum behaviour. Left: full-Bloch probe MAPE versus wall-clock time. The cached-water stage reached the best basin, then full3 regressed. Right: the learned policy was not collapsed; TI varies across episodes and within an episode, but the final policy still under-sampled the long-T1 regime.

Figure 5.2 is the main diagnostic, and the logged switch reasons make the controller’s behaviour concrete. The analytic stage was left on a *ranking breakdown* (the cheap probe stopped ordering policies like the full probe) after 8.2k steps; it had already collapsed the full-probe MAPE from $\approx 42\%$ to $\approx 10\%$ essentially for free. The **cached3** stage was left on a *target plateau* at 41k steps, having improved the

full-probe score into an 8–10% basin — the best the run ever reached. The problem occurred after the switch to `full13`: the target-fidelity score jumped back up and ended worse than the cached basin, and that stage was only left on the *budget reserve* override at 74k steps, handing the last 20% of wall-clock to the `full` stage. The 3 mm-water full-Bloch environment was therefore not a harmless bridge to the 1 mm target; its optimum differs from the 1 mm optimum the probe scores (rank correlation ≈ 0.6 , bias $\pm 5\%$).

The one-off V2 lookahead gave a useful warning at exactly this boundary. When the `cached3` slope collapsed, the controller cloned the policy, trained it for one rollout (≈ 230 s) at `full13`, and re-scored it: the clone’s full-Bloch MAPE got *worse* ($10.37\% \rightarrow 11.69\%$), so it correctly *declined* to promote on slope grounds. The plateau rule promoted anyway, and the regression the clone had predicted in 230 s then played out over the whole 3.9 h stage. This is the informative negative result: the lookahead signal was right, but the controller’s action space did not include “skip this rung and keep the global best”, which is what the evidence called for — so the cheap warning could not be acted on.

Policy or schedule	Mean MAPE	95% CI	p90	Mean time
<code>cr_optimal_alpha</code> fixed schedule	4.70%	[4.42, 5.00]	5.68%	240 s
<code>log_grid_trmatched</code> fixed schedule	5.80%	[5.24, 6.40]	8.06%	218 s
Run A cached-best checkpoint	9.44%	[8.18, 10.94]	14.11%	231 s
Run A final policy	11.68%	[10.42, 13.03]	15.22%	232 s
Run A analytic checkpoint	16.03%	[13.97, 18.56]	20.87%	223 s

Table 5.5: Run A comparison on 24 strict seed-600000 held-out evaluation episodes; that seed was not used for training, curriculum switching or checkpoint selection, so the result is not a leakage artefact. The Cramér–Rao schedules are solved as in Section 3.8.1, by multi-start coordinate descent (1000 log-uniform starts, refine the best 10, seed 0) on the nominal 14-sphere fleet. The agent was adaptive, but the fixed schedules were better on the full 14-sphere task.

That final policy was using its T_1 estimate, not replaying one fixed list of actions. Over 24 episodes its TI choices varied both within a scan (intra-episode $\log \sigma$ 0.281) and across phantoms (inter-episode $\log \sigma$ 0.290), the most-used action bin was only 11.9% of choices, and the chosen TI correlated positively with the running T_1 estimate and with the most uncertain sphere ($r = +0.50$ and $r = +0.64$). The only repeated actions were the first two exploratory shots, taken before the T_1 estimate was informative; thereafter the timings tracked the running estimate.

The erosion is visible in the learned TIs. The cached-best policy had pushed its TI ramp up to ≈ 0.8 s, long enough to characterise the mid-to-long- T_1 spheres, cutting Pool 2/3/4 ($T_1 = 1.43/1.03/0.75$ s) error to 18.7/13.8/10.7%. The two expensive stages then shrank the ramp back to 0.3–0.6 s and reduced intra-episode exploration (TI $\log\text{-}\sigma$ 0.38 $\rightarrow$ 0.28), re-inflating those same pools to 28.2/23.8/19.0%. The longest sphere, Pool 1 ($T_1 = 1.88$ s), was beyond reach for every checkpoint ($\approx 41\%$): even the cached-best 0.8 s TIs fall short of its ≈ 1.3 s inversion null, so this is an action-coverage ceiling, not only a training-time problem.

The flip angle carried no useful signal: it saturated at the 90° bound, and the curriculum only drove it there harder (α std $2.4^\circ \rightarrow 0.0^\circ$ across checkpoints), with the fixed Cramér–Rao baseline independently choosing 90° too. It was fixed at 90° in Run B, leaving TI/TR timing as the optimisation target.

The two lessons carry into Run B: global-best checkpointing is necessary because later fidelity stages can erase a good cheap-stage policy, and the full 14-sphere task — which rewards robust global coverage — is not the sharpest test of closed-loop adaptivity, since one fixed schedule already serves a near-fixed fleet well.

5.6 Run B: five-sphere continuous- T_1 task

Run B is a problem chosen specifically to make adaptation more useful. Five spheres are kept from the plate and their T_1 values are resampled continuously each episode over the phantom range, with T_2 and T_2^* holding their nominal ratios to T_1 . The geometry is fixed but the realised relaxation distribution changes from scan to scan, which is the variation a fixed grid cannot anticipate and an adaptive policy can.

The main completed Run B result is the 240 s no-uncertainty GPU run. It used the ladder

`analytic` $\rightarrow$ `cached3` $\rightarrow$ `cached` $\rightarrow$ `full13` $\rightarrow$ `full`,

adding an extra target-resolution `cached` stage and reducing the per-stage minimum step counts, to counter the Run A `full13` failure. Removing `full13` entirely might have been cleaner.

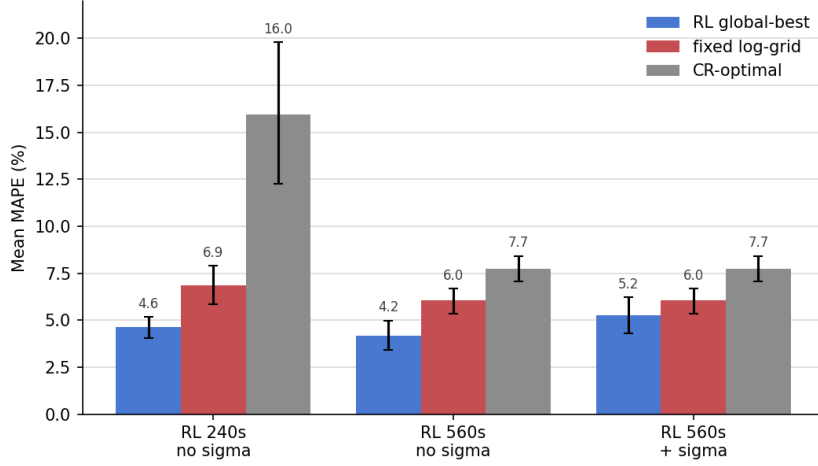


Figure 5.3: Run B five-sphere results, all on the held-out seed: global-best RL policies versus the matched fixed log-grid and CR-optimal baselines at 240 s and 560 s, plus the σ -channel memory arm. The CR-optimal schedule collapses at 240 s because it is solved for one nominal fleet (Section 5.6).

Policy or schedule	Mean MAPE	95% CI	p90	Success < 5%
Run B RL global best	4.62%	[4.06, 5.19]	6.24%	62.5%
Fixed log grid, TR 1.7 s (4 blocks)	6.86%	[5.83, 7.90]	10.11%	20.8%
Fixed log grid, TR 1.24 s (6 blocks)	9.70%	[8.16, 11.27]	16.46%	12.5%
Fixed <code>cr_optimal</code>	15.96%	[12.26, 19.81]	29.29%	0.0%

Table 5.6: Run B 240 s five-sphere results, 24 held-out episodes per row. The RL policy beats the best fixed log grid (TR 1.7 s) by ≈ 2 points; a fixed grid forced to the policy’s own realised TR (1.24 s, matching its block count) does *worse*, not better, so the gain is adaptive placement rather than block count; and the Cramér–Rao schedule collapses under per-episode T_1 variation.

The 240 s policy beats the best fixed log-grid comparator (TR 1.7 s) by about two percentage points of mean MAPE on the held-out seed, with non-overlapping confidence intervals (Figure 5.3, Table 5.6). The Cramér–Rao schedule collapses on this task at 240 s (15.96%, p90 29.29%): solved over (TI, TR) for a nominal fleet, it commits its budget to four long blocks and under-covers the episode’s realised T_1 values. This is the static-optimal versus static-average distinction of Beracha et al. [8], [33]: a Cramér–Rao schedule solved for a fixed nominal fleet is the static-*optimal* design for those values and wins when the evaluation fleet matches it (Run A, Table 5.5), but under per-episode T_1 variation it is mis-targeted, while the robust log grid (effectively a range-averaged design) degrades more gracefully. The log grid does not beat an information bound; it beats a schedule that was optimal for the wrong values, and not one that was merely under-solved, since re-solving the schedule across seeds 0–3 moves the objective by only 2.2% (15.17–15.51) — far smaller than its gap to the grid. The adaptive policy in turn recovers the per-episode performance the static-optimal schedule cannot deliver without knowing T_1 in advance. Binning the held-out errors by *realised* T_1 (the meaningful breakdown here, since each sphere is an independent draw from the clinical range) shows the estimate is roughly uniform across that range: the low, mid and high T_1 tertiles score 4.3%, 3.8% and 5.7% mean MAPE respectively, so no T_1 band is badly estimated, with only a mild rise at long T_1 where the inversion null moves toward the upper end of the policy’s TI reach.

The learned schedule in Figure 5.4 is qualitatively different from Run A. The global-best 240 s policy uses 5.92 blocks on average, with no timing repairs, usually starting with two high-TI probes around 0.78–0.83 s and then moving to shorter refinements. The correlation between TI and current T_1 estimate is negative ($r = -0.720$); rather than a monotonic “higher T_1 , longer TI” rule, this is a two-phase strategy: first buy a long inversion probe that is informative across the sampled range, then shorten TI once the estimate has stabilised.

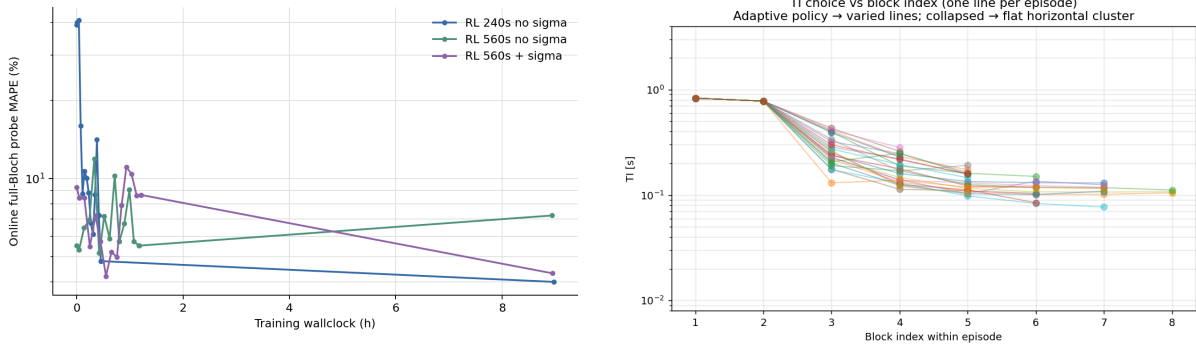


Figure 5.4: Run B 240 s policy behaviour. Left: full-Bloch probes during the curriculum. The final full stage is noisy and global-best selection is needed. Right: the global-best policy opens with a high-TI probe and then uses shorter adaptive refinements.

The advantage holds at a 560 s budget: the global-best RL policy scores **4.16%** MAPE against **6.04%** for the fixed `log_grid_trmatched` schedule, whose TR is set to ≈ 1.7 s (the policy’s approximate mean) with the TI-point count derived from the budget as $\lfloor \text{budget} / (N_{\text{pe}} \text{TR}) \rfloor$ (ten points at 560 s).

The policy’s *realised* mean TR is shorter (≈ 1.24 s), so it fits more blocks than the baseline in the same budget (≈ 14 versus 10 at 560 s, ≈ 6 versus 4 at 240 s), raising the question of whether the gain is adaptive placement or simply more measurements. Re-solving the fixed grid at the policy’s own TR settles it (Table 5.6): the TR-1.24 s, six-block grid does *worse* than the TR-1.7 s one, not better. A shorter TR leaves less longitudinal recovery per block, so six poorly-recovered measurements lose to four better-spaced ones; the policy nonetheless wins in that regime, so its advantage is adaptive TI/TR placement, not a block-count gain. A legacy seven-point grid at a different TR scores 5.86%, confirming the comparison is insensitive to the exact point count.

The 560 s comparison is not a pure scan-time ablation of the 240 s run, because the 240 s run still learned α . The fixed log grid is essentially flat from 240 to 560 s (5.60% $\rightarrow$ 5.71%), so the RL gain is not simply “more blocks”: the 560 s policy uses its ≈ 14 blocks in a different, closed-loop allocation, and the 240 s policy is the stronger efficiency story, beating the fixed schedule with only ≈ 6 blocks.

This is the chapter’s strongest evidence for A3: on a task where a fixed global schedule is less naturally optimal, the RL policy found a compact closed-loop strategy that beat the fixed comparator on both mean and p90 error at two budgets, confirmed on strict held-out seeds each time. The result is still bounded by one training seed per arm.

5.7 Memory ablation: what should the policy remember?

The RL environment is a partially observed MDP. The observation carries the *running fit* but not the *acquisition history*: the per-sphere T_1 estimates are a lossy summary of which (TI, TR) blocks have already been spent. Two different sampled-TI sets can produce similar estimate vectors while the information value of the next TI depends precisely on which TIs have already been bought, so a policy that cannot see its own history can only learn open-loop schedules modulated by the current estimate. The final experiment therefore compares three mechanisms for carrying that history, each added to the 560 s five-sphere no-memory control of the previous section with everything else held fixed: the same active spheres, continuous- T_1 sampler, fidelity ladder, 9 h wall-clock budget, $\alpha = 90^\circ$, and global-best checkpointing.

Estimator summary (σ -channel). Appends $\log_{10}(\sigma_{T_1}/T_{1,\text{est}})$ per sphere, clamped to $[-3, 0]$, from the fitter’s bootstrap uncertainty, with 0 as the no-fit sentinel. This is a confidence proxy, not a calibrated posterior: the fitter estimates (T_1, A) jointly and uses its residual-based noise estimate only beyond four measurements, so early in the episode — exactly when guidance is most valuable — the channel is sentinel- or noise-floor-driven.

Task-informed sufficient statistic (TI-coverage histogram). Appends twelve bins, uniform in the agent’s own logarithmic TI action coordinate over $[0.01, 3.0]$ s (about 0.21 decades per bin), each holding the count of *executed* blocks in that bin divided by the 20-block episode cap. Executed, post-repair TIs are recorded, so the channel stays truthful when the TR-lift guardrail changes what was actually played. The encoding is deliberately order-invariant: the T_1 fitter consumes the *set* of (TI, magnitude) pairs, so a permutation-invariant coverage vector matches the estimator’s sufficient statistic in twelve dimensions, where a padded action sequence would need forty. TR history is not encoded; the time-used fraction already carries spent scan time.

Learned summary (LSTM). Keeps the control’s observation unchanged and swaps PPO for RecurrentPPO (sb3-contrib `MlpLstmPolicy`) with the same $[256, 256]$ torso, a default 256-unit LSTM, and identical optimiser and PPO hyperparameters. Running it without the histogram keeps the ablation clean: memory enters through recurrence *instead of* state augmentation, so the difference between the two new arms is attributable to the mechanism, not to stacked channels.

All four arms were evaluated with the Run B protocol: 24 strict held-out episodes on each run’s global-best checkpoint, with the fixed schedules re-evaluated by the baseline harness on the same seed. Table 5.7 and Figure 5.5 give the result.

Policy or schedule (560 s)	Mean MAPE	95% CI	p90	Succ. < 5%	Blocks
TI-coverage histogram	2.93%	[2.58, 3.30]	4.19%	95.8%	13.6
No-memory control	4.16%	[3.41, 4.99]	6.64%	66.7%	14.0
σ -channel	5.25%	[4.32, 6.22]	9.34%	58.3%	14.2
LSTM (RecurrentPPO)	7.31%	[6.10, 8.65]	12.21%	25.0%	11.0
Fixed <code>log_grid_trmatched</code>	6.04%	[5.36, 6.70]	7.81%	25.0%	10
Fixed <code>cr_optimal</code>	7.74%	[7.08, 8.42]	9.78%	8.3%	10

Table 5.7: Memory-mechanism ablation at 560 s on the five-sphere task: 24-episode strict held-out evaluations of each arm’s global-best policy, with the fixed schedules on the same seed. The TI-coverage histogram is the best result in the project; its confidence interval does not overlap any other row’s.

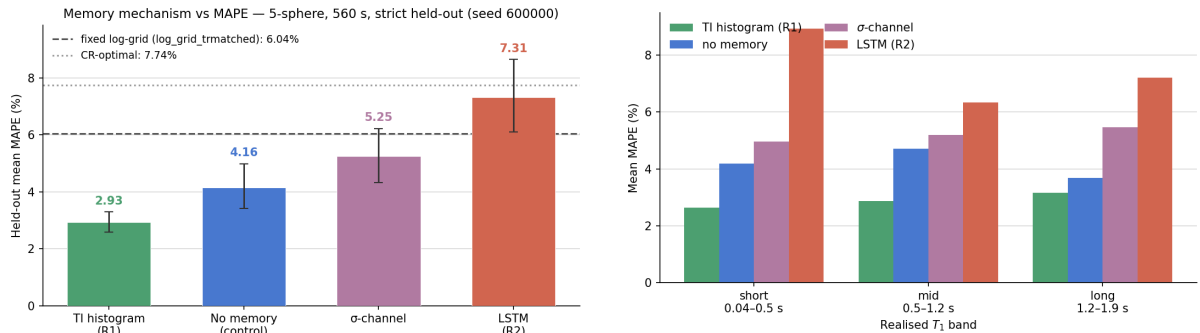


Figure 5.5: Memory ablation on the strict held-out evaluation. Left: mean MAPE with 95% CIs per mechanism; horizontal lines mark the fixed schedules on the same seed. Right: mean MAPE binned by realised T_1 (the meaningful breakdown, since each sphere is an independent draw), showing the histogram arm is the most accurate in every band.

The mechanisms rank decisively: histogram $\gg$ no memory $>$ σ -channel $>$ LSTM, and only the first two beat the best fixed schedule. The histogram policy’s 2.93% mean MAPE is the best result in this project: 1.2 percentage points better than the memoryless control and 3.1 points better than the fixed log grid, with 95.8% of episodes under 5% error. Binning the held-out errors by realised T_1 (Figure 5.5, right) shows the histogram arm is the most accurate in every band, from 2.6% at short T_1 to 3.2% at long T_1 , so the gain holds across the clinical range rather than coming from one easy regime.

The σ -channel gives no measurable benefit. Its global-best policy scores 5.25% on the strict held-out seed, a point estimate worse than the no-memory control’s 4.16% but with overlapping confidence intervals, so the two are not statistically distinguishable. A plausible mechanism is the one flagged in its design: the uncertainty proxy is degenerate before two measurements and noisy just above, so it carries little usable signal in the early probe blocks. The takeaway is that the adaptive gain in Run B does not require handing the policy an explicit uncertainty estimate.

The LSTM number carries an important compute caveat and should not be read as “recurrence is intrinsically worse”. The recurrent rollout was about $5\times$ slower per environment step (0.4–0.5 s versus 0.08–0.10 s at the full-Bloch stage), so under the shared 9 h wall-clock budget it received roughly $7\times$ fewer target-fidelity training steps (38k versus 275k), and its full-Bloch stage never improved on an early cached-stage checkpoint — which is what global-best selection returned (confirmed at 5.68%). The fair claim is that under a fixed wall-clock budget the LSTM was both slower and worse, and never trained productively at the target fidelity; a step-matched rerun is required to separate mechanism from compute. The histogram result has no such caveat: its global best is a late full-stage checkpoint whose evaluations were still improving when the budget expired (3.71% $\rightarrow$ 2.91% over the final stage evaluations).

The diagnostics show the two memory mechanisms learned qualitatively different timing rules. The LSTM learned the intuitive monotonic rule: a short fixed probe (TI $\approx$ 0.12 s) followed by rising TI ramps, strongly correlated with the running estimate ($r = +0.78$). The histogram policy’s TI is nearly uncorrelated with the running estimate ($r = -0.12$): it opens with a fixed two-block probe (TI 1.05 $\rightarrow$ 0.5 s), fans out per episode, and conditions instead on which log-TI bins remain unsampled — the channel it was given. The intuitive estimate-tracking rule, learned cleanly by the LSTM, performs worse than coverage-driven sampling. This is direct behavioural support for the section’s hypothesis: the value of the next measurement depends on which measurements have already been bought, not only on the current estimate, and an explicit coverage statistic is currently the best way to expose that dependence.

The ablation also reconfirms the global-best mechanism. The no-memory control’s final policy drifted to 7.12% despite an earlier confirmed global best near 3.45%, and the LSTM’s best policy predates its final stage entirely; only the histogram arm’s final policy stayed close to its best (3.16%). In a multi-fidelity curriculum the last policy is routinely not the best policy; checkpoint selection on the target fidelity is part of the method, not an optional extra.

5.8 Quantitative positioning against published results

Table 5.1 positioned this chapter qualitatively; Table 5.8 adds the headline numbers on each side. These are anchors, not a ranking: the tasks differ (model-based T2/MRS adaptation, offline sequence optimisation, shape classification, fitted T_1 mapping), so the numbers are not comparable across rows. What the table shows is that prior work has stronger hardware validation and, in some cases, larger demonstrated acceleration, while this chapter contributes a different experimental object: a closed-loop RL policy evaluated on fitted T_1 error against matched fixed schedules in a validated 2-D Bloch-simulation pipeline.

Work	Reported benefit	Metric	Comparability caveat
Adaptive model-based MR [8]	1.7–3.3 $\times$ simulated acceleration; 2.5 $\times$ in healthy volunteers	T2/MRS acquisition time at matched precision	Real hardware, but Bayesian model-based selection on a different estimation task
MRzero [7]	Learned sequences executed on a clinical 3 T scanner, phantom and in vivo	Target-contrast fidelity, SAR, scan time	Offline supervised optimisation; no closed-loop adaptation
Walker-Samuel [20]	99.8% shape-classification accuracy with 4–6 k-space lines	Classification accuracy vs scan time	RL scanner control, but a classification objective, not qMRI estimation
This chapter, Run A (14 spheres, 240 s)	9.44–11.68% vs 4.70% for the fixed Cramer–Rao schedule	Held-out fitted T_1 MAPE	Negative result: robust fixed coverage wins on the full plate
This chapter, Run B (5 spheres, 240 s)	4.62% vs 6.86% for the matched fixed log grid	Held-out fitted T_1 MAPE	Matched scan budget and estimator; simulator-only
This chapter, TI-histogram memory (5 spheres, 560 s)	2.93% vs 6.04% for the best fixed schedule	Held-out fitted T_1 MAPE	Best controlled result; five-sphere task, one training seed per arm

Table 5.8: Quantitative anchors for this chapter and the nearest published work. Rows are not mutually comparable because the tasks and metrics differ; this project’s own rows share one estimator, simulator, noise level, and evaluation protocol.

These rows report error at a matched scan time, not acceleration at matched precision, so they are not directly comparable to the 1.7–3.3 $\times$ (T_2) and 1.7–4 $\times$ (simulated T_1) speedups of Beracha et al. [8]. Applying their own precision-squared-to-time heuristic, the histogram policy’s improvement from 6.04%

to 2.93% MAPE (a $\approx 2.06\times$ precision ratio) corresponds to a nominal $\approx 4\times$ acceleration: at the top of their reported range, and on a harder task that maps a whole fleet rather than a single voxel.

That figure probably understates the effect. Doubling the fixed grid’s scan time barely changed its error (5.60% $\rightarrow$ 5.71% from 240 to 560 s, Section 5.6), so the fixed protocol does not reach the policy’s precision at any budget tested; the gain is then not a simple speedup but precision the fixed schedule cannot buy with more scan time, most plausibly from better information-per-block allocation under noise. Two caveats keep this honest: MAPE bundles bias and finite-SNR effects that the standard-deviation-based bound excludes, so the precision ratio is a loose analogy; and because everything here is simulated, part of the gain could be the policy exploiting regularities of the KomaMRI forward model rather than transferable physics. Both point to the same next step, benchmarking the learned policy against fixed protocols on a physical phantom, rather than discounting the in-simulation result.

5.9 Limitations

The main limitation is scope. The positive result is a proof of concept on a controlled five-sphere task, not a solved full-phantom mapping problem. That task was deliberately engineered so that adaptivity could matter: the geometry was fixed, the active tissues were few, and the relaxation values changed between episodes. This is the right experiment for isolating closed-loop timing, but it is not yet evidence that the same policy class scales to all 14 T1 spheres, the T2 and PD plates, or pose uncertainty.

The second limitation is task size. With five active spheres, the state estimate is compact and the reward signal is relatively direct. A full plate introduces a different regime: the policy must cover long and short relaxation times at once, avoid sacrificing hard pools for easy pools, and handle a larger set of coupled estimation errors. Run A shows this distinction clearly. It learned a state-dependent policy, but robust fixed coverage remained stronger.

The third limitation is statistical breadth. The final tables use 24-episode evaluations and one training seed per arm. This is adequate for the current conclusion: Run A is a strict held-out negative result, and Run B and the memory ablation are controlled positive results with non-overlapping confidence intervals. It is not enough to claim that the method is insensitive to PPO initialisation, curriculum timing, or stochastic simulator noise. A related gap is that the ROI-averaging radius introduced for Run B was never ablated in isolation; the RL-versus-fixed comparisons are unaffected because every policy and baseline uses the same measurement extraction, but absolute MAPE levels should not be compared across runs with different ROI settings.

The fourth limitation is algorithmic breadth. PPO was a pragmatic baseline, not a claim that on-policy learning is the best optimiser for this problem. Because simulator samples are expensive, an off-policy actor-critic such as SAC could be a better fit: replay would reuse costly transitions and might reduce the number of simulator calls needed to reach a useful policy. That comparison was not run, so the results should be read as evidence that adaptive qMRI can be learned with a robust baseline, not as evidence that PPO is the most sample-efficient or highest-performing RL method for the task.

The fifth limitation is external validation. The original project plan included testing the learned protocol on real scanner hardware, but this was not reached. All results in this chapter are therefore simulator results. They test whether closed-loop sequence design can work inside the KomaMRI-based digital twin; they do not yet show that the learned timings transfer to scanner constraints, sequence programming details, reconstruction pipelines, or real phantom data. The policy is also trained at a single field strength (1.5 T) and one calibrated noise level; since T_1 is field-dependent and SNR varies with coil and hardware, transfer across fields or SNR regimes would likely need retraining or domain randomisation over both.

A related geometric simplification is the slice thickness. To keep the spin count tractable, the reported runs image a single 1 mm axial slab through the T1 plate, whereas a real slice-selective acquisition would excite a 3–8 mm slice and so average over more through-plane water and partial-volume material. This makes the simulated scene cleaner than a clinical slice. The choice is not believed to change the qualitative result: experiments with thicker slabs gave similar T_1 -fit-versus-truth behaviour, because the fitted quantity depends on the per-sphere recovery curve rather than on the absolute spin count, and

the noise level is recalibrated to the geometry in any case (Appendix A.6). A thicker slice mainly raises simulation cost and partial-volume blurring, both of which the multi-fidelity machinery is designed to absorb, but a slice-selective study at clinical thickness remains future work.

A further limitation is analytical: the achievable error floor is not pinned down, so it is unclear how much of the residual MAPE is the sequence design and how much is the estimator, the noise and the simulator itself. A noise-free calibration with a finer 500-point fitter grid reached $\approx 0.48\%$ mean MAPE across the 14 spheres, already well below the $\approx 1\%$ quantisation floor of the coarser 300-point grid the RL fitter uses (Chapter 3); adopting the same 500-point grid in the RL fitter would remove that floor at negligible cost. That residual is largely insensitive to the forward model’s perfect-spoiling assumption (modelling explicit crusher gradients only moves it from 0.48% to 0.43%), so it most likely reflects Bloch time discretisation in the simulator rather than a fundamental estimator floor. Under realistic noise there is then limited headroom above this floor for any sequence-design method, and establishing it directly (a Cramér–Rao bound at the true T_1 , or an oracle-grid fit) is missing here. Relatedly, the ablations cleanly attribute the gain to adaptivity and acquisition history, but a deeper mechanistic account of what the policy exploits, and why short and long T_1 behave differently, would strengthen the claim.

A scope limitation concerns localisation. The agent reads per-sphere ROI signals extracted at the spheres’ *known* pixel positions, so although pose is randomised each episode the policy is only made robust to that variation; it never has to find the spheres. The twin’s pose sampler sets up genuinely unknown-pose operation, but joint localisation-and-mapping is not attempted here.

The multi-fidelity speedup is also structural rather than general. The analytic surrogate and the cached- and coarsened-water rungs exist only because the background water is a separable, homogeneous component that can be cached and down-sampled (Section 5.4.2); a problem without such a cheap, faithful approximation would not gain the $\sim 30\times$ ladder. The efficiency contribution is therefore a recipe that exploits problem structure, not a black-box accelerator.

Because reward flows through one fitter, the policy is also co-adapted to that estimator and could exploit its quirks rather than the underlying physics — the failure mode behind the degenerate E1 policy. The reported gain is thus partly specific to this fitter; a different estimator, such as the EPG or learned fit suggested in Chapter 3, could shift the optimal policy.

The policy is also far less interpretable than the model-based alternative. It is a black box that emits point T_1 estimates with no calibrated uncertainty, and the one mechanism that tried to expose uncertainty — the fitter- σ channel (Section 5.7) — gave no measurable benefit. The Bayesian scheme of Beracha et al. [8] instead carries an explicit posterior $p(T_1, PD)$ throughout the scan, so it yields per-voxel standard deviations and credible intervals for free, along with a principled stopping rule (halt once the posterior is narrow enough). The adaptive gain here therefore comes at the cost of the interpretability and uncertainty quantification a model-based prior provides intrinsically — the trade-off the greedy-Cramér–Rao baseline of Section 5.10 is designed to probe.

Finally, the action space is still narrow. The current policy tunes an IR-SE-style block, mostly through TI and TR. It cannot choose a different sequence family, change the phase-encode budget, or stop early to trade accuracy against scan time, and it carries no hardware constraints — SAR, gradient slew or B_1^+ limits — that a deployable adaptive sequence would have to respect. The chapter should therefore be read as an adaptive T1 timing study, not as a general autonomous MRI protocol designer.

5.10 Future work

The next algorithmic step is to improve the curriculum controller. Run A showed that `full3` can be a biased rung; Run B showed that global-best checkpointing protects against late drift. A simpler ladder,

$$\text{analytic} \rightarrow \text{cached} \rightarrow \text{full},$$

should be tested against the current five-rung ladder. A more ambitious version would sample from several fidelities in parallel rather than treating the ladder as a serial hand-off. In the current runs, the GPU is largely idle during the analytic and cached stages; a parallel design could keep full-Bloch probes

running while the cheap simulators generate high-throughput rollouts. Longer term, this becomes pooled multi-fidelity learning: cheap and expensive rollouts are used together with bias correction or control variates. This direction is directly connected to multi-fidelity control-variate and Gaussian-process RL methods [39], [40].

The state-representation question is now partly settled empirically: the executed-TI coverage histogram is the best observed memory mechanism, and the fitter’s own σ estimate is not a reliable summary. The open question is the recurrent alternative. The LSTM arm lost under a fixed wall-clock budget, but it was $\approx 5\times$ slower per step and consequently undertrained at the target fidelity, so a step-matched RecurrentPPO rerun is needed before concluding that recurrence is intrinsically worse. Extending the coverage idea to richer action spaces — joint TI–TR coverage, or coverage per estimated- T_1 band — is the natural next encoding.

The most important scientific extension is to widen the action space and repeat the experiment beyond the T_1 plate. First, the same adaptive framework should be tested with different sequence families: inversion-recovery spin echo, ordinary spin echo, multi-spin-echo trains, and possibly spoiled-GRE-like readouts. In parallel, the target should expand from the T_1 spheres to the T_2 and PD plates, where the informative acquisition parameters are different. The next step is a combined multi-parameter task in which the policy chooses not only TI and TR, but also echo time, echo-train structure, flip angle, and possibly readout budget. That would test a more interesting adaptive question: whether the next block should reduce T_1 uncertainty, T_2 uncertainty, PD uncertainty, or spatial uncertainty. This is closer to magnetic resonance fingerprinting, where sequence schedules are designed to make tissue trajectories distinguishable [15], [16]. It also requires a more general fitter or dictionary model, because the current fitter assumes one generalized IR spin-echo signal family. Extending the target to T_2 and T_2^* also brings off-resonance into play: the twin already stores a per-spin $\Delta\omega$ channel and an augmentation knob for it (Chapter 3), so a natural extension is a policy that must capture and fit off-resonance jointly with relaxation, rather than treating it as fixed nuisance.

Two further extensions would close the gap to a realistic acquisition. First, the reported runs use the hard-pulse approximation — RF treated as an instantaneous rotation. Replacing the hard excitation and refocusing pulses with shaped slice-selective pulses (for example a windowed-sinc profile) under a slice-select gradient would make the simulated acquisition match a real slice-selective sequence, at the cost of finer RF time-stepping; this is also the regime where the KomaMRI timing fixes of Chapter 4 matter most, because shaped pulses spread the RF area over many samples. Second, the experiments image a single thin axial slab; generalising to genuine 3-D slice selection — exciting and encoding a stack of slices, or a slab with through-plane phase encoding — would let the policy trade through-plane coverage against scan time, and would exercise the twin’s anisotropic water grid and $SO(3)$ pose sampler that are implemented but unused here.

The objective could also become a Pareto curve rather than one fixed scan time. In clinical use, a protocol that reaches 5% MAPE in 120 s may be more valuable than one that reaches 3.5% in 240 s. A stop action or explicit scan-time penalty would let the policy choose this trade-off, while fixed baselines can be compared across the same accuracy–time frontier.

A practical prerequisite for deployment is real-time feasibility. Each block’s fit (a Levenberg–Marquardt or grid scan) and the policy’s forward pass must complete within the inter-block interval for the loop to adapt *online* on a scanner, rather than being replayed offline as it is here. The model itself is small: its forward pass measures ≈ 0.3 ms on CPU (sub-millisecond), against acquisitions that run for minutes, so policy inference is nowhere near the bottleneck. What has not been validated is the rest of the pipeline — reconstruction and fitting between blocks — running inside that timing budget end to end on real hardware.

The broadest direction is clinical scoping. This chapter is deliberately general proof-of-concept work: it shows that closed-loop timing can reduce T_1 MAPE on a calibration phantom, but it has not been mapped to a specific clinical qMRI acquisition where that reduction would matter, nor optimised for one. The natural progression is to move from the calibration phantom to a digital twin of patient anatomy — realistic tissue distributions, partial-volume boundaries and motion — and test whether the MAPE reduction holds there, and ultimately whether it translates into better downstream outcomes such as sharper tissue characterisation or shorter clinical scans. Only at that point does the method stop being a proof of concept and become a candidate acquisition strategy.

One baseline this chapter identifies but does not implement would sharpen the attribution of the gain: a per-sphere Bayesian posterior with greedy Cramér–Rao re-targeting, the natural bridge between the static CR baseline and the learned policy. Each sphere would carry its own grid posterior $p(T_1, \text{PD})$, updated from its ROI signal after every block exactly as the fitter already evaluates it, and the next (TI, TR) block would be chosen by re-minimising the fleet CRLB at the current posterior *means* rather than a fixed nominal fleet. Because one block images all spheres at once, the per-sphere optima must be aggregated into a single action — precisely the fleet-averaging the static objective already performs, but now over the sharpening estimates. Such a scheme would re-target the realised T_1 values like the RL policy, would be fully interpretable, and would supply the calibrated uncertainty and natural stopping criterion (halt when every sphere’s posterior SD falls below a threshold) that the σ -channel of Section 5.7 failed to provide. It would also isolate *why* reinforcement learning helps: greedy CRLB selection is myopic, optimising only the next block, whereas the PPO policy optimises the whole-episode return and demonstrably learns a non-myopic probe-then-refine strategy (Section 5.6). Whether that planning advantage survives against a strong greedy-adaptive baseline — and, paired with a TR matched to the policy’s realised mean, whether the gain is TI placement or block count — is the cleanest open question left by this chapter.

5.11 Summary

This chapter produced three conclusions. First, multi-fidelity training is a necessary engineering mechanism for simulation-in-the-loop RL with KomaMRI. It made the experiments feasible and exposed where simulator bias affects policy selection. Second, adaptivity only showed a clear quantitative advantage on the task designed to require it. The full 14-sphere agent learned a state-dependent policy but lost to fixed schedules; the five-sphere continuous- T_1 agent beat the fixed log-grid comparator at 240 s and 560 s on strict held-out episodes, using a compact probe-then-refine strategy. Third, what the policy remembers matters as much as whether it adapts: an order-invariant histogram of executed inversion times produced the chapter’s best result (2.93% versus 6.04% for the best fixed schedule), while a fitter-uncertainty channel and a recurrent policy both underperformed the memoryless control — the latter confounded by its $\approx 5\times$ per-step cost under a shared wall-clock budget. The limitations above define the work needed to turn these controlled demonstrations into a robust qMRI sequence-design method.

Summary

This project asked whether reinforcement learning can design adaptive quantitative MRI sequences that estimate tissue relaxation more efficiently than fixed clinical protocols, and built the full simulation-in-the-loop pipeline needed to test that question honestly. This chapter draws the three technical chapters together: what was achieved against the challenges set out in Chapter 1, the limitations that bound those achievements, and the work that would take the project further.

5.12 Achievements

The three challenges (C1, simulator validation for qMRI; C2, computational efficiency under expensive simulation; C3, adaptive sequence design from tissue-parameter estimates) map onto three achievements, one per technical chapter.

A1 — a digital twin of the calibration phantom (Chapter 3). `MRISystemPhantom.jl` is a configurable, open-source digital twin of the QalibreMD Model 130 system phantom that returns standard `KomaMRI.Photom` objects, ships a tested forward pipeline (phantom construction, sequence generation, reconstruction, ROI extraction, and fitting), and exposes the randomisation hooks the RL loop needs. It is the ground-truth foundation for C1 and a reusable contribution in its own right: a digital twin of a widely deployed calibration device against which simulated qMRI methods can be checked before scanner deployment.

A2 — a validated Bloch-simulation pipeline (Chapter 4). Validating that pipeline against the phantom’s known T_1 values — the strict test that distinguishes qMRI from plausible imaging — exposed two floating-point time-discretisation failures in `KomaMRI` that only appear at the long cumulative sequence times RL training demands. Both were reduced to minimal reproducers, traced to specific ϵ -scale timing mechanisms, and fixed upstream; the no-noise 14-sphere T_1 fit improved from 39.4% to 0.48% mean MAPE. Together with A1 this answers C1, and the fixes are a contribution back to the wider simulation community, not only to this project.

A3 — adaptive qMRI sequence design via multi-fidelity RL (Chapter 5). A full-Bloch step at the target configuration measured ≈ 4 s on CPU, making a single-fidelity run roughly nine days before evaluation. The multi-fidelity curriculum answers C2 by learning cheaply on an analytic surrogate and cached or coarsened water, then warm-starting up a fidelity ladder spanning a $\sim 30\times$ per-step cost range, with promotion gated on held-out *full-simulator* validation so a biased rung cannot certify its own policy. On the adaptive question (C3), the learned closed-loop policy beat the matched fixed log-grid schedule at two scan budgets on strict held-out episodes (4.62% versus 6.86% MAPE at 240 s; 4.16% versus 6.04% at 560 s), with non-overlapping confidence intervals. A memory ablation then produced the project’s strongest result: giving the policy an order-invariant histogram of the inversion times it has already executed reached 2.93% MAPE — better than the memoryless control, a fitter-uncertainty channel, a recurrent policy, and every fixed schedule, on every active sphere. The value of the next acquisition depends on what has already been measured, and exposing that history explicitly matters.

5.13 Limitations

Digital twin (Chapter 3). The twin models the three contrast plates and fiducial grid but not the resolution insets or slice-profile wedges, and the sphere-ring geometry matches the manual photographs rather than a measured device. Two material entries are approximate: PD-plate relaxation values use bulk water, and the fiducial relaxation values are generic placeholders. Reference values are fixed at 20 °C, with the manual’s temperature corrections unimplemented. None of these affect the T_1 - and T_2 -plate experiments reported here, but all must be addressed before any simulation-to-real comparison involving

the PD plate or fiducials. Coarsening the background water also perturbs the boundary geometry, a bias quantified rather than eliminated.

Simulator validation (Chapter 4). The validation covers the IR-SE sequence family and an idealised scanner: RF pulses are exact and field inhomogeneities and eddy currents are not modelled. Other sequence families would need the same timing and no-noise recovery checks before being trusted for quantitative claims, and the second timing fix (PR #789) remains open upstream, pinned in a project fork.

Adaptive RL (Chapter 5). The positive result is a proof of concept on a deliberately simplified task, not solved full-phantom mapping: on the full 14-sphere plate the agent learned a genuine state-dependent policy but still lost to robust fixed coverage, because that task rewards global experimental design over per-episode adaptation. The evaluations rest on 24 episodes and one training seed per arm, so they do not bound sensitivity to initialisation or curriculum timing; the recurrent-policy negative is further confounded by its $\approx 5\times$ per-step cost under a shared wall-clock budget. The action space is narrow (one IR-SE block, mostly TI and TR), the agent acts only on fitted estimates rather than images, and every result is simulator-only at a single 1 mm slab — the planned MR-Linac hardware validation was not reached.

5.14 Future work

The most immediate extensions are concrete. For the twin (Chapter 3), a boundary-aware water grid, verified PD/fiducial values, and temperature correction of the relaxation tables would remove the known biases and unlock the PD plate and any sim-to-real comparison; modelling the omitted resolution insets and slice-profile wedges would extend it to geometric-distortion and slice-profile experiments. The twin’s architecture also already separates labelled material descriptors from voxelisation, so other calibration phantoms — including the open-hardware designs of Section 3.3 — could be supported by writing new descriptor builders while reusing the voxelisation, randomisation, reconstruction, and fitting pipeline, turning the single-phantom twin into a small framework for executable qMRI phantom twins. For the simulator validation (Chapter 4), the parameter-recovery test should be repeated for the other shipped sequence families (spin echo, multi-echo, turbo spin echo, spoiled GRE) before any of them carries quantitative claims, the still-open PR #789 should be landed upstream, and the idealised-scanner assumption should be relaxed to include RF inhomogeneity, eddy currents, and B_0 imperfections. For the RL environment (Chapter 5), a step-matched recurrent-policy rerun would separate the memory mechanism from its compute cost, and a simpler `analytic`→`cached`→`full` ladder should be tested against the biased `full3` rung.

The scientifically important direction is to widen the action space and move beyond the T_1 plate. Letting the agent choose the acquisition family (IR-SE, spin echo, multi-echo trains, spoiled GRE) and the relevant per-family parameters would test the more publishable claim that RL’s advantage is in choosing *which contrast mechanism* to deploy, and would extend naturally to joint $T_1/T_2/T_2^*$ and off-resonance estimation — closer to magnetic resonance fingerprinting. Two realism steps accompany this: replacing the hard-pulse approximation with shaped slice-selective (windowed-sinc) pulses, and generalising the single thin slab to genuine 3-D slice selection, both of which the twin already supports but the experiments do not yet exercise. Replacing the single fixed scan budget with an accuracy–time Pareto objective, via a stop action or scan-time penalty, would reflect the real clinical trade-off, where reaching adequate accuracy sooner can matter more than a lower asymptotic error. Finally, executing a learned protocol on the MR-Linac would close the sim-to-real gap the project was ultimately motivated by, testing whether the learned timings survive real scanner constraints, reconstruction, and phantom data.

5.15 Concluding remarks

The project delivered a validated, reusable digital twin; surfaced and fixed real numerical bugs in a widely used simulator; and built a bias-aware multi-fidelity training loop that made simulation-in-the-loop qMRI RL tractable. Within that pipeline it showed, on a controlled task, that an adaptive policy

with an explicit memory of its own acquisitions can beat strong fixed protocols. The honest boundary of the work is equally clear: adaptivity paid only where the task was constructed to require it, and every result remains simulated.

The wider context is the one set out in Chapter 1: MRI demand is growing faster than capacity, and adaptive acquisition attacks the scan-time bottleneck at the measurement-design level rather than after the data are collected. Within MRI digital-twin research, which remains concentrated on diagnosis and treatment planning, protocol- and infrastructure-level twins are the underdeveloped fraction, held back by simulation cost and validation [19] — precisely the two barriers the multi-fidelity ladder and the parameter-recovery validation address in this bounded calibration setting. The phantom twin, validated simulator, and training loop together form a testbed in which adaptive qMRI protocols can be developed and compared before any patient or scanner is involved. Turning this controlled demonstration into a robust, hardware-validated qMRI sequence designer is the natural continuation, and the infrastructure built here is intended to make that continuation straightforward.

Bibliography

- [1] W. C. R. Fund, *UK cancer statistics*, en-GB. [Online]. Available: <https://www.wcrf.org/preventing-cancer/cancer-statistics/uk-cancer-statistics/> (visited on 01/21/2026).
- [2] R. Atun, D. A. Jaffray, and M. B. e. a. Barton, "Expanding global access to radiotherapy," *The Lancet Oncology*, vol. 16, no. 10, pp. 1153–1186, Sep. 2015. DOI: 10.1016/s1470-2045(15)00222-3. [Online]. Available: [https://doi.org/10.1016/s1470-2045\(15\)00222-3](https://doi.org/10.1016/s1470-2045(15)00222-3).
- [3] X. Liu, Z. Li, and Y. Yin, "Clinical application of MR-Linac in tumor radiotherapy: A systematic review," *Radiation Oncology*, vol. 18, no. 1, p. 52, Mar. 2023, ISSN: 1748-717X. DOI: 10.1186/s13014-023-02221-8. [Online]. Available: <https://doi.org/10.1186/s13014-023-02221-8>.
- [4] J. de Leon, T. Twentyman, M. Carr, M. Jameson, and V. Batumalai, "Optimising the mr-linac as a standard treatment modality," *Journal of Medical Radiation Sciences*, vol. 70, no. 4, pp. 491–497, 2023. DOI: <https://doi.org/10.1002/jmrs.712>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/jmrs.712>. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/jmrs.712>.
- [5] R. Heckel, M. Jacob, A. Chaudhari, O. Perlman, and E. Shimron, "Deep learning for accelerated and robust MRI reconstruction," *Magnetic Resonance Materials in Physics, Biology and Medicine*, vol. 37, no. 3, pp. 335–368, Jul. 2024, ISSN: 1352-8661. DOI: 10.1007/s10334-024-01173-8. [Online]. Available: <https://doi.org/10.1007/s10334-024-01173-8>.
- [6] B. Zhu, J. Liu, N. Koonjoo, B. R. Rosen, and M. S. Rosen, "Automated pulse sequence generation (autoseq) using bayesian reinforcement learning in an mri physics simulation environment," in *Proceedings of the Joint Annual Meeting ISMRM-ESMRMB*, 2018, p. 0438.
- [7] A. Loktyushin, K. Herz, N. Dang, *et al.*, "Mrzero - automated discovery of mri sequences using supervised learning," *Magnetic Resonance in Medicine*, vol. 86, no. 2, pp. 709–724, 2021. DOI: <https://doi.org/10.1002/mrm.28727>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/mrm.28727>. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/mrm.28727>.
- [8] I. Beracha, A. Seginer, and A. Tal, "Adaptive model-based magnetic resonance," *Magnetic Resonance in Medicine*, vol. 90, no. 3, pp. 839–851, 2023. DOI: <https://doi.org/10.1002/mrm.29688>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/mrm.29688>. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/mrm.29688>.
- [9] D. B. Plewes and W. Kucharczyk, "Physics of mri: A primer," *Journal of Magnetic Resonance Imaging*, vol. 35, no. 5, pp. 1038–1054, 2012. DOI: <https://doi.org/10.1002/jmri.23642>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/jmri.23642>. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/jmri.23642>.
- [10] R.-J. M. van Geuns, P. A. Wielopolski, H. G. de Bruin, *et al.*, "Basic principles of magnetic resonance imaging," *Progress in Cardiovascular Diseases*, vol. 42, no. 2, pp. 149–156, 1999. DOI: [https://doi.org/10.1016/S0033-0620\(99\)70014-9](https://doi.org/10.1016/S0033-0620(99)70014-9). [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0033062099700149>.
- [11] R. B. Buxton, "Mapping the mr signal," in *Introduction to Functional Magnetic Resonance Imaging: Principles and Techniques*. Cambridge University Press, 2009, pp. 205–231.
- [12] T. Stöcker, K. Vahedipour, D. Pflugfelder, and N. J. Shah, "High-performance computing MRI simulations," *Magnetic Resonance in Medicine*, vol. 64, no. 1, pp. 186–193, 2010. DOI: 10.1002/mrm.22406.
- [13] F. Liu, J. V. Velikina, W. F. Block, R. Kijowski, and A. A. Samsonov, "Fast realistic MRI simulations based on generalized multi-pool exchange tissue model," *IEEE Transactions on Medical Imaging*, vol. 36, no. 2, pp. 527–537, 2017. DOI: 10.1109/TMI.2016.2620961.

- [14] C. Castillo-Passi, R. Coronado, G. Varela-Mattatall, C. Alberola-López, R. Botnar, and P. Irarrazaval, “Komamri.jl: An open-source framework for general mri simulations with gpu acceleration,” *Magnetic Resonance in Medicine*, vol. 90, no. 1, pp. 329–342, 2023. DOI: <https://doi.org/10.1002/mrm.29635>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/mrm.29635>. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/mrm.29635>.
- [15] B. Mehta, S. Coppo, D. F. McGivney, *et al.*, “Magnetic resonance fingerprinting: A technical review,” *Magnetic Resonance in Medicine*, vol. 81, no. 1, pp. 25–46, 2019. DOI: 10.1002/mrm.27403.
- [16] S. P. Jordan, S. Hu, I. Rozada, *et al.*, “Automated design of pulse sequences for magnetic resonance fingerprinting using physics-inspired optimization,” *Proceedings of the National Academy of Sciences*, vol. 118, no. 40, e2020516118, 2021. DOI: 10.1073/pnas.2020516118. eprint: <https://www.pnas.org/content/118/40/e2020516118.full.pdf>. [Online]. Available: <https://www.pnas.org/content/118/40/e2020516118>.
- [17] K. E. Keenan, K. V. Jordanova, S. E. Ogier, *et al.*, “Phantoms for quantitative body MRI: A review and discussion of the phantom value,” *Magnetic Resonance Materials in Physics, Biology and Medicine*, vol. 37, no. 4, pp. 535–549, 2024. DOI: 10.1007/s10334-024-01181-8.
- [18] S. E. Russek, M. A. Boss, H. C. Charles, *et al.*, “MRI scanner characterization with the ISMRM/NIST system phantom,” in *Proceedings of the International Society for Magnetic Resonance in Medicine*, 2020. [Online]. Available: <https://cds.ismr.org/protected/20MProceedings/PDFfiles/3384.html>.
- [19] J. Greggio, N. Stogiannos, K. L. Stewart, *et al.*, “Exploring digital twinning in MRI: A systematic review of current applications, barriers, and future opportunities,” *Radiography*, vol. 32, p. 103 413, 2026. DOI: 10.1016/j.radi.2026.103413. [Online]. Available: <https://doi.org/10.1016/j.radi.2026.103413>.
- [20] S. Walker-Samuel, “Using deep reinforcement learning to actively, adaptively and autonomously control a simulated mri scanner,” in *Proceedings of the Joint Annual Meeting ISMRM-ESMRMB*, 2019, p. 0473. [Online]. Available: <https://cds.ismr.org/protected/19MProceedings/PDFfiles/0473.html>.
- [21] L. Pineda, S. Basu, A. Romero, R. Calandra, and M. Drozdal, “Active MR k-space sampling with reinforcement learning,” in *Medical Image Computing and Computer Assisted Intervention – MICCAI 2020*, Springer, 2020, pp. 23–33. DOI: 10.1007/978-3-030-59713-9_3.
- [22] A. Ocanto, L. Torres, M. Montijano, *et al.*, “MR-LINAC, a New Partner in Radiation Oncology: Current Landscape,” *Cancers*, vol. 16, no. 2, p. 270, Jan. 2024, ISSN: 2072-6694. DOI: 10.3390/cancers16020270. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10813892/> (visited on 01/20/2026).
- [23] A. Kuisma, I. Ranta, J. Keyriläinen, *et al.*, “Validation of automated magnetic resonance image segmentation for radiation therapy planning in prostate cancer,” *Physics and Imaging in Radiation Oncology*, vol. 13, pp. 14–20, 2020, ISSN: 2405-6316. DOI: <https://doi.org/10.1016/j.phro.2020.02.004>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S240563162030004X>.
- [24] A. D. Elster, *Magnetism*, en. [Online]. Available: <http://mriquestions.com/ai-in-clinical-mri.html> (visited on 01/20/2026).
- [25] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018. [Online]. Available: <http://incompleteideas.net/book/the-book-2nd.html>.
- [26] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017. DOI: 10.48550/arXiv.1707.06347. arXiv: 1707.06347. [Online]. Available: <https://arxiv.org/abs/1707.06347>.
- [27] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” in *International Conference on Learning Representations*, 2016. DOI: 10.48550/arXiv.1506.02438. arXiv: 1506.02438. [Online]. Available: <https://arxiv.org/abs/1506.02438>.
- [28] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. DOI: 10.1162/neco.1997.9.8.1735.
- [29] MRI Standards, *PhantomViewer: Python analysis package for MRI phantoms*, Accessed 11 June 2026, 2013. [Online]. Available: <https://github.com/MRIStandards/PhantomViewer>.

- [30] J. C. Korte, Z. Chin, M. Carr, L. Holloway, and R. D. Franich, “Magnetic resonance biomarker assessment software (MR-BIAS): An automated open-source tool for the ISMRM/NIST system phantom,” *Physics in Medicine & Biology*, 2023, Software: <https://github.com/JamesCKorte/mrbias>. DOI: 10.1088/1361-6560/acbcbb.
- [31] S. Pasini, S. Ringgaard, T. Vendelboe, *et al.*, “Multi-center and multi-vendor evaluation study across 1.5 T and 3 T scanners (part 2): T1 and T2 standardization in the ISMRM/NIST MR phantom,” *Magnetic Resonance Materials in Physics, Biology and Medicine*, vol. 38, no. 3, pp. 611–627, 2025. DOI: 10.1007/s10334-025-01260-4.
- [32] M. Besançon, T. Papamarkou, D. Anthoff, *et al.*, “Distributions.jl: Definition and modeling of probability distributions in the juliastats ecosystem,” *Journal of Statistical Software*, vol. 98, no. 16, pp. 1–30, 2021. DOI: 10.18637/jss.v098.i16. [Online]. Available: <https://www.jstatsoft.org/article/view/v098i16>.
- [33] J. A. Jones, P. Hodgkinson, A. L. Barker, and P. J. Hore, “Optimal sampling strategies for the measurement of spin–spin relaxation times,” *Journal of Magnetic Resonance, Series B*, vol. 113, no. 1, pp. 25–34, 1996, Cramér–Rao analysis showing the information-optimal sampling pattern for an exponential decay is a few carefully placed points (e.g. one at zero time, the rest near 1.28 T), the foundation for CRLB-based sequence optimisation. DOI: 10.1006/jmrb.1996.0151.
- [34] T. Knopp and M. Grosser, “MRIReco.jl: An MRI reconstruction framework written in Julia,” *Magnetic Resonance in Medicine*, vol. 86, no. 3, pp. 1633–1646, 2021. DOI: 10.1002/mrm.28792.
- [35] C.-H. Sohn, R. J. Sevik, R. Frayne, H.-W. Chang, S.-P. Kim, and D.-K. Kim, “Fluid attenuated inversion recovery (flair) imaging of the normal brain: Comparisons between under the conditions of 3.0 tesla and 1.5 tesla,” *Korean Journal of Radiology*, vol. 11, no. 1, pp. 19–24, 2010. DOI: 10.3348/kjr.2010.11.1.19. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC2799645/>.
- [36] K. Murphy, J. Bodurka, and P. A. Bandettini, “How long to scan? the relationship between fmri temporal signal to noise ratio and necessary scan duration,” *NeuroImage*, vol. 34, no. 2, pp. 565–574, 2007. DOI: 10.1016/j.neuroimage.2006.09.032. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC2223273/>.
- [37] H. Sifaou and O. Simeone, *Multi-fidelity hybrid reinforcement learning via information gain maximization*, 2025. arXiv: 2509.14848 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2509.14848>.
- [38] M. Cutler, T. J. Walsh, and J. P. How, “Real-world reinforcement learning via multifidelity simulators,” *IEEE Transactions on Robotics*, vol. 31, no. 3, pp. 655–671, 2015. DOI: 10.1109/TRO.2015.2419431.
- [39] S. Khairy and P. Balaprakash, “Multi-fidelity reinforcement learning with control variates,” *Neurocomputing*, vol. 597, p. 127963, 2024. DOI: 10.1016/j.neucom.2024.127963.
- [40] V. Suryan, N. Gondhalekar, and P. Tokekar, “Multi-fidelity reinforcement learning with gaussian processes: Model-based and model-free algorithms,” *IEEE Robotics & Automation Magazine*, vol. 27, no. 2, pp. 117–128, 2020. DOI: 10.1109/MRA.2020.2988800.
- [41] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, “Policy gradient methods for reinforcement learning with function approximation,” in *Advances in Neural Information Processing Systems*, vol. 12, 1999. [Online]. Available: <https://papers.nips.cc/paper/1713-policy-gradient-methods-for-reinforcement-learning-with-function-approximation>.
- [42] R. J. Williams, “Simple statistical gradient-following algorithms for connectionist reinforcement learning,” *Machine Learning*, vol. 8, pp. 229–256, 1992. DOI: 10.1007/BF00992696. [Online]. Available: <https://doi.org/10.1007/BF00992696>.
- [43] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, “Trust region policy optimization,” in *Proceedings of the 32nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 37, PMLR, 2015, pp. 1889–1897. [Online]. Available: <https://proceedings.mlr.press/v37/schulman15.html>.

Declarations

Use of Generative AI

Generative AI tools were used during this project as assistive tools. The tools used were OpenAI ChatGPT/Codex (OpenAI, <https://openai.com/>) and Claude Opus/Sonnet (Anthropic, <https://www.anthropic.com/claude>).

For the codebase, generative AI was used for code navigation, implementation suggestions, code generation, debugging suggestions, refactoring support, and test or diagnostic-script drafting. For the report, generative AI was used for planning, drafting support, proofreading, wording suggestions, structural feedback, and LaTeX editing.

Generative AI outputs were not accepted as authoritative sources of fact. All technical claims, citations, experimental results, figures, report wording, and committed source code were reviewed, checked, and edited by the author before submission. The author remains responsible for the correctness, originality, and academic integrity of the submitted work.

Ethical Considerations

This project used simulated MRI acquisitions, digital phantoms, and physical-phantom-oriented sequence-design methods. It did not involve human participants, patient images, identifiable personal data, protected health information, or animal subjects. The ethics review with the project supervisor therefore identified no requirement for formal human-subject ethical approval for the work reported here.

The main ethical risks considered were clinical safety, hardware safety, and the risk of over-claiming from simulation-only results. These were mitigated by treating the reinforcement-learning policy as a research prototype rather than a deployable clinical controller; constraining sequence parameters to physically plausible ranges; validating against known phantom ground-truth relaxation values; and reporting limitations, failure modes, and simulator-fidelity assumptions explicitly. No learned policy was deployed on an MR-Linac or clinical scanner during this project. If future work tests such policies on scanner hardware, it would require local MR safety approval, vendor and site constraints, and supervision by qualified MR staff, with explicit limits on RF power, SAR, gradient switching, and related hardware-safety parameters.

Sustainability

The project was conducted with attention to computational efficiency. Training and evaluation were staged from low-cost diagnostic experiments to more expensive simulator runs, so that faults in reward design, action spaces, and Julia–Python integration could be identified before long training jobs were launched. The implementation reuses simulator state where possible, uses reduced-resolution training configurations for reinforcement learning, caches repeated calculations where appropriate, and reserves higher-fidelity simulation for validation and final figures.

This reduced unnecessary compute time and energy use while preserving the evidence needed for the report. Training and larger experiments used Imperial-provided GPU resources hosted in a data centre, believed to be located in Poland, while experiments were only repeated when they addressed a concrete validation question, such as noise robustness, baseline comparison, or simulator-fidelity transfer. During report and code-assistance workflows, prompts were kept targeted and iterative context was managed to reduce unnecessary generative-AI token usage.

Availability of Data and Materials

The source code, experiment scripts, LaTeX report source, generated figures, raw outputs, trained policies, and training artefacts used for this project are held in the public development repositories `RLxMRI` and `MRISystemPhantom.jl`. The reinforcement-learning and report repository is available as a public GitHub repository. The digital-phantom Julia package is available as a public GitHub repository, with package documentation. The principal materials are included in those repositories: report source files are in `report_latex/`, simulation and reinforcement-learning code is in `julia/` and `python/`, and selected outputs are in `runs/` and `report_plots/`.

A

Technical Derivation Notes

This appendix records technical derivations that are useful for understanding the library implementation, but too detailed for the main digital-twin chapter. The first sections derive the fitting models from the Bloch-equation free-precession solutions under two assumptions: RF pulses are treated as instantaneous rotations, and residual transverse magnetisation is spoiled before the next block. Under those assumptions the fitters only need to carry the scalar longitudinal magnetisation M_z between shots.

A.1 Spin-Echo T2 Fit

For a spin echo, the 90° pulse tips equilibrium magnetisation into the transverse plane. The 180° pulse refocuses reversible static dephasing, so the echo magnitude is governed by irreversible T2 decay:

$$|S(\text{TE})| = S_0 e^{-\text{TE}/T_2}. \quad (\text{A.1})$$

Taking logs gives a straight-line model,

$$\log |S| = \log S_0 - \frac{\text{TE}}{T_2}, \quad (\text{A.2})$$

which is the complete model used by `fit_t2_se`.

A.2 Inversion-Recovery T1 Fit

For long-TR inversion recovery, a preparation pulse of angle θ_{inv} leaves $M_z = M_0 \cos \theta_{\text{inv}}$. Recovery for inversion time TI therefore gives

$$M_z(\text{TI}) = M_0 \left[1 - (1 - \cos \theta_{\text{inv}}) e^{-\text{TI}/T_1} \right]. \quad (\text{A.3})$$

For a textbook 180° inversion, this reduces to $M_0(1 - 2e^{-\text{TI}/T_1})$. Magnitude reconstruction removes the sign around the null point, so the practical conventional estimator fits

$$|S| = \left| A - B e^{-\text{TI}/T_1} \right|. \quad (\text{A.4})$$

This is the model used by `fit_t1_ir`. It is intentionally simple and is only appropriate for the conventional long-TR baseline.

A.3 Finite-TR and Transient IR Models

The RL acquisitions cannot assume long TR or fixed flip angles. For one IR-prepared block, define

$$E_1 = e^{-\text{TI}/T_1}, \quad E_2 = e^{-(\text{TR}-\text{TI})/T_1}. \quad (\text{A.5})$$

With inversion angle θ_{inv} and excitation angle α_{exc} , the longitudinal recurrence is

$$M_z^{\text{TI}} = (1 - E_1) + \cos(\theta_{\text{inv}})E_1 M_z^{\text{pre}}, \quad (\text{A.6})$$

$$M_z^{\text{pre}'} = (1 - E_2) + \cos(\alpha_{\text{exc}})E_2 M_z^{\text{TI}}. \quad (\text{A.7})$$

Solving the fixed point gives `steady_state_mz_at_excite`. Iterating the recurrence for the first `Npe` shots and averaging M_z^{TI} gives `transient_mz_at_excite_npe`. This second model matches the actual 2D IR-SE acquisition used in E2: one phase-encode row is acquired per shot, starting from thermal equilibrium.

The measured transverse signal also contains the excitation projection $\sin(\alpha_{\text{exc}})$. When excitation angles vary across blocks, the caller divides magnitudes by this known factor before fitting; otherwise the fitter would confuse flip-angle scaling with T1 contrast.

A.4 Joint T1/T2 Identifiability

Joint T1/T2 fitting becomes possible only when TE varies. The IR-TSE model is

$$|S| = \left| A M_z^{\text{exc}}(T_1; \text{TI}, \text{TR}, \theta_{\text{inv}}, \alpha_{\text{exc}}) e^{-\text{TE}/T_2} \right|. \quad (\text{A.8})$$

If TE is constant, the exponential T2 factor is absorbed into the fitted amplitude A , so T2 is unidentifiable. This is why `fit_t1_t2_generalized_ir` is reserved for multi-echo schedules and reports a wide T2 uncertainty for constant-TE data.

A.5 Finite K-space Sampling and Truncation Artefacts

A.5.1 Why Reconstruction Blurs and Rings

Chapter 2 established that an MR image is recovered by applying an inverse Fourier transform to sampled k-space. That Fourier relationship is exact only for infinite k-space. In practice, we acquire a finite $N_{pe} \times N_{fe}$ window, so the reconstructed image is not the true spin density $\rho(\mathbf{r})$ but a degraded version of it. The cleanest way to quantify the degradation is to model finite sampling as the full, infinite k-space multiplied by a rectangular box function: equal to 1 inside the sampled window and 0 outside.

The consequence follows from the convolution theorem: multiplication in one domain is convolution in the other. For a Fourier pair, image space $\leftrightarrow$ k-space,

$$\mathcal{F}\{f \cdot g\} = \mathcal{F}\{f\} * \mathcal{F}\{g\}. \quad (\text{A.9})$$

So multiplying the true k-space by a box is equivalent, in the image domain, to convolving the true image with the Fourier transform of that box. That transform is the system's point-spread function

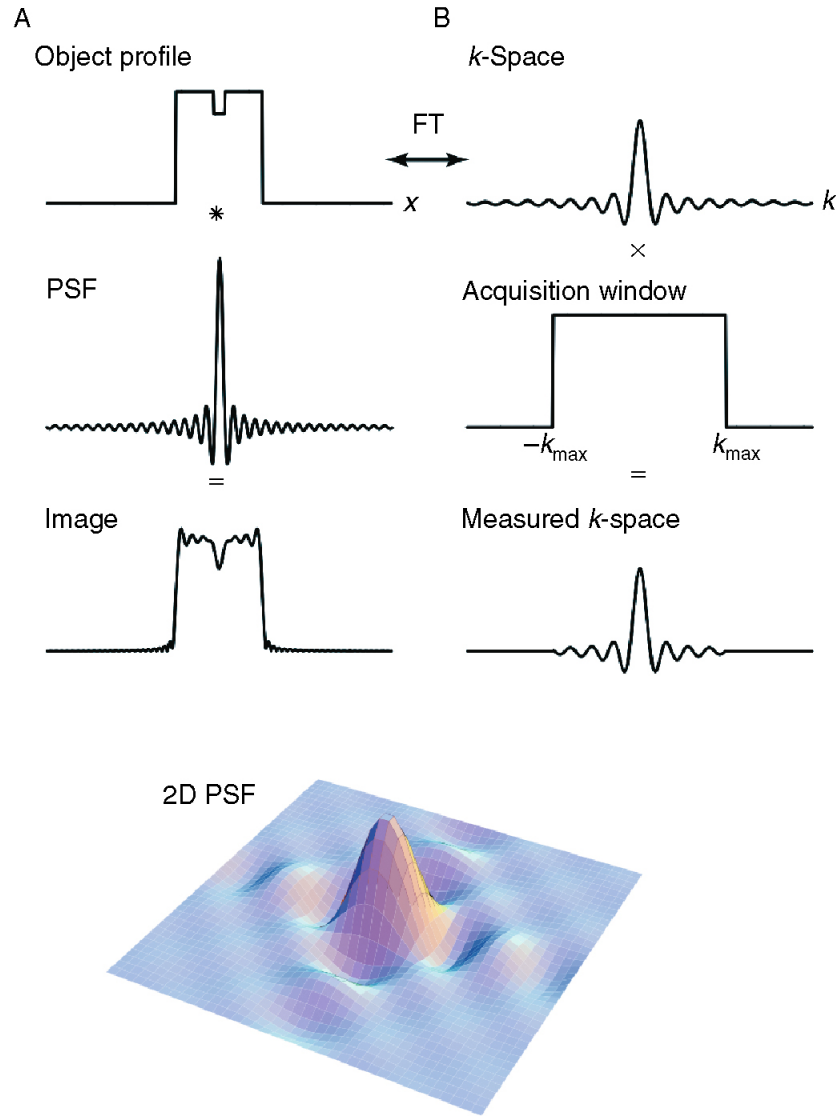


Figure A.1: The point-spread function. Producing a blurred image can be viewed equivalently in the image domain or in k -space. The limited extent of sampling in k -space is described by multiplying the full k -space distribution by a windowing function that removes high spatial frequencies. The resulting image is the convolution of the true image with the Fourier transform of that windowing function, the PSF. Figure adapted from [11].

(PSF): each ideal point in the object is smeared into a small PSF-shaped blob, and where the object has a sharp edge those blobs fail to add up cleanly, producing oscillations: Gibbs ringing.

An intuitive way to read the convolution theorem is that a convolution is “slide, multiply, integrate”. Under a Fourier transform each shift becomes a phase factor e^{-ikx} , the integral collapses the slide, and only a pointwise product survives. This is also why FFT-based convolution is fast: transform, multiply, transform back.

For a rectangular window of N samples the PSF is the Dirichlet kernel, the periodic sinc:

$$\text{PSF}(x) = \frac{\sin(\pi Nx/\text{FOV})}{N \sin(\pi x/\text{FOV})}. \quad (\text{A.10})$$

Reading it as numerator over denominator explains its shape. Let the pixel spacing be $\Delta = \text{FOV}/N$.

- **Unit peak at $x = 0$.** Both numerator and denominator vanish. L’Hopital’s rule gives the ratio $\rightarrow 1$, so the centre pixel passes through at full weight. The kernel is normalised; no rescaling of ρ

Quantity	Scaling with N
Main-lobe width, physical units	FOV/ N , shrinks as $1/N$
Main-lobe width, pixels	1 pixel, always
First side-lobe height	$\approx -21\%$, always
Side-lobe decay	$1/x$, always
Number of side-lobes across the FOV	grows as N

Table A.1: Increasing the sampled matrix size improves physical resolution, but the rectangular-window PSF has the same side-lobe structure in pixel units.

is required.

- **Zeros one pixel apart.** The numerator vanishes when $\pi Nx/\text{FOV} = m\pi$, i.e. at $x = m \text{FOV}/N = m\Delta$. The main lobe is always exactly one pixel wide.
- **Fixed side-lobes.** The first negative side-lobe is approximately -21% of the peak, or about -13 dB, with a $1/x$ decay. This is set purely by the rectangular window shape.

A useful consequence is that, because the zeros sit on integer pixel offsets, an on-grid point source leaks nothing into its neighbours at the sample points. Ringing only becomes visible at a discontinuity, where the contributions on either side of the edge no longer cancel and the side-lobes ring across neighbouring pixels. This is the classic $\approx 9\%$ Gibbs overshoot on each side of a sharp edge.

A.5.2 Increasing the Matrix Does Not Fix It

The natural reflex is to sample more of k-space, i.e. to use a bigger box. However, a larger N only rescales the ringing; it does not remove it.

In pixel units the PSF shape barely changes: a larger N packs the same ringing into a finer physical scale, giving better resolution, and as $N \rightarrow \infty$ the main lobe collapses towards a delta function, recovering the ideal image. But the -21% side-lobe is intrinsic to the rectangular window. To actually suppress ringing, one must change the window shape, not its size. This is apodisation.

A.5.3 Apodisation: The Hamming Window

If a hard rectangular cut is what produces the strong side-lobes, then softening the edge of the k-space window should reduce them. This is apodisation: weighting k-space by a function that tapers smoothly to zero at the edges rather than truncating abruptly. Many window functions exist; a standard choice is the Hamming window,

$$w(n) = 0.54 - 0.46 \cos\left(\frac{2\pi n}{N-1}\right), \quad n = 0, \dots, N-1. \quad (\text{A.11})$$

which retains full weight near the centre, DC, and tapers to 0.08 at the k-space edges. Because the high spatial frequencies live at those edges, down-weighting them trades resolution for smoothness: the Fourier transform of the Hamming window has a main lobe roughly twice as wide, giving a blurrier image, but side-lobes suppressed from -13 dB to -43 dB, giving far less ringing.

The effect on an actual edge is the case that matters for the phantom, whose water–sphere boundaries are the sharpest features in the scene. The box PSF’s side-lobes fail to cancel across the step and produce the $\approx 9\%$ Gibbs overshoot; the Hamming PSF’s far weaker side-lobes suppress it, at the price of a softer, wider edge transition.

Two practical remarks close the loop. First, apodisation is a reconstruction-side operation: it is an element-wise multiply applied to the acquired k-space before the inverse FFT, costing a fraction of a millisecond and entirely independent of the expensive Bloch simulation. It can therefore be toggled freely

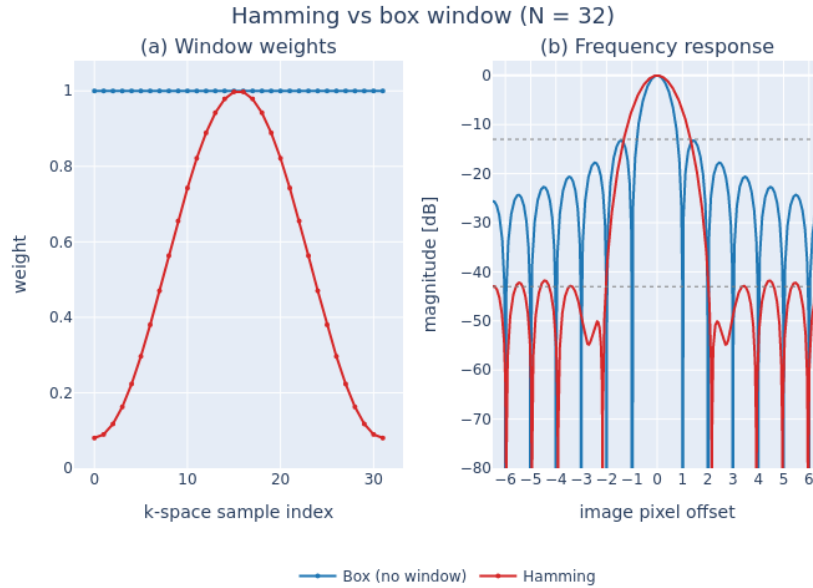


Figure A.2: **Hamming apodisation: window weights and their point-spread response.** (a) K-space weighting applied before reconstruction for $N = 32$ phase-encode lines. The box window, i.e. no apodisation, weights every line equally at 1.0; the Hamming window $0.54 - 0.46 \cos(2\pi n/(N - 1))$ tapers smoothly to 0.08 at the k-space edges, retaining full weight only near the centre. (b) Corresponding point-spread functions, shown as magnitude in decibels normalised to the main-lobe peak. The box window’s abrupt truncation produces -13 dB side-lobes that ring across neighbouring pixels; the Hamming taper suppresses these to -43 dB at the cost of an approximately $2\times$ wider main lobe, i.e. reduced spatial resolution.

without re-acquiring or re-simulating. Second, it must not be confused with zero-padding. Padding k-space with zeros before the FFT interpolates the image onto a finer grid but adds no new measured frequencies, so it does not remove ringing. It only resamples the same Dirichlet PSF more densely. Genuine suppression requires changing the window shape, which is exactly what the phantom library’s Hamming option does, and the mechanism here is what underpins its use against the coarse-water truncation artefact in Chapter 3.

A.6 E2 Noise Calibration

The E2 noise model adds independent complex Gaussian noise of absolute standard deviation σ to every k-space sample. Because the noise-free signal scale is a linear sum over simulated spins, the mapping from σ to image SNR depends on the spin count, and therefore on voxel size, slice thickness, matrix size and whether background water is included. A σ calibrated at one geometry does not transfer to another: for example, slicing a 1 mm slab through the T1 plate keeps 4,816 of the ~ 27 k full-3D sphere spins, shrinking the k-space signal scale by a factor of ~ 5.6 . The operating σ was therefore derived by a sweep at exactly the training configuration (T15 field, 1 mm slab, 1 mm voxels, 64×32): for each σ , 25 noise seeds were simulated under the Cramer–Rao-optimal fixed schedule and the resulting T_1 -fit MAPE and three SNR conventions were recorded.

Of the SNR conventions, the NEMA MS-1 dual-acquisition measurement (signal from the mean of two acquisitions, noise from their difference image) is the one comparable with clinical phantom QA; the single-image NEMA variant reads systematically low on coarse matrices because Gibbs ringing inflates the ROI variance. Figure A.4 summarises the sweep. At $\sigma = 50$ the dual-acquisition SNR is ≈ 27 on the brightest spheres (the peak convention plotted on the figure’s top axis) and ≈ 17 averaged across the T1 plate at the reference block (`scripts/e2_snr_report.jl`). Both bracket the in-vivo brain IR range of roughly 16–30 at 1.5 T, and the fixed-schedule fit error at $\sigma = 50$ is still close to its noise-free floor. Fit quality degrades gracefully up to $\sigma \approx 85$ and collapses beyond, so $\sigma = 50$ stresses the agent with

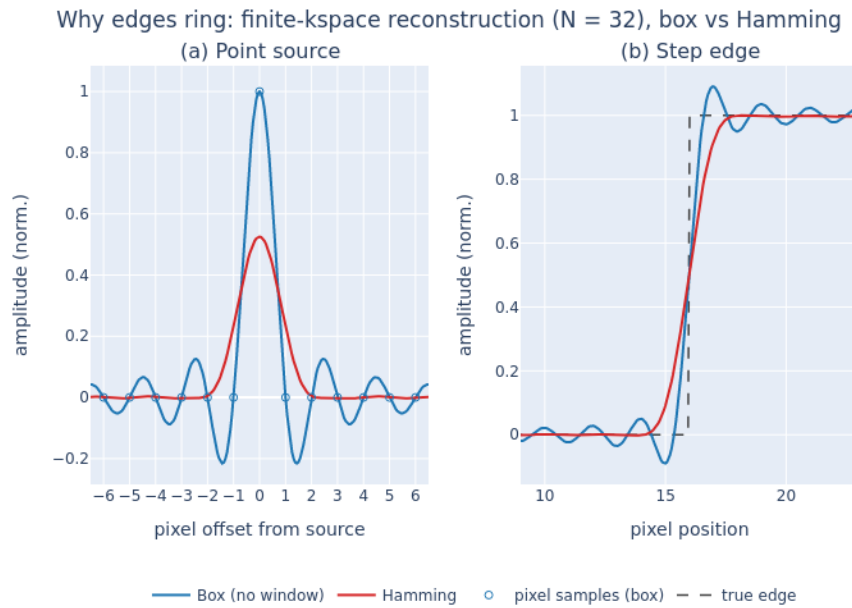


Figure A.3: **Why finite k-space sampling rings, and how Hamming apodisation suppresses it.** One-dimensional reconstruction from a finite acquisition of $N = 32$ phase-encode lines, comparing no window, or box truncation, with a Hamming window. **(a)** Point-source response: the reconstruction PSF. The box truncation yields a Dirichlet, sinc-like PSF whose zero-crossings fall exactly on integer pixel offsets. An on-grid point leaks nothing into neighbouring pixels at the sample points. The Hamming PSF has an approximately $2\times$ wider main lobe, nonzero at ± 1 px, so it blurs adjacent pixels, and a lower peak reflecting the window's DC attenuation. **(b)** The same two PSFs convolved with a step edge. The box PSF's -13 dB side-lobes fail to cancel across the discontinuity, producing Gibbs ringing with $\approx 9\%$ overshoot on both sides. The Hamming window's -43 dB side-lobes suppress this ringing, at the cost of a softer, wider edge transition. Equivalently, panel (a) is the PSF and panel (b) is that PSF convolved with an edge: the point-spread function explains the edge artefact.

clinically realistic noise without making the task estimation-limited.

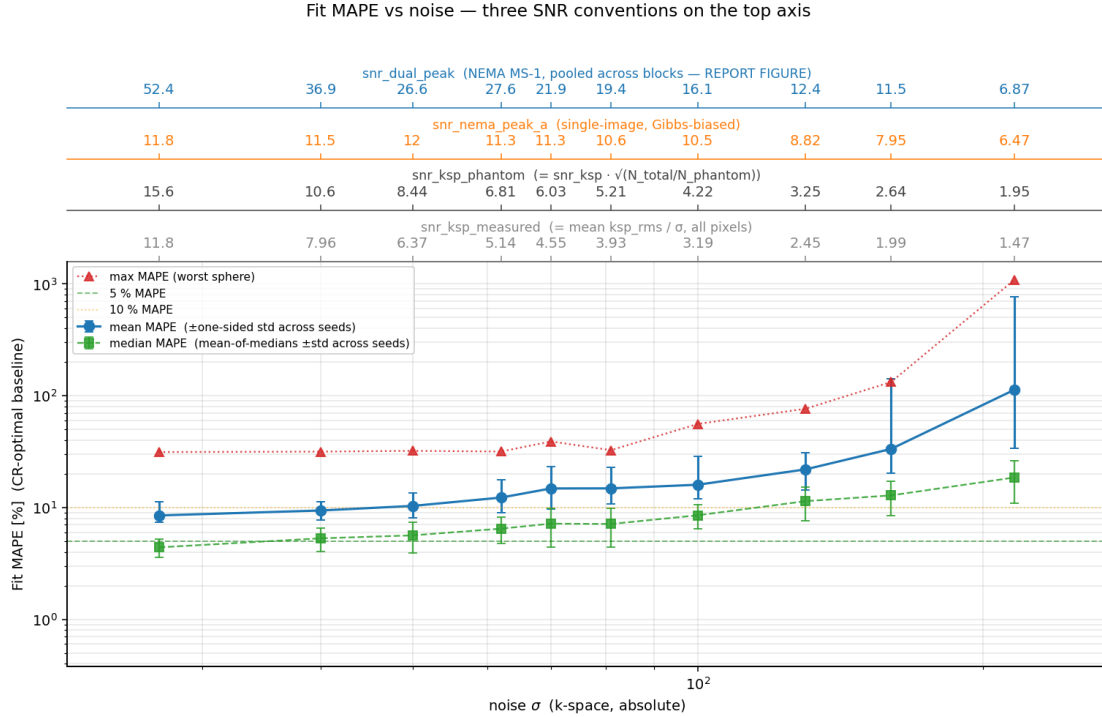


Figure A.4: **Noise calibration sweep at the E2 training geometry** (25 seeds per σ , Cramer–Rao-optimal fixed schedule). Mean, median and worst-sphere T_1 -fit MAPE versus absolute k-space σ , with the corresponding SNR conventions on the top axes. The NEMA MS-1 dual-acquisition row (top, blue) is the clinically comparable measurement; the operating point $\sigma = 50$ corresponds to peak dual-acquisition SNR ≈ 27 . The worst-sphere curve is dominated by the longest- T_1 sphere, which the fixed schedule cannot constrain even at $\sigma = 0$.

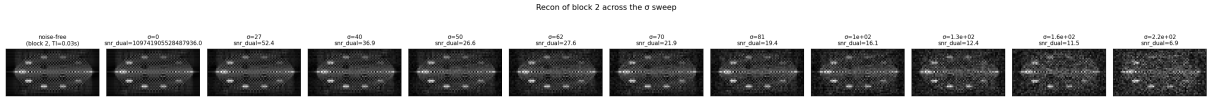


Figure A.5: **Visual artefact level across the noise sweep.** Reconstructed magnitude images of one acquisition block (TI = 0.03 s) as σ increases left to right, annotated with the measured dual-acquisition SNR. At the operating point $\sigma = 50$ (peak SNR ≈ 27) the spheres remain clearly resolved over a visibly noisy background; by $\sigma \geq 160$ the background noise approaches the sphere contrast and fits collapse.

A.7 Cached Background-Water Validation

This appendix supports Section 5.4.2: it shows why the background water must be Bloch-accurate but does not need to be re-simulated every step, and quantifies the accuracy of the cached-template model used by the **cached** fidelity rungs.

Figure A.6 compares reconstructions of the same acquisition under four water models. Two cached variants are shown: the *scalar* variant rescales the whole template by a single shot amplitude, while the *per-line* variant rescales each k-line by its own shot amplitude; E2 uses the per-line variant, which is the exact factorisation for a multi-shot acquisition. Both cached variants are visually indistinguishable from the full simulation (their signed difference images are blank at this colour scale), whereas analytic water leaves structured residuals across both the spheres and the background. Figure A.7 quantifies the consequence for the quantity that actually matters, the fitted T_1 : the cached models stay at or below the $\sim 1\%$ fitter grid floor on every sphere, while analytic water produces 4–40% per-sphere deviations and even the fully analytic forward model 2–16%. An analytic-water rung would therefore corrupt the reward signal; the cached rung does not.

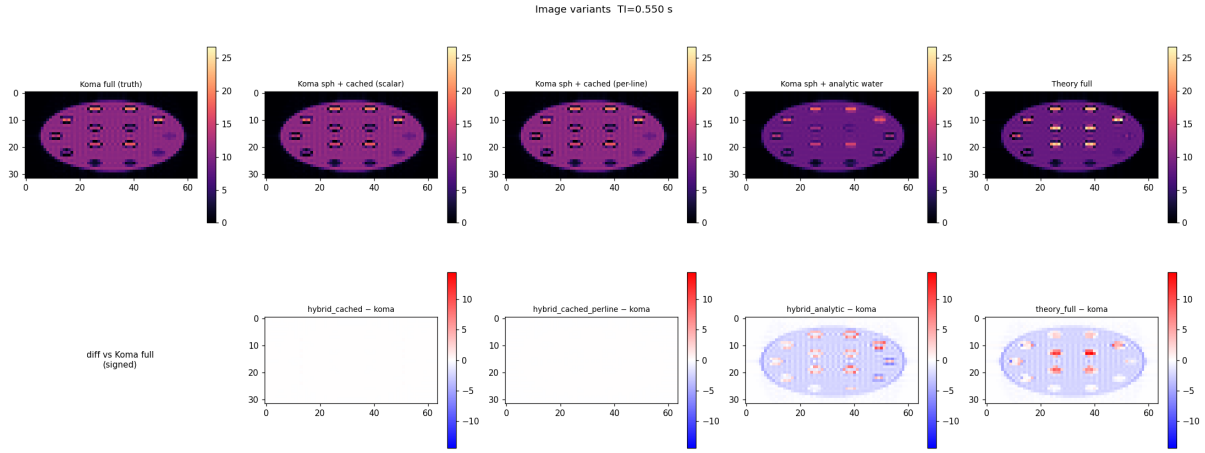


Figure A.6: **Water-model comparison at matched sequence settings** ($T_I = 0.55$ s, $\alpha = 30^\circ$, 64×32). Top row: reconstructed magnitude images for the full Bloch simulation (truth), Bloch spheres with cached water (scalar and per-line variants), Bloch spheres with analytic water, and the fully analytic forward model. Bottom row: signed difference against the full simulation. The cached variants are exact at this colour scale; the analytic water model leaves structured sphere and background residuals that no T_I rescaling can remove.

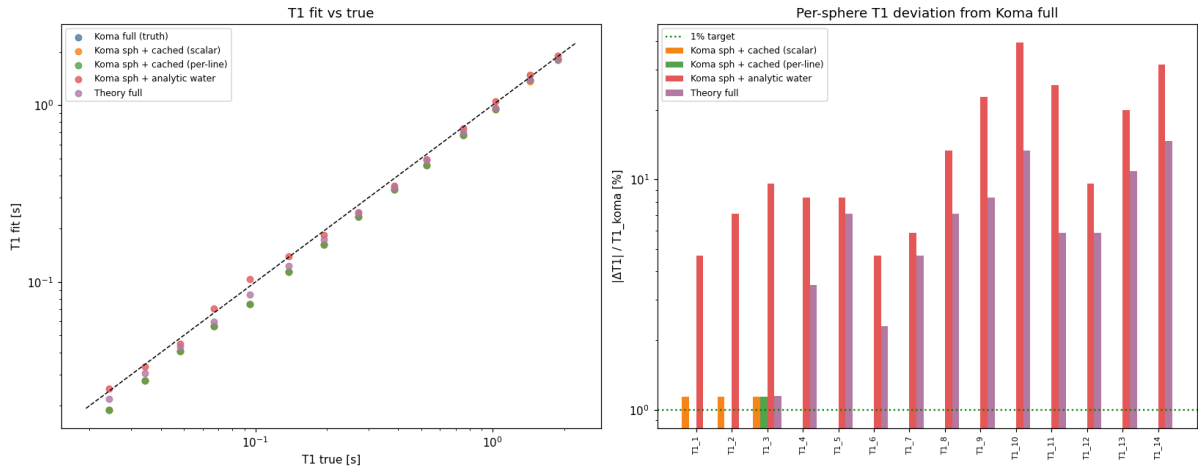


Figure A.7: **Per-sphere T_1 -fit impact of the water model**. Left: fitted versus true T_1 for each variant. Right: per-sphere deviation from the full-Bloch fit. The cached variants sit at or below the $\sim 1\%$ fitter grid floor on every sphere (most bars are below the axis); analytic water deviates by 4–40%, worst at short T_1 , and the fully analytic model by 2–16%.

Figure A.8 validates the α -template bank of Section 5.4.2 (36 templates spanning the action range) across TI and flip angle. The cached water k-space matches the Bloch water to 10^{-3} – 10^{-2} relative error over the operating TI range, with two readable features: the error collapses to $\sim 10^{-8}$ at the template’s reference TI (where the factorisation is evaluated at its construction point), and it rises towards TI ≈ 1.6 – 2 s, where the water signal approaches its own inversion null and the relative-error denominator vanishes. For context, the right panel shows that modelling the water without B_0 inhomogeneity changes the water k-space by 10–40%: the cache approximation error is one to two orders of magnitude below a routine modelling choice, so it is not the binding fidelity constraint. End-to-end, cached-water T_1 fits match the full simulation to 0.12% on average (worst sphere 1.15%, the fitter’s discrete-grid floor), and the per-step speedup is $8.0\times$ (4.47 s $\rightarrow$ 0.56 s at the benchmark settings) for a one-off bank build cost of 153 s.

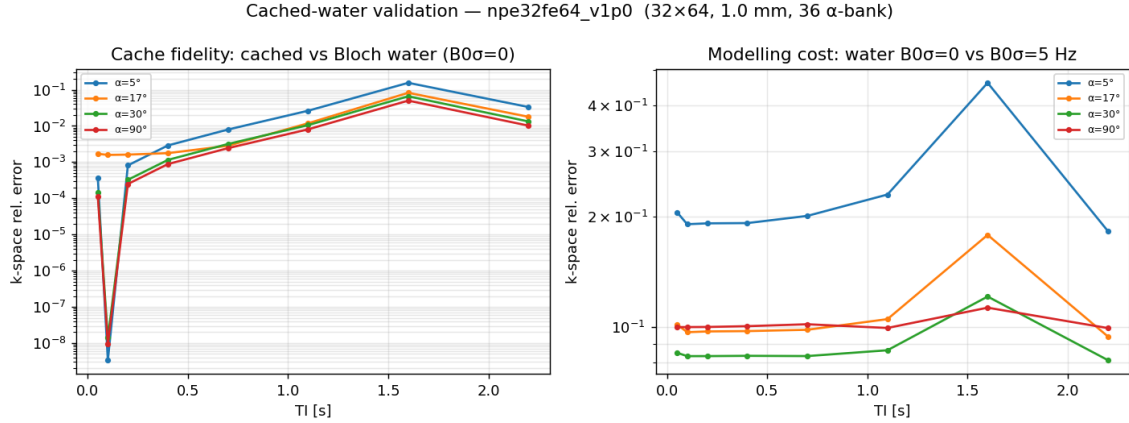


Figure A.8: **α -bank cache fidelity across TI and flip angle** (36-template bank, 64×32 , 1 mm). Left: relative k-space error of cached versus Bloch water at $\alpha \in \{5^\circ, 17^\circ, 30^\circ, 90^\circ\}$; the dip is the template’s reference TI and the rise towards TI ≈ 1.6 – 2 s reflects the water signal approaching its inversion null. Right: the error made by modelling water without B_0 inhomogeneity ($B_0\sigma = 0$ versus 5 Hz), one to two orders of magnitude larger than the cache error at matched settings.

A.8 Reinforcement Learning Background

This section gives the policy-gradient, value-function, and PPO derivations summarised in the reinforcement-learning discussion of Chapter 2. The notation follows the main text: a policy $\pi_\theta(a \mid s)$, return $G_t = \sum_{k \geq 0} \gamma^k r_{t+k}$, and objective $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G_0]$.

A.8.1 Policy gradients

Policy-gradient methods optimise $J(\theta)$ directly.[41], [42] The probability of a trajectory under a policy is

$$p_\theta(\tau) = \rho_0(s_0) \prod_{t=0}^{T-1} P(s_{t+1} \mid s_t, a_t) \pi_\theta(a_t \mid s_t). \quad (\text{A.12})$$

The simulator dynamics P do not depend on the policy parameters. Applying the log-derivative trick gives the basic policy-gradient expression:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t) G_t \right]. \quad (\text{A.13})$$

In practice this expectation is approximated using sampled rollouts:

$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} \nabla_{\theta} \log \pi_{\theta}(a_{i,t} \mid s_{i,t}) \hat{A}_{i,t}. \quad (\text{A.14})$$

This is the central sample-based approximation of policy-gradient RL: the expectation over all trajectories is estimated from a finite batch collected from the simulator, making actions with better-than-expected outcomes more likely and worse-than-expected actions less likely. Crucially, it does not require differentiating the simulator. The gradient flows only through the policy’s log-probability $\log \pi_{\theta}(a_t \mid s_t)$, which is differentiable because the policy is a neural network, so KomaMRI can stay a black-box physics simulator used only to sample transitions and rewards. The weighting term is written as an advantage estimate $\hat{A}_t$ rather than the raw return G_t , which lowers variance by measuring whether an action beat what the policy already expected in that state.

A.8.2 Value and advantage functions

The value function of a policy is the expected future return from a state:

$$V^{\pi}(s) = \mathbb{E}_{\tau \sim \pi}[G_t \mid s_t = s]. \quad (\text{A.15})$$

The action-value function is the expected return after taking a particular action and then continuing with the policy:

$$Q^{\pi}(s, a) = \mathbb{E}_{\tau \sim \pi}[G_t \mid s_t = s, a_t = a]. \quad (\text{A.16})$$

Equivalently, it can be written in one-step form as the expected immediate reward plus the value of the next state:

$$Q^{\pi}(s_t, a_t) = \mathbb{E}[r_t + \gamma V^{\pi}(s_{t+1}) \mid s_t, a_t]. \quad (\text{A.17})$$

The advantage function compares these two quantities:

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s). \quad (\text{A.18})$$

An action with positive advantage performed better than the policy’s average action at that state; an action with negative advantage performed worse. This is the quantity used to decide which sampled actions should become more or less probable.

In deep RL we usually do not know V^{π} , Q^{π} , or A^{π} exactly. PPO therefore learns two approximations at the same time: an actor, $\pi_{\theta}(a \mid s)$, which chooses acquisition actions, and a critic, $V_{\phi}(s)$, which predicts the expected return from the current state. The critic provides a baseline for the policy-gradient update. A simple one-step temporal-difference residual is formed from an actual sampled transition: the actor samples $a_t \sim \pi_{\theta}(\cdot \mid s_t)$, and the simulator returns the immediate reward r_t and the next state s_{t+1} , giving

$$\delta_t = r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t). \quad (\text{A.19})$$

Here $V_{\phi}(s_t)$ is the critic’s prediction before taking the action, while $r_t + \gamma V_{\phi}(s_{t+1})$ is the one-step target after seeing the simulator’s response. Generalised Advantage Estimation (GAE) combines these residuals over multiple future steps:[27]

$$\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma \lambda)^l \delta_{t+l}, \quad (\text{A.20})$$

where λ controls the bias-variance trade-off. GAE interpolates between two estimates of the same advantage $A^\pi(s_t, a_t)$: the full Monte Carlo return ($\lambda \rightarrow 1$) is unbiased but high-variance because it leans on one noisy trajectory, while the one-step residual ($\lambda \rightarrow 0$) leans on the learned critic, cutting variance at the cost of bias when the critic is inaccurate.

A.8.3 From TRPO to the PPO objective

Vanilla policy-gradient updates can be unstable because a single gradient step can move the policy too far. If the new policy differs greatly from the policy that generated the rollout batch, the sample estimate may no longer describe the behaviour of the updated policy. Trust Region Policy Optimisation (TRPO) addresses this by constraining the KL divergence between the old and new policies, but it requires a more complex second-order optimisation procedure.[43]

The underlying update is an update to the neural-network weights θ . Even if the parameter change looks small in weight space, it can cause a much larger change in the induced probability distribution over actions, $\pi_\theta(\cdot | s)$. Since PPO is on-policy, the rollout batch was collected using the old policy. If the update makes the new policy assign very different action probabilities, the batch becomes stale: it reflects what was useful under the old action distribution, not necessarily what is useful under the new one. Large updates can therefore overreact to lucky sampled actions or collapse exploration.

Proximal Policy Optimisation (PPO) keeps the same basic idea – improve the policy while preventing large destructive updates – but uses a simpler clipped objective.[26] For samples collected with the old policy $\pi_{\theta_{\text{old}}}$, define the likelihood ratio

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}. \quad (\text{A.21})$$

If $r_t(\theta) > 1$, the new policy assigns higher probability to the sampled action than the old policy did; if $r_t(\theta) < 1$, it assigns lower probability. PPO maximises

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]. \quad (\text{A.22})$$

For positive-advantage actions, the objective encourages increasing their probability, but only up to the clipping threshold. For negative-advantage actions, it encourages decreasing their probability, again only up to the threshold. The clipping term therefore removes the incentive for the policy to move far away from the sampled policy in a single update. The full optimisation also trains the value function and usually includes an entropy bonus to preserve exploration:

$$L(\theta, \phi) = -L^{\text{CLIP}}(\theta) + c_v \mathbb{E}_t \left[\left(V_\phi(s_t) - \hat{G}_t \right)^2 \right] - c_H \mathbb{E}_t [\mathcal{H}(\pi_\theta(\cdot | s_t))]. \quad (\text{A.23})$$

Here c_v and c_H weight the critic loss and entropy bonus. The practical loop is to collect rollouts with the current policy, estimate returns and advantages, update the actor and critic for a few minibatch epochs, then discard the rollouts and collect fresh ones. This makes PPO *on-policy*: unlike off-policy methods such as DQN or SAC, which keep a *replay buffer* of past transitions and reuse them many times, PPO trains only on data from the current policy and throws it away after each update. That costs sample efficiency but avoids the instability of learning from a stale action distribution, and it removes the replay buffer’s memory overhead.